

claude-windows / 902e27a6-e026-4b55-8c26-f2c47329cc44

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\902e27a6-e026-4b55-8c26-f2c47329cc44.jsonl`  
- SHA-256: `e427207caa3bcf9a5bec20625680ac232f4abe815809580fca66d1dbcfc2c1df`  
- Source modified: `2026-07-07T17:39:14+00:00`  
- Imported at: `2026-07-08T15:59:14+00:00`  
- project: `C--Users-avidu-Projects-badminton-highlight-indexer`  
- session_id: `902e27a6-e026-4b55-8c26-f2c47329cc44`
```

Transcript

1. user (2026-07-07T14:24:37.082Z)

On the cloud-serving path, a video that PASSES the control-plane's validation gets REJECTED by the worker's redundant re-validation because the control-plane's validation config is not forwarded to the worker.

Observed live 2026-07-07 (CUJ, "Video 1 - Badminton.MP4", a 5312x2988 10-bit HEVC clip, sharpness 11.9): control-plane baked config has validation.min_blur_threshold=8.0 ? it PASSED there and dispatched to the Cloud Run worker Job (rally-worker). The worker (backend/worker/__main__.py cmd_infer, ~line 262: `validator\_registry.run\_all\_with\_context(...)`) RE-RAN validation using the worker's OWN config ? which has the DEFAULT min_blur_threshold=20.0 (the control-plane's 8.0 is baked into the control-plane's config.json and is NOT forwarded on dispatch; backend/orchestration.py forwards only skip_ai_handoff + the wasb.* knobs from #509, no validation.* keys). Result: `[worker] validation FAILED: blur\_check: Average sharpness 11.9 < minimum 20.0`, worker exit rc=2, and `[COMPUTE] requested=cloud actual=cloud\_run dispatched=True status=failed` ? the CUJ failed AFTER a successful GPU dispatch.

This is redundant (the control-plane already validated + passed before dispatching) AND incorrect (different thresholds ? contradictory decisions), and it also wastes worker time re-decoding the video for validation.

Preferred fix: on the dispatch path, have the worker SKIP re-validation (the control-plane is the authoritative gate) ? e.g. the dispatcher passes a flag / `--set` that cmd_infer honors to skip validator_registry.run_all, since the control-plane already ran it and only dispatches on pass. Alternative/complementary: forward the control-plane's resolved validation config (at least validation.min_blur_threshold, ideally the whole validation section) to the worker as `--set` overrides (mirror the #509 wasb-knob forwarding in orchestration.py::_maybe_dispatch_remote) so a worker re-validation matches the control-plane's decision. Prefer skip-on-dispatch (also saves the redundant decode); keep worker-side validation for the direct `worker infer` CLI path (no control-plane).

Acceptance: a video that passes the control-plane gate is NOT rejected by the worker on the cloud path; add a test in tests/test_api_cloud_dispatch.py (or worker tests) asserting the dispatch either skips worker validation or forwards validation.min_blur_threshold. Repo pre-merge gate (full suite + ruff + mypy + coverage), branch from origin/master. Related: #509 (wasb-knob forwarding), task 'Guard compute=local on the hosted control-plane'.

2. assistant (2026-07-07T14:24:45.768Z)

I'll start by orienting myself. Let me read my memory files to find the actual repo location and current state.

3. user (2026-07-07T14:24:46.835Z)

```
1      ---  
2      name: actual-repo-location
```

```

3      description: "The badminton-highlight-indexer code lives under khelsutra-guru/, not the
primary working dir stub"
4      metadata:
5          node_type: memory
6          type: project
7          originSessionId: df5d5fb2-534a-447a-8181-b222f94655f3
8      ---
9
10     The primary working directory `C:\Users\avidu\Projects\badminton-highlight-indexer` is a
near-empty
11     **stub** ? it contains only `AGENTS.md`, an empty `.claude/`, and
`output/nightly_regression/`
12     (a `LATEST_STATUS.txt`). The actual `khelsutra-guru/badminton-highlight-indexer`
codebase lives at:
13
14     `C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer`
15
16     (git remote `github.com/Khelsutra/badminton-highlight-indexer`, default branch
`master`).
17
18     **How to apply:** when a task references this repo's code (backend/worker,
pipeline/segmenters,
19     config.json, run_all_tests.py, etc.), `cd` to the khelsutra-guru path and work there ?
the stub dir
20     has none of it. Confirmed 2026-07-06 while fixing the wf294 silent-detector-failure
issue.
21
22     Repo conventions verified there: CI = ruff 0.15.16 (`ruff check .`), mypy 2.1.0 (`mypy
backend
23     training`; `mypy.ini` quarantines several modules incl. `backend.worker.__main__`,
`gemini`,
24     `yolo_hybrid` via `ignore_errors=True`), full suite via `python run_all_tests.py`,
coverage floor
25     `--cov-fail-under=70` (actual ~86%). Segmenters use a `Success`/`Failure` Result
contract
26     ([[efficient-timeouts-and-retries]] is unrelated).
27

```

4. user (2026-07-07T14:24:47.823Z)

```

1      ---
2      name: session-handoff-gpu-resident
3      description: "Start here to pick up the GPU-resident WASB rollout (issue #506) ?
implementation PR #507 is open; golden gate next"
4      metadata:
5          type: project
6      ---
7
8      # Session Handoff ? GPU-resident WASB inference (rollout) . issue #506 . PR #507
MERGED
9
10     ## SHIPPED 2026-07-07 ? GPU-resident NVDEC decode is LIVE in cloud-serving
11     Full arc done: PR #507 (feature) + PR #508 (cloud-serving preset default =
nvdec_resident) both merged to master (44a2f47, 143dbb9). The `rally-worker` Cloud Run Job
(asia-southeast1) runs the new image `rally-worker:latest` `sha256:465a9250` (torch 2.6 +
PyNvVideoCodec 2.1.0 + baked NVDEC libs) with `WASB_DECODE_BACKEND=nvdec_resident` set ? **nvdec
is the live serving default**, validated on the real L4 (GX010128 served trajectory reproduced
the VM: 26479?26481 visible; cpu+nvdec+canary smokes all ?). Record:
`docs/COMPUTE_DECOUPLED_SERVING/runlogs/DEPLOY_REPORT_cloud-serving_gpu-resident-
nvdec_2026-07-07.md`.
12     - **ROLLBACK levers:** `gcloud run jobs update rally-worker --region asia-southeast1
--remove-env-vars WASB_DECODE_BACKEND` (? torch-2.6 cpu path, same image); or `--image ?/rally-
worker@sha256:3bfefaf6` (? prior torch-1.11 image).
13     - **PRs:** #507 (feature) + #508 (preset default) + #509 (dispatch forwarding + release
hardening) all merged. master @ 66756ad.

```

14 - ****DONE ? control-plane + worker both on image D2 `sha256:09d2edf7` (built from master 66756ad = #507+#508+#509).** **** Redeployed 2026-07-07: `gcloud run services update rally-control-plane --image ? :latest` ? rev ****00017-qbd**** (100% traffic, booted healthy: uvicorn up, cloud-serving profile ACTIVE, edge-auth on); `gcloud run jobs update rally-worker --image ? :latest`. So the #509 dispatch ****forwarding**** + #508 preset default are now LIVE in the control-plane image. NB the FIRST control-plane deploy this session used the stale `465a9250` (built from 44a2f47, pre-#508/#509) ? rev 00016; superseded by 00017. Rollback: `gcloud run services update-traffic rally-control-plane --to-revisions rally-control-plane-00015-kp9=100` (last pre-nvdec rev) or `--to-revisions ?-00016-?` .**

15 - ****WASB_DECODE_BACKEND=nvdec_resident` env is still set on the worker Job**** ? now REDUNDANT with the forwarded --set + preset (both say nvdec, env just wins), kept as belt-and-suspenders + manual override. Safe to remove ONLY after a real /api/process dispatch confirms the forwarding path live (needs a CF Access JWT ? not verifiable this session; forwarding is deployed + unit-tested but not live-dispatch-verified).

16 - `DEPLOY\_REPORT?gpu-resident-nvdec\_2026-07-07.md` is current: PR #510 added the \$5 "Update" (D2 redeploy of control-plane rev 00017 + worker) and corrected \$6. All four PRs merged: #507 #508 #509 #510; master @ 3be0935.

17 - ****Wrinkles fixed (#509):**** dispatch now forwards indexing.wasb.{decode_backend,batch_size,prefetch_batches} (worker has no DEPLOYMENT_PROFILE ? never applied the preset; batch tuning was silently lost too); added `.gcloudignore` (Cloud Build had none ? relied on the .gitignore fallback to skip 8 GB output/scratch); added `deploy/cloudrun/post\_deploy\_smoke.ps1`. See [[gpu-vm-setup-gotchas]].

18 - WASB-C3 weights-license gate still open (unchanged; pre-existing).

19

20 **## (historical) release/rollout phase ? superseded by SHIPPED above**

21 PR #507 squash-merged to master as ****44a2f47**** (branch deleted). Pre-merge gate reproduced locally: full suite 1764 passed / coverage 85.02% / ruff+mypy clean; CI 11/11 green; 2 optional review follow-ups landed (PyNvVideoCodec==2.1.0 pin in both deploy files; decode_backend persisted in native-runner detection_params). Real-L4 validation + golden gate PASS + latency/resource A/B all done (see PR #507 comment + below). ****Remaining = the serving release (image rebuild + gated nvdec flip); decode_backend still defaults cpu so nothing in prod has changed yet.****

22

23 **### Serving release plan (live infra: Job `rally-worker` + service `rally-control-plane`, asia-southeast1; ONE image `rally-worker:latest` drives both ? gen_service_yaml.py:193)**

24 - ****Phase A (ship image, no behavior change):**** `gcloud builds submit --config=deploy/cloudrun/cloudbuild.yaml --project gen-lang-client-0306026826 .` ? new `rally-worker:latest` (torch2.6+pyncv2.1.0+NVDEC libs). Push does NOT change prod (Job/service pin a digest). Cut over via `gcloud run jobs update rally-worker --region asia-southeast1 --image ? :latest` + redeploy service (`gen\_service\_yaml.py` ? `services replace`). decode_backend defaults cpu ? cpu path under torch2.6 (validated F1 0.576?0.582). Smoke 1 clip. Rollback: `jobs update --image <old digest>`.

25 - ****Phase B (enable nvdec, gated + reversible):**** flip via WORKER env `WASB\_DECODE\_BACKEND=nvdec\_resident` (native runner reads env first ? no code/preset change, instant rollback by removing it). ? THE untested risk: NVDEC on the real Cloud Run L4 ? the image bakes libnvcuvid 580.159.03; Cloud Run's host-driver ABI is unverified there (only tested on a self-managed L4 VM). MUST smoke 1 clip on the Job before trusting it. Then canary; then durable default = add `decode\_backend: nvdec\_resident` to the cloud-serving preset (presets.py) + rebuild + Launch Report.

26 - Cost: build ~\$0.3; each Job smoke ~\$0.4?0.7 (L4). Rollback levers at every step.

27

28

29 **## You are here**

30 The implementation is DONE and ****PR #507 is open**** (branch `feat/506-gpu-resident-nvdec`, 2026-07-07): default-OFF `indexing.wasb.decode\_backend = "cpu" | "nvdec\_resident"`, the validated NVDEC?affine-`grid\_sample`?GPU-normalize path wired into `backend/pipeline/detectors/wasb\_infer.py` (`NvdecResidentFrontend` + `run\_video\_nvdec\_resident`, same resumable cache keyed by decode_backend), native-runner threading + healthcheck gates (CUDA + PyNvVideoCodec), and the env bump (wasb env ? py3.10 + torch 2.6.0+cu124 + PyNvVideoCodec; libnvcuvid/libnvidia-encode staged in `deploy/cloudrun/Dockerfile` + `deploy/digitalocean/gpu\_setup.sh`). Verified locally: 1764 tests green, coverage 85% (floor 70), ruff/mypy/link/leak clean; a 14-agent adversarial review confirmed 7 minor findings, all fixed (notably `padding\_mode="zeros"` = cv2 BORDER_CONSTANT for non-16:9 sources). Research evidence unchanged: [[gpu-resident-modernization]], issue #506 comments (F1 0.574 ? baseline 0.582 on GX010128 at SERVED, ~1.7x, GPU 47?79%).

```

31
32     ## Real-L4 validation ? DONE + golden gate PASS 2026-07-07 (this session, ~$3 of the
$100 budget; VM torn down)
33     Validated end-to-end on an L4 (`rally-506`, torn down clean, no orphan disks) ? full
evidence in the PR #507 comment. All GREEN: (1) `gpu_setup.sh` builds the env clean (py3.10 +
torch 2.6.0+cu124 cuda True + PyNvVideoCodec 2.1.0 + NVDEC lib extraction); (2) on-box **parity
gate PASS** (`pytest -k parity`, 480s ? needed test-name fix 9267255, the documented `-k parity`
selected 0 tests); (3) crash/resume byte-identical; (4) cpu?nvdec cache isolation correct; (5)
**Cloud Run image builds + `--gpus all` smoke PASS** (baked NVDEC libs decode; native
healthcheck ok with nvdec_resident); throughput 43.7 fps / GPU 82% on GX010128.
34     **Golden gate @ SERVED = PASS** (10 baseline-comparable clips, 445 rallies): mean
?f1@0.5 **+0.020**, mean ?tolF1 +0.006, worst ?0.009 (**GX010128 must-pass: 0.574 vs 0.582 ?**),
8/10 improved-or-flat. The other 5 golden clips are 4K-sourced (baselines are 1080p) so a vs-
baseline number would conflate resolution with decode ? the parity gate already covers nvdec?cpu
on identical pixels, so they add nothing to the verdict.
35     **Latency + resource A/B** (2nd L4 `rally-506b`, torn down; GX010128, batch 8, telemetry
in `gs://?nightly/dev/506/results/bench/`): nvdec **43.7 fps @ GPU 83% / CPU 30%** vs cpu-
stream(no-prefetch) 15.0 fps @ GPU 28% / CPU 49%; GPU mem +170 MiB (of 24GB), RAM ~2?3 GB flat.
The win is the **bottleneck flip (GPU?, CPU?)**; the 2.9x is vs the untuned streaming path ? vs
prefetch-tuned cpu the realistic end-to-end is ~1.6?1.8x (#506 spike). All in the PR #507
comment.
36
37     ## Staged assets (durable)
38     - **Golden video working set**: all 14 originals in `gs://khelsutra-rally-
corpus/videos/golden/<name>.mp4` + the 4 baseline-source 1080p proxies in `videos/golden_1080p/`
(mahadevpura_2, Badminton_BXH_2, Boxhill_Doubles, GX020094). Frame-count-verified against each
golden trajectory. (GOLDEN_VIDEOS.md could record this GCS location in a follow-up ? kept out of
PR #507 to not muddy its scope.)
39     - Gate trajectories: `gs://khelsutra-rally-nightly/dev/506/results/golden/*_nvdec.csv`.
Re-score with repo-root `score_golden_gate.py` (untracked dev tool, like `score_local.py`;
labels from `?corpus/labels/`, baselines from `?corpus/trajectories/`).
40
41     ## Next steps
42     1. **Merge PR #507** ? real-L4-validated + golden-gate PASS; CI green locally.
43     2. **Flip the default**: owner Launch Report ? set `decode_backend=nvdec_resident` in
the cloud-serving preset (`backend/config/presets.py`). The gate that gated this is now green.
44
45     ## Ramp-up kit
46     - PR #507 (full change description + verification record) and issue #506 (all comments ?
measured evidence).
47     - New tests: `tests/test_wasb_nvdec_resident.py` (grid/cv2 parity incl. non-16:9, cache
keying, CLI guards, the skippable on-box parity gate).
48     - Reference code still in `scratch/gpu_resident/` (`exp_t2_full.py`, `score_local.py`,
`t2setup_env.sh`, `telemetry.sh`).
49
50     ## Key decisions (don't relitigate)
51     - GPU resize MUST be affine `grid_sample` matching `cv2.warpAffine` ? NOT
`F.interpolate` (F1 0.58?0.07; only shows on hard clips). Locked by unit tests now.
52     - `padding_mode="zeros"` (cv2's BORDER_CONSTANT(0)), not "border" ? matters only for
non-16:9 sources; identical to the validated run on 16:9.
53     - PyNvVideoCodec (MIT) for GPU decode; TorchCodec couldn't load on the DL image.
54     - Score with SERVED preset (cap6+R4); raw (0.02,2.0,2.0) is degenerate.
55     - Default-OFF; nvdec/cpu detection caches never cross-replay (manifest keyed by
decode_backend).
56     - WASB weights/NYCU-dataset license (WASB-C3) is a separate launch gate, not a perf
blocker.
57

```

5. user (2026-07-07T14:24:48.162Z)

```

1     ---
2     name: efficient-timeouts-and-retries
3     description: How the owner wants Claude to handle waits, retries, timeouts, and Windows
gcloud SSH ? fast, visible, diagnostic
4     metadata:

```

```

5         type: feedback
6         ---
7
8         The owner strongly dislikes slow, silent retry loops. On 2026-07-06, diagnosing a GPU
VM, I burned ~3 minutes on a 12x15s SSH-readiness loop that suppressed stderr with 2>/dev/null,
so it "failed" invisibly while the owner had already SSH'd into the box manually. The owner
interrupted repeatedly ("be really smart at your job").
9
10        **Why:** wasteful polling plus hidden errors makes me look slow and not smart; the owner
watches actively and interrupts. One visible failure I actually diagnose beats many blind
retries.
11
12        **How to apply:**
13        - ONE attempt with visible stderr, read the real error, then fix ? do not blind-retry a
broken command.
14        - Short, reasonable timeouts (seconds, not minutes). Escalate only with a concrete
reason.
15        - Never hide stderr (2>/dev/null) on a command whose success is uncertain.
16        - Root cause: on Windows, `gcloud compute ssh` uses PuTTY plink.exe/pscp.exe, which
reject OpenSSH `-o` flags and mishandle some `~` remote paths. Use plain `gcloud compute ssh
<vm> --zone --project --command="..."`; to push a file, base64-encode into --command (echo B64 |
base64 -d > file), not scp. To launch detached work without plink hanging ~60s, redirect all fds
and close stdin.
17        - For long remote work: run detached (nohup, log to file/GCS), poll a status file; avoid
many SSH round-trips.
18

```

6. assistant (2026-07-07T14:24:55.272Z)

The real code lives under `khelsutra-guru/badminton-highlight-indexer`. Let me pull the latest and read the key files referenced in the task.

7. user (2026-07-07T14:24:59.688Z)

On branch perf/validation-shared-decode-pass
Your branch is up to date with 'origin/master'.

Changes to be committed:

```

(use "git restore --staged <file>..." to unstage)
    modified:   backend/orchestration.py
    modified:   tests/test_api_cloud_dispatch.py

```

Changes not staged for commit:

```

(use "git add <file>..." to update what will be committed)
(use "git restore <file>..." to discard changes in working directory)
    modified:   backend/pipeline/validators/base.py
    modified:   backend/pipeline/validators/blur.py
    modified:   backend/pipeline/validators/relevance.py
    modified:   backend/pipeline/validators/scene_cut.py

```

Untracked files:

```

(use "git add <file>..." to include in what will be committed)
    backend/pipeline/validators/frame_sampler.py
    score_golden_gate.py
    score_local.py

```

---BRANCH---

perf/validation-shared-decode-pass

---HEAD---

```

3be0935 docs(runlog): DEPLOY_REPORT addendum ? control-plane redeployed to the forwarding image
(#510)
66756ad chore(serving): forward WASB decode/tuning knobs to worker + release hardening (#509)
143dbb9 feat(serving): default cloud-serving to nvdec_resident decode (#508)

```

8. assistant (2026-07-07T14:25:12.212Z)

There's an in-progress branch `perf/validation-shared-decode-pass` with uncommitted changes to exactly the files this task touches (`orchestration.py`, `test\_api\_cloud\_dispatch.py`) plus validator files. Per my global instructions I must not clobber WIP ? let me understand this state before doing anything.

9. user (2026-07-07T14:25:14.897Z)

```
===== STAGED DIFF (orchestration + test) =====
backend/orchestration.py      | 49 ++++++
tests/test_api_cloud_dispatch.py | 108 ++++++
2 files changed, 157 insertions(+)

===== UNSTAGED DIFF (validators) =====
warning: in the working copy of 'backend/pipeline/validators/blur.py', LF will be replaced by
CRLF the next time Git touches it
warning: in the working copy of 'backend/pipeline/validators/relevance.py', LF will be replaced
by CRLF the next time Git touches it
warning: in the working copy of 'backend/pipeline/validators/scene_cut.py', LF will be replaced
by CRLF the next time Git touches it
backend/pipeline/validators/base.py      | 98 ++++++
backend/pipeline/validators/blur.py      | 56 ++++++-----
backend/pipeline/validators/relevance.py | 93 ++++++-----
backend/pipeline/validators/scene_cut.py | 99 ++++++-----
4 files changed, 220 insertions(+), 126 deletions(-)

===== log of branch vs origin/master =====
```

10. assistant (2026-07-07T14:25:26.165Z)

No commits yet ? just uncommitted work on a branch at `origin/master`. There are two distinct change sets: a **staged** set (orchestration + dispatch test ? exactly this task's fix) and an **unstaged** validator refactor (the "shared-decode-pass" the branch is named for). Let me read the staged diff in full to see whether this task's fix is already partly done.

11. user (2026-07-07T14:25:27.552Z)

```
diff --git a/backend/orchestration.py b/backend/orchestration.py
index 5e9f510..bfe39f5 100644
--- a/backend/orchestration.py
+++ b/backend/orchestration.py
@@ -155,6 +155,38 @@ def _ingest_remote_segments(
     return n

+# Trajectory segmenters whose IN-PROCESS detect needs model weights present on THIS box. A
+hosted
+# cloud-serving control-plane bakes NO detector weights into its image (only the worker Cloud
+Run Job
+# GCSFuse-mounts them), so forcing compute="local" for one of these there can never succeed ?
+the
+# guardrail in _maybe_dispatch_remote fails such a request FAST instead of the cryptic deep
+# "WASB weights not found". (Names are the registered segmenter identities; see
+backend/pipeline/segmenters.)
+_WEIGHTS_BACKED_SEGMENTERS = frozenset({"wasb_hybrid", "tracknet_hybrid", "fusion_hybrid"})
+
+
+def _local_detector_weights_present(cfg, seg_name: str) -> bool:
+    """Whether the trajectory weights the IN-PROCESS ``seg_name`` detector needs are present +
+readable
+    on THIS box. Mirrors the native/WSL runners' resolution (env override ?
+``indexing.<det>.weights_path``)
+    and the ``/api/capabilities`` probe, so the guardrail agrees with the healthcheck that
```

```

would
+   otherwise fail deep in the run. Fusion follows its configured substrate (WASB default /
TrackNet).
+   A non-weights segmenter returns True (nothing local to check)."""
+   name = (seg_name or "").strip().lower()
+   if name not in _WEIGHTS_BACKED_SEGMENTERS:
+       return True
+   idx = getattr(cfg, "indexing", None)
+   substrate = (
+       getattr(getattr(idx, "fusion", None), "substrate", "") or "wasb"
+   ).strip().lower()
+   if name == "tracknet_hybrid" or (name == "fusion_hybrid" and substrate == "tracknet"):
+       path = getattr(getattr(idx, "tracknet", None), "weights_path", "") or ""
+   else: # wasb_hybrid, or fusion on the (default) WASB substrate
+       path = (
+           os.environ.get("WASB_WEIGHTS")
+           or getattr(getattr(idx, "wasb", None), "weights_path", "")
+           or ""
+       )
+   return bool(path) and os.path.exists(os.path.expanduser(path))
+
+
+   def _cloud_available(cfg) -> bool:
+       """Whether THIS server can satisfy a per-request ``compute="cloud"`` dispatch (item 3).

@@ -446,6 +478,23 @@ def _maybe_dispatch_remote(
    )
    choice = ""
    if choice == "local":
+       # A hosted cloud-serving control-plane bakes NO detector weights into its image (only
the
+       # worker Cloud Run Job GCSFuse-mounts them), so forcing a weights-backed trajectory
detector
+       # to run IN-PROCESS here can never succeed ? it dies deep in the segmenter with a
cryptic
+       # "WASB weights not found" (observed live: job f29d41d0?, 2026-07-07). Fail FAST with
an
+       # actionable message instead of that silent trap. A truly-local box (not cloud-
capable), or a
+       # cloud-wired box that ACTUALLY has the weights on disk, is unaffected ? it runs in-
process
+       # below exactly as before (the weights-present check is the escape hatch).
+       if (
+           (seg_name or "").strip().lower() in _WEIGHTS_BACKED_SEGMENTERS
+           and _cloud_available(cfg)
+           and not _local_detector_weights_present(cfg, seg_name)
+       ):
+           return _failed(
+               f"this hosted server can't run detection locally ? the '{seg_name}' model
weights "
+               "live only on the cloud worker, not on the control-plane. Choose 'Run in
cloud'.",
+               False,
+           )
+       return None # explicit "run on my machine" ? in-process regardless of the server's
target
    if choice == "cloud":
        if not _cloud_available(cfg):

```

12. assistant (2026-07-07T14:26:17.579Z)

The staged `orchestration.py` change is actually the `"""Guard compute=local"""` related task (adds `\_WEIGHTS\_BACKED\_SEGMENTERS` + a guardrail), not my task. And the unstaged validator refactor is the "shared-decode-pass" the branch is named for. So this shared checkout has `two` other in-progress efforts tangled in the exact files my task touches. I must not clobber them.

Let me finish reading (read-only, safe) to fully understand before deciding how to isolate my work.

13. user (2026-07-07T14:26:18.871Z)

(Bash completed with no output)

14. user (2026-07-07T14:26:23.869Z)

(Bash completed with no output)

15. user (2026-07-07T14:26:32.238Z)

```
---numstat---
---try GIT_PAGER off---
```

16. assistant (2026-07-07T14:26:37.709Z)

The index is changing under me between commands ? exactly the concurrent-session hazard. Let me re-check state right now.

17. user (2026-07-07T14:26:39.668Z)

On branch perf/validation-shared-decode-pass
Your branch is up to date with 'origin/master'.

Changes not staged for commit:

```
(use "git add <file>..." to update what will be committed)
(use "git restore <file>..." to discard changes in working directory)
    modified:   backend/pipeline/validators/base.py
    modified:   backend/pipeline/validators/blur.py
    modified:   backend/pipeline/validators/relevance.py
    modified:   backend/pipeline/validators/scene_cut.py
```

Untracked files:

```
(use "git add <file>..." to include in what will be committed)
    backend/pipeline/validators/frame_sampler.py
    score_golden_gate.py
    score_local.py
    tests/test_frame_sampler.py
```

no changes added to commit (use "git add" and/or "git commit -a")

===BRANCH===

perf/validation-shared-decode-pass

===HEAD===

3be0935 docs(runlog): DEPLOY_REPORT addendum ? control-plane redeployed to the forwarding image (#510)

66756ad chore(serving): forward WASB decode/tuning knobs to worker + release hardening (#509)

143dbb9 feat(serving): default cloud-serving to nvdec_resident decode (#508)

18. assistant (2026-07-07T14:27:07.278Z)

Confirmed: a ****sibling session is actively editing this checkout right now**** ? between my commands the staged ``orchestration.py`/`test_api_cloud_dispatch.py`` changes vanished and a new ``tests/test_frame_sampler.py`` appeared. The tree is volatile. Per my instructions I'll isolate my work in a fresh worktree from ``origin/master`` rather than entangle with the live session.

Let me load the worktree tool and, in parallel, read the stable ``origin/master`` versions of the files I need to design against.

19. user (2026-07-07T14:27:10.474Z)

```
===== grep run_all_with_context in worker/__main__.py (origin/master) =====
origin/master:backend/worker/__main__.py:39:from backend.pipeline.validators.base import
```



```
registry as validator_registry
origin/master:backend/worker/__main__.py:263:     val_results, val_context =
validator_registry.run_all_with_context(
origin/master:backend/worker/__main__.py:309:         "[worker] validation FAILED: "
```

20. assistant (2026-07-07T14:27:41.226Z)

I have the worktree tool, but its usage is gated on the user explicitly saying "worktree" ? which they didn't. The equivalent safe mechanism is a plain `git worktree add` from `origin/master`, which isolates my work without carrying the sibling's uncommitted changes and without touching their checkout. This directly satisfies my "branch from origin/master" + "never clobber WIP" instructions.

Let me create an isolated worktree based on `origin/master`.

21. user (2026-07-07T14:27:44.383Z)

```
Preparing worktree (new branch 'fix/worker-skip-revalidation-on-dispatch')
branch 'fix/worker-skip-revalidation-on-dispatch' set up to track 'origin/master'.
HEAD is now at 3be0935 docs(runlog): DEPLOY_REPORT addendum ? control-plane redeployed to the
forwarding image (#510)
===LIST===
C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer
3be0935 [perf/validation-shared-decode-pass]
C:/Users/avidu/AppData/Local/Temp/claude/c--Users-avidu-Projects-badminton-highlight-
indexer/6e9d0958-a2ec-4a8e-bb2d-6cce9833c527/scratchpad/cfg-wt      20b2d5a [claude/wasb-
config-validation]
C:/Users/avidu/AppData/Local/Temp/claude/c--Users-avidu-Projects-badminton-highlight-
indexer/6e9d0958-a2ec-4a8e-bb2d-6cce9833c527/scratchpad/compile-wt  b09a1d4 [claude/compile-
reliability]
C:/Users/avidu/AppData/Local/Temp/claude/c--Users-avidu-Projects-badminton-highlight-
indexer/6e9d0958-a2ec-4a8e-bb2d-6cce9833c527/scratchpad/pr458-pre-wt  355dad6 (detached HEAD)
C:/Users/avidu/AppData/Local/Temp/claude/c--Users-avidu-Projects-badminton-highlight-
indexer/6e9d0958-a2ec-4a8e-bb2d-6cce9833c527/scratchpad/pr458-wt      5de1ef2 (detached HEAD)
C:/Users/avidu/Projects/khelsutra-guru/bhi-wt-worker-reval
3be0935 [fix/worker-skip-revalidation-on-dispatch]
```

22. assistant (2026-07-07T14:27:58.907Z)

Worktree created cleanly at `bhi-wt-worker-reval` on `fix/worker-skip-revalidation-on-dispatch` from `origin/master` (3be0935). (The listing confirms heavy parallel activity ? several other worktrees exist.) Now let me read the key files from this clean base.

23. user (2026-07-07T14:28:01.816Z)

```
===== orchestration: dispatch/forwarding landmarks =====
backend/orchestration.py:4:Cloud Run dispatch + ingest path (`_maybe_dispatch_remote` /
`_ingest_remote_segments`),
backend/orchestration.py:391:def _maybe_dispatch_remote(
backend/orchestration.py:546:     # skip_ai_handoff=True the WASB windows become rallies directly
(no Gemini key needed); without
backend/orchestration.py:547:     # forwarding the cloud job would default to the handoff and
yield 0 rallies when it has no key.
backend/orchestration.py:551:     sk = getattr(idx_cfg, "skip_ai_handoff", None)
backend/orchestration.py:553:         overrides.append(f"indexing.skip_ai_handoff={'true' if sk
else 'false'}")
backend/orchestration.py:556:     # without forwarding, its decode_backend + GPU-feed tuning
silently fall back to the model
backend/orchestration.py:557:     # defaults (cpu, batch 8) regardless of what the control-plane
resolved. decode_backend is
backend/orchestration.py:559:     # (? #507); batch_size/prefetch_batches are forwarded only when
set (None = the parity default).
backend/orchestration.py:560:     wasb_cfg = getattr(idx_cfg, "wasb", None)
backend/orchestration.py:561:     if wasb_cfg is not None:
```

```

backend/orchestration.py:562:         decode_backend = getattr(wasb_cfg, "decode_backend", None)
backend/orchestration.py:563:         if decode_backend:
backend/orchestration.py:564:         overrides.append(f"indexing.wasb.decode_backend={decode_backend}")
backend/orchestration.py:565:         batch_size = getattr(wasb_cfg, "batch_size", None)
backend/orchestration.py:567:         overrides.append(f"indexing.wasb.batch_size={batch_size}")
backend/orchestration.py:568:         prefetch = getattr(wasb_cfg, "prefetch_batches", None)
backend/orchestration.py:570:         overrides.append(f"indexing.wasb.prefetch_batches={prefetch}")
backend/orchestration.py:721:         # _maybe_dispatch_remote when it dispatches. None ? nothing ran
(validation/config fail).
backend/orchestration.py:798:         remote = _maybe_dispatch_remote(

```

24. user (2026-07-07T14:28:10.773Z)

```

391     def _maybe_dispatch_remote(
392         cfg,
393         db,
394         req: "ProcessRequest",
395         video_id: str,
396         seg_name: str,
397         val_results,
398         play_wm: int,
399         cand_wm: int,
400         notify,
401     ) -> Optional[Tuple[Dict[str, Any], bool]]:
402         """Cloud-serving (COMPUTE_DECOUPLED_SERVING M6 ? the API path): when
`deployment.compute_target
403         == "cloud_run"`, run the heavy DETECT on a Cloud Run GPU Job and INGEST its rally
intervals into
404         THIS DB, so the rest of the flow (query ? compile/local-stitch) is unchanged.
405
406         Returns `(response_dict, ran)` when the cloud path is taken ? `ran` is whether
the cloud job
407         actually EXECUTED (`False` for a pre-dispatch config rejection, so the
observability report
408         doesn't claim a fake segmentation run) ? or `None` to fall through to the in-
process segmenter
409         (**DEFAULT-OFF*: `get_compute_backend` returns `None` for every
non-`cloud_run` target, so
410         the in-process path stays byte-identical)."""
411         from backend.compute import ComputeJobSpec, get_compute_backend
412
413         def _failed(msg: str, ran: bool) -> Tuple[Dict[str, Any], bool]:
414             # Shape-match the in-process segmenter-fail branch + drop any partial NEW rows
(id > play_wm);
415             # prior good rows (id <= play_wm) are preserved (destroy-AFTER-confirm).
416             db.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
417             # Provenance even on failure, and HONEST about what ran: `path` is "cloud_run"
only when the
418             # job actually dispatched + executed remotely (ran=True, e.g. a non-zero worker
rc / ingest
419             # error); a pre-dispatch reject (ran=False) ran NOWHERE ? "not_run". `target`
records the
420             # intended backend, `requested` the picker choice, `dispatched` whether the
remote job ran.
421             return (
422                 {
423                     "video_id": video_id,
424                     "status": "failed",
425                     "message": msg,
426                     "results": [r.model_dump() for r in val_results],
427                     "play_segments": [],
428                     "compute": {

```

```

429         "path": "cloud_run" if ran else "not_run",
430         "requested": req.compute,
431         "target": "cloud_run",
432         "dispatched": ran,
433     },
434 },
435 ran,
436 )
437
438 # SPA compute-picker (item 3): a per-request choice overrides the server's
configured default.
439 # "local" forces the in-process segmenter; "cloud" forces a Cloud Run dispatch
(guarded by
440 # _cloud_available so we fail clearly rather than silently running local); None /
unknown ? default.
441 choice = (req.compute or "").strip().lower()
442 if choice and choice not in ("local", "cloud"):
443     logger.warning(
444         "[API] ignoring unknown compute=%r (expected 'local'|'cloud'); using server
default.",
445         req.compute,
446     )
447     choice = ""
448 if choice == "local":
449     return None # explicit "run on my machine" ? in-process regardless of the
server's target
450 if choice == "cloud":
451     if not _cloud_available(cfg):
452         return _failed(
453             "cloud-serving was requested but this server isn't configured for it "
454             "(no Cloud Run target). Pick 'run on my machine', or configure cloud
serving.",
455             False,
456         )
457     compute = get_compute_backend(cfg, target_override="cloud_run")
458 else:
459     compute = get_compute_backend(
460         cfg
461     ) # server default (default-OFF unless cloud_run)
462 if compute is None:
463     return None # default-OFF ? run the in-process segmenter below (unchanged)
464
465 storage_cfg = getattr(cfg, "storage", None)
466
467 # The Cloud Run job fetches the input FROM THE BUCKET ? it can't read a path local
to this box.
468 try:
469     input_ref = storage.resolve_input_ref(req.video_path, storage_cfg)
470 except ValueError as e:
471     return _failed(
472         f"cloud-serving: unresolvable input '{req.video_path}': {e}", False
473     )
474 if input_ref.backend == "local":
475     # A browser upload lands on THIS box's local disk; the Cloud Run job (a
different machine)
476     # can't read it. Stage it UP to the input bucket first, then dispatch from there
(#461). Only
477     # possible when a CLOUD input_root is configured; without one we genuinely can't
stage ? fail.
478     # Classify the ROOT ITSELF (parse_uri), not storage.input_backend: the staged
URI below is
479     # built by joining under input_root, and storage.put routes by
parse_uri(staged_uri).backend ?
480     # so an explicit cloud root (gs://?, s3://?) stages fine with input_backend left
at its

```

```

481         # default, while a bare/local-looking root can't stage no matter what
input_backend says
482         # (resolve_input_uri only consults input_backend when input_root is EMPTY, and
an empty root
483         # gives us nothing to build the staged URI under).
484         input_root = (getattr(storage_cfg, "input_root", "") or "").strip()
485         try:
486             root_backend = (
487                 storage.parse_uri(input_root).backend if input_root else "local"
488             )
489         except ValueError: # unsupported scheme in input_root ? nowhere real to stage
to
490             root_backend = "local"
491         if root_backend == "local":
492             return _failed(
493                 "cloud-serving needs a cloud input (e.g. gs://?), or a cloud
storage.input_root to "
494                 "stage local uploads into; a path local to this machine can't be read by
the Cloud "
495                 "Run job.",
496                 False,
497             )
498         local_src = input_ref.key # resolved local path of the upload
499         staged_uri =
f"{input_root.rstrip('/')}/{video_id}/{os.path.basename(local_src)}"
500         try:
501             notify("staging upload to bucket")
502             # Thread the active storage settings so backend-specific config survives (S3
endpoint_url/
503             # region/addressing for R2·D0·MinIO; GCS project/client) ? not a fresh
default client.
504             storage.put(
505                 local_src,
506                 staged_uri,
507                 config=(
508                     storage_cfg.model_dump() if storage_cfg is not None else None
509                 ),
510             )
511         except Exception as e: # noqa: BLE001 ? a staging failure is a clean job
failure, not a 500
512             logger.error(
513                 "[API] cloud-serving: failed to stage %s -> %s: %s",
514                 local_src,
515                 staged_uri,
516                 e,
517                 exc_info=True,
518             )
519             return _failed(
520                 f"cloud-serving: failed to stage the uploaded file to {input_root}:
{e}",
521                 False,
522             )
523         # Repoint BOTH the dispatch and the DB video record at the staged bucket object:
the L4 job
524         # DETECTs from it, and a later /api/compile re-fetches the same source (the
local upload is
525         # gone once this instance recycles). add_video is an UPSERT ? it only updates
filepath.
526         req.video_path = staged_uri
527         input_ref = storage.resolve_input_ref(staged_uri, storage_cfg)
528         db.add_video(
529             video_id, staged_uri, "processed", [r.model_dump() for r in val_results]
530         )
531         out_root = (getattr(storage_cfg, "output_root", "") or "").strip()
532         if not out_root:

```

```

533         return _failed(
534             "cloud-serving needs storage.output_root set to a bucket (e.g. gs://your-
bucket) "
535             "? that's where the Cloud Run job writes the result.",
536             False,
537         )
538
539     notify("dispatching (cloud_run)")
540     logger.info(
541         "[API] cloud-serving: dispatching %s -> Cloud Run Job (out=%s)",
542         req.video_path,
543         out_root,
544     )
545     # Forward the engine's AI-handoff stance so the cloud job runs the SAME detection:
with
546     # skip_ai_handoff=True the WASB windows become rallies directly (no Gemini key
needed); without
547     # forwarding the cloud job would default to the handoff and yield 0 rallies when it
has no key.
548     # (player_pool / max_frames are still dropped on the cloud path ? a separate
documented follow-up.)
549     overrides = []
550     idx_cfg = getattr(cfg, "indexing", None)
551     sk = getattr(idx_cfg, "skip_ai_handoff", None)
552     if sk is not None:
553         overrides.append(f"indexing.skip_ai_handoff={'true' if sk else 'false'}")
554     # Forward the resolved WASB serving knobs so the PRESET governs the worker
(#506/#507). The
555     # worker Job has no DEPLOYMENT_PROFILE, so it does NOT apply the cloud-serving
preset itself ?
556     # without forwarding, its decode_backend + GPU-feed tuning silently fall back to the
model
557     # defaults (cpu, batch 8) regardless of what the control-plane resolved.
decode_backend is
558     # cuda-only (validator-enforced) and requires a worker image that understands
--decode-backend
559     # (? #507); batch_size/prefetch_batches are forwarded only when set (None = the
parity default).
560     wasb_cfg = getattr(idx_cfg, "wasb", None)
561     if wasb_cfg is not None:
562         decode_backend = getattr(wasb_cfg, "decode_backend", None)
563         if decode_backend:
564             overrides.append(f"indexing.wasb.decode_backend={decode_backend}")
565         batch_size = getattr(wasb_cfg, "batch_size", None)
566         if batch_size:
567             overrides.append(f"indexing.wasb.batch_size={batch_size}")
568         prefetch = getattr(wasb_cfg, "prefetch_batches", None)
569         if prefetch:
570             overrides.append(f"indexing.wasb.prefetch_batches={prefetch}")
571     spec = ComputeJobSpec(
572         video_uri=req.video_path,
573         out_uri=out_root,
574         segmenter=seg_name,
575         config_overrides=tuple(overrides),
576     )
577     try:
578         # Dispatch the Cloud Run Job + bucket-poll to completion. on_wait is the live
579         # heartbeat (#482): job.progress moves on every poll tick instead of freezing at
580         # "segmenting" for the whole GPU run (the 2026-07-05 silent 22-min spinner) ?
which
581         # also gives the SPA a stall signal to time out on, instead of a blind wall
clock.
582         result = compute.run_infer(
583             spec,
584             on_wait=lambda elapsed: notify(_wait_progress_message(seg_name, elapsed)),

```

```

585         )
586         except Exception as e: # noqa: BLE001 ? any dispatch/poll error is a clean job
failure, not a 500
587             logger.error("[API] cloud dispatch error: %s", e, exc_info=True)
588             return _failed(f"cloud dispatch error: {e}", True)
589         if result.returncode != 0:
590             return _failed(f"cloud job failed (returncode {result.returncode}).", True)
591         if result.video_id and result.video_id != video_id:
592             # The worker re-hashes the bucket bytes; a divergence means the input changed
between this
593             # box's hash and the worker's ? ingest under the LOCAL id but flag it so it's
diagnosable.
594             logger.warning(
595                 "[API] cloud video_id %s != local %s ? input bytes differ between the API
hash "
596                 "and the worker hash; ingesting under the local id.",
597                 result.video_id,
598                 video_id,
599             )
600
601         notify("ingesting (cloud_run)")
602         storage_dict = (
603             storage_cfg.model_dump() # type: ignore[union-attr] # hasattr-guarded; mypy
can't narrow
604             if hasattr(storage_cfg, "model_dump")
605             else {}
606         )
607         try:
608             _ingest_remote_segments(db, video_id, result.outputs_uri, storage_dict)
609         except Exception as e: # noqa: BLE001 ? a missing/partial/corrupt result must not
500 or leak rows
610             logger.error("[API] cloud ingest failed: %s", e, exc_info=True)
611             return _failed(
612                 f"cloud-serving: could not read the result from {result.outputs_uri}: {e}",
613                 True,
614             )
615
616         # Only the NEW cloud rows (id > the pre-run watermark); _finalize_reingest then
drops the old ones.
617         segments = [
618             s for s in db.query_play_segments(video_id, {}) if int(s.get("id", 0)) > play_wm
619         ]
620         _finalize_reingest(db, video_id, segments, play_wm, cand_wm)
621         # Compute-path PROVENANCE: the GCP-verifiable proof the cloud path ran. `execution`
is the real
622         # Cloud Run execution name (cross-check: gcloud run jobs executions describe
<execution>).
623         provenance = {
624             "path": "cloud_run",
625             "requested": req.compute,
626             "job": os.environ.get("CLOUD_RUN_JOB", "rally-worker"),
627             "region": os.environ.get("CLOUD_RUN_REGION", "asia-south1"),
628             "execution": result.execution,
629             "outputs_uri": result.outputs_uri,
630         }

```

25. assistant (2026-07-07T14:28:25.749Z)

Now I understand the #509 forwarding pattern (`overrides` list ? `config\_overrides` tuple ? worker `--set` args). Let me read the worker's `cmd\_infer` validation block and how `--set`/args are parsed, plus how `config\_overrides` becomes worker CLI args.

26. user (2026-07-07T14:28:26.767Z)

```

210         segment_count: int,

```

```

211         status: str,
212         storage_cfg: dict,
213         scratch: str,
214     ) -> None:
215         """M6: write a tiny JSON completion marker to ``done_uri`` (via the storage layer)
for a remote
216         dispatcher's bucket-poll. Best-effort ? never raises into the worker's success
return."""
217         try:
218             marker = {
219                 "video_id": video_id,
220                 "returncode": returncode,
221                 "segment_count": segment_count,
222                 "status": status,
223             }
224             local = os.path.join(scratch, "_done.json")
225             with open(local, "w", encoding="utf-8") as f:
226                 json.dump(marker, f)
227                 storage.put(local, done_uri, config=storage_cfg)
228                 logger.info("[worker] wrote completion marker -> %s", done_uri)
229             except Exception as e: # never fail a good run because the marker push hiccuped
230                 logger.warning("[worker] could not write done-marker %s: %s", done_uri, e)
231
232
233     def cmd_infer(args: argparse.Namespace) -> int:
234         config = load_config(args.config) if args.config else load_config()
235         _apply_overrides(config, args.set)
236         if not _run_startup_checks(config):
237             return 2
238
239         # M6: a cloud compute_target DISPATCHES to a Cloud Run Job instead of running in-
process.
240         # DEFAULT-OFF: get_compute_backend returns None for ""/local_gpu/cpu_mock ? fall
through to the
241         # unchanged inline path below. The dispatched container runs THIS cmd_infer inline
(the
242         # dispatcher forces compute_target=local_gpu), so it never recursively re-
dispatches.
243         compute = get_compute_backend(config)
244         if compute is not None:
245             return _dispatch_remote(compute, args)
246
247         storage_cfg = config.storage.model_dump()
248
249         scratch = args.scratch or tempfile.mkdtemp(prefix="rally-worker-")
250         os.makedirs(scratch, exist_ok=True)
251         config.output.output_dir = scratch
252
253         # 1. PULL ? local:// / bare path = identity passthrough; s3:// / gcs:// = download
to scratch.
254         local_video = storage.fetch(
255             args.video_uri, os.path.join(scratch, "in.mp4"), config=storage_cfg
256         )
257         print(f"[worker] pulled {args.video_uri} -> {local_video}")
258
259         # 2. IDENTITY ? whole-file MD5 (the labels?video join key; pulled bytes must be
byte-identical).
260         video_id = compute_video_id(local_video)
261
262         # 3. VALIDATE (same registry as the main CLI).
263         val_results, val_context = validator_registry.run_all_with_context(
264             local_video, config
265         )
266         failures = [r for r in val_results if not r.passed]
267         db = Database(os.path.join(scratch, config.output.db_name))

```

```

268         db.add_video(
269             video_id,
270             local_video,
271             "failed" if failures else "processed",
272             [r.model_dump() for r in val_results],
273         )
274
275     def _fail(rc: int) -> int:
276         # A worker-side FAILURE (validation, unknown segmenter, or a detector
277         # segmenter surfaced as a Failure) must NOT masquerade as a 0-rally "success".
278         # a status="failed" report.json/intervals.csv (0 segments) so the control
279         # truth (submit-cache probe, restart-recovery) and the alpha-user result surface
280         # explicit FAILURE ? the wf294 silent-failure fix. Best-effort: a push hiccup
281         # mask the original failure. THEN the M6 done-marker (real rc + status) so a
282         # dispatcher's bucket-poll terminates FAST instead of hanging until its budget
283         # expires.
284         try:
285             _serialize_and_push_outputs(
286                 scratch,
287                 args.out_uri,
288                 video_id,
289                 [],
290                 "failed",
291                 storage_cfg,
292                 str(getattr(args, "video_uri", "")) or "",
293             )
294         except Exception as e: # never mask the real failure on an artifact-push hiccup
295             logger.warning("[worker] could not push failure artifacts: %s", e)
296             # DEFAULT-OFF: no --done-uri ? no marker written (the unchanged local run).
297             if getattr(args, "done_uri", None):
298                 _write_done_marker(
299                     args.done_uri, video_id, rc, 0, "failed", storage_cfg, scratch
300                 )
301             return rc
302
303     segments: List[dict] = []
304     segmenter_actual = None
305     segmentation_ran = False
306     status = "failed"
307     try:
308         if failures:
309             print(
310                 "[worker] validation FAILED: "
311                 + "; ".join(f"{r.validator_name}: {r.message}" for r in failures)
312             )
313             return _fail(2)
314         seg_name = args.segmenter or config.indexing.default_segmenter
315         segmenter_cls = segmenter_registry.get(seg_name)
316         if not segmenter_cls:
317             print(
318                 f"[worker] unknown segmenter: {seg_name!r} "
319                 f"(have {list(segmenter_registry.list_available())})"
320             )
321             return _fail(2)
322         segmenter_actual = seg_name
323         result = segmenter_cls(db, config).process_video(
324             video_id, local_video, max_frames=args.max_frames
325         )
326         segmentation_ran = True

```



```

326         if isinstance(result, Failure):
327             print(f"[worker] segmenter FAILED: {result.message}")
328             return _fail(3)
329         segments = result.value if hasattr(result, "value") else result
330         status = "success"
331         # Loud, never-silent diagnostic: a 0-segment success is almost always a
misconfig, not an
332         # empty match ? the cold-start failure mode (the substrate detector yielded no
usable
333         # trajectories). Explain it so a run is analysable from the log alone (re-run -v
for more).
334         if not segments:
335             detector_impl = (
336                 os.environ.get("RALLY_DETECTOR_IMPL")
337                 or getattr(config.indexing, "detector_impl", None)
338                 or "auto"
339             )

```

27. assistant (2026-07-07T14:28:56.278Z)

Now the critical question for the "skip validation" approach: `run\_all\_with\_context` returns a `val\_context` that may feed downstream segmentation. If so, I can't blindly skip it. Let me trace `val\_context` usage and the worker's arg parser before choosing an approach.

28. user (2026-07-07T14:29:00.798Z)

===== val_context usages (worker) =====

```

backend/worker/__main__.py:263:     val_results, val_context =
validator_registry.run_all_with_context(
backend/worker/__main__.py:361:         val_context=val_context,

```

===== run_all_with_context definition + context consumers (repo-wide) =====

```

backend/main.py:1513:         val_results, val_context = registry.run_all_with_context(
backend/main.py:1579:             val_context=val_context,
backend/orchestration.py:718:     val_context: Dict[str, Any] = {}
backend/orchestration.py:747:         val_results, val_context =
registry.run_all_with_context(local_video, config)
backend/orchestration.py:884:         val_context=val_context,
backend/pipeline/validators/base.py:50:     def run_all_with_context(self, video_path: str,
config: Any):
backend/pipeline/validators/base.py:76:     def run_all(self, video_path: str, config: Any) ->
List[ValidationResult]:
backend/pipeline/validators/base.py:78:         results, _ =
self.run_all_with_context(video_path, config)
backend/reporting/__init__.py:36:     val_context: Optional[Dict[str, Any]] = None,
backend/reporting/__init__.py:65:         val_context=val_context,
backend/reporting/harvest.py:33:     val_context: Optional[Dict[str, Any]], video_path:
Optional[str]
backend/reporting/harvest.py:44:     ffmata = (val_context or {}).get("ffprobe_meta") or {}
backend/reporting/harvest.py:209:     val_context: Optional[Dict[str, Any]],
backend/reporting/harvest.py:231:     vm = video_meta_from_context(val_context, video_path)

```

29. user (2026-07-07T14:29:08.474Z)

```

340         logger.warning(
341             "infer produced 0 segments for video_id=%s (segmenter=%s,
detector_impl=%s). "
342             "The detector substrate yielded no usable trajectories/windows ? check
the detector "
343             "logs above; common causes: a missing ${AI_PROVIDER}_API_KEY with "
344             "skip_ai_handoff=False (the segmenter bails BEFORE detection ? set the
key or "
345             "indexing.skip_ai_handoff=true; cloud-serving now fails this at
startup), native "

```

```

346         "runner UNAVAILABLE (weights/repo/WASB_PYTHON env), the WASB env
produced no "
347         "detections, or the input resolution differs from the tuned proxy
resolution. "
348         "Re-run with -v for the native command + frame counts.",
349         video_id,
350         segmenter_actual,
351         detector_impl,
352     )
353     finally:
354         # Best-effort per-video observability report (never raises) ? parity with the
main CLI.
355         emit_indexer_report(
356             db=db,
357             video_id=video_id,
358             video_path=local_video,
359             config=config,
360             val_results=val_results,
361             val_context=val_context,
362             segmenter_requested=args.segmenter,
363             segmenter_actual=segmenter_actual,
364             was_reingest=False,
365             segmentation_ran=segmentation_ran,
366         )
367
368     # 5. SERIALIZE + 6. PUSH (outputs/<video_id>/?). idempotent on video_id.
369     pushed = _serialize_and_push_outputs(
370         scratch,
371         args.out_uri,
372         video_id,
373         segments,
374         status,
375         storage_cfg,
376         str(getattr(args, "video_uri", "") or ""),
377     )
378
379     print(
380         f"[worker] infer done: {len(segments)} segment(s); pushed {len(pushed)}
artifact(s):"
381     )
382     for u in pushed:
383         print("    ", u)
384
385     # M6: a remote dispatcher passes --done-uri; write the SUCCESS completion marker so
its bucket-poll
386     # terminates (carries video_id ? the dispatcher's output path + rc=0). Worker
FAILURES (rc 2/3)
387     # write their own marker via _fail() above, so the dispatcher fails FAST either way
(no 30-min hang
388     # on a bad video). DEFAULT-OFF: no --done-uri ? no marker written (the unchanged
local run).
389     if getattr(args, "done_uri", None):
390         _write_done_marker(
391             args.done_uri, video_id, 0, len(segments), status, storage_cfg, scratch
392         )
393     return 0
394
395
396     #: Video extensions accepted under <corpus>/videos/ (mirrors the labels/ '.csv' filter
so a
397     #: per-video sidecar ? .srt/.json/thumbnail ? can't shadow the real video on a stem
collision).
398     _VIDEO_EXTS = (".mp4", ".mov", ".mkv", ".avi", ".webm", ".m4v")
399
400     #: Leading ISO-date prefix on a canonical label name (`labels/<YYYY-MM-

```

```

DD>_<name>.rallies.csv``,
401     #: docs/DATA_IN_GCS.md §7.1). It disambiguates recycled GoPro names by authoring date
but is NOT
402     #: part of the clip's join key, so it's stripped when deriving the videos/ stem.
403     _LABEL_DATE_PREFIX = re.compile(r"^\d{4}-\d{2}-\d{2}_")
404
405
406     def _label_clip_name(fname: str) -> str:
407         """Derive the clip name (the ``videos/<name>.<ext>`` join key) from a label
filename.
408
409         Strips both the ``.rallies.csv`` / ``.csv`` suffix and an optional leading ISO-date
prefix, so a

```

30. assistant (2026-07-07T14:29:34.681Z)

`val\_context` feeds only the observability **report** (``emit_indexer_report` ? `harvest.py`
ffprobe meta`), not segmentation ? so skipping validation on the worker is functionally safe for
detection. Let me confirm the control-plane's own flow: that it validates and gates dispatch,
and emits its own rich report.

31. user (2026-07-07T14:29:36.250Z)

```

700         raise
701     try:
702         existing_video = db.get_video(video_id)
703     except Exception:
704         storage.cleanup_acquired_local_input(
705             req.video_path, local_video, getattr(config, "storage", None)
706         )
707         raise
708     was_reingest = existing_video is not None
709
710     # typed AppConfig: attribute access for the default segmenter (with safe fallback)
711     default_seg = getattr(
712         getattr(config, "indexing", None), "default_segmenter", "motion"
713     )
714     seg_name = req.segmenter or default_seg
715     segmenter_actual = None
716     segmentation_ran = False
717     val_results: List[Any] = []
718     val_context: Dict[str, Any] = {}
719     # Compute-path PROVENANCE (the validation method): which backend actually ran the
DETECT ?
720     # "in_process" once the local segmenter is reached, set on the response as
"cloud_run" by
721     # _maybe_dispatch_remote when it dispatches. None ? nothing ran (validation/config
fail).
722     compute_actual: Optional[str] = None
723     response: Optional[Dict[str, Any]] = None
724     try:
725         if (
726             existing_video
727             and existing_video.get("status") == "processed"
728             and not req.force
729         ):
730             cached_segments = db.query_play_segments(video_id, {})
731             if cached_segments:
732                 logger.info(
733                     "[API] Reusing cached segmentation for video_id=%s; force=true
bypasses cache.",
734                     video_id,
735                 )
736             response = _cached_process_payload(
737                 video_id,

```

```

738         existing_video.get("validation_results", []),
739         cached_segments,
740         getattr(req, "compute", None),
741     )
742     return response
743
744     # 1. Run pluggable validators (with-context variant surfaces ffprobe_meta for
the report)
745     notify("validating")
746     logger.info(f"Running validations for {req.video_path}...")
747     val_results, val_context = registry.run_all_with_context(local_video, config)
748     failures = [r for r in val_results if not r.passed]
749
750     if failures:
751         # add_video is an UPSERT (no cascade) ? a failed re-validation only updates
752         # status; it no longer destroys the video's prior good segments.
753         db.add_video(
754             video_id,
755             req.video_path,
756             "failed",
757             [r.model_dump() for r in val_results],
758         )
759         response = {
760             "video_id": video_id,
761             "status": "failed",
762             "message": _validation_failure_message(failures),
763             "results": [r.model_dump() for r in val_results],
764         }
765         return response
766
767     # 2. Run segmenter & indexer
768     logger.info("[API] Video passed checks. Segmenting and indexing...")
769     db.add_video(
770         video_id, req.video_path, "processed", [r.model_dump() for r in val_results]
771     )
772
773     segmenter_cls = segmenter_registry.get(seg_name)
774     if not segmenter_cls:
775         # Submit-time validation already 400s unknown names; this is the defensive
776         # in-worker path (e.g. config default pointing at an unregistered
segmenter).
777         available = list(segmenter_registry.list_available().keys())
778         response = {
779             "video_id": video_id,
780             "status": "failed",
781             "message": f"Segmenter '{seg_name}' not found. Available segmenters:
{available}",
782             "results": [r.model_dump() for r in val_results],
783             "play_segments": [],
784         }
785         return response
786
787     logger.info(f"[API] Using segmenter: {seg_name}")
788     notify(f"segmenting ({seg_name})")
789     segmenter_actual = seg_name
790     # Destroy-AFTER-confirm: remember the pre-run rows, then keep new + drop old
only
791     # if the run yields segments; otherwise drop the new partial rows and keep the
old.
792     play_wm, cand_wm = db.segment_watermarks(video_id)
793     # COMPUTE_DECOUPLED_SERVING M6 ? API: with deployment.compute_target=cloud_run,
run the heavy
794     # DETECT on a Cloud Run GPU Job + ingest its rallies HERE so query/compile are
unchanged.
795     # DEFAULT-OFF: returns None for every non-cloud target ? the in-process

```

```

segmenter below runs
796         # byte-identically. (player_pool/max_frames aren't threaded to the worker
`infer` CLI yet, so
797         # they're dropped on the cloud path in this slice ? a documented follow-up.)
798         remote = _maybe_dispatch_remote(
799             config, db, req, video_id, seg_name, val_results, play_wm, cand_wm, notify
800         )
801         if remote is not None:
802             response, segmentation_ran = (
803                 remote # `ran` = did the cloud job actually execute (report)
804             )
805             return response # the cloud branch already stamped response["compute"]
(path=cloud_run)
806         # In-process from here on ? record the actual path for the provenance stamp in
`finally`.
807         compute_actual = "in_process"
808         segmenter = segmenter_cls(db, config)
809         try:
810             # Serialize the CPU/GPU-heavy in-process detection (see
_LOCAL_COMPUTE_LANE). A remote
811             # dispatch never reaches here (it returned above), so remote polls still
overlap freely.
812             with _LOCAL_COMPUTE_LANE:
813                 result = segmenter.process_video(
814                     video_id, local_video, req.player_pool, req.max_frames
815                 )
816             except Exception:
817                 # A raising segmenter must not leave half-written rows: restore the pre-run
818                 # state (same destroy-after-confirm contract as the Failure branch), then
let
819                 # the job worker record the failure.

```

32. assistant (2026-07-07T14:29:55.909Z)

Confirmed: the control-plane validates at line 747 and **only dispatches on pass** (failures return "failed" before `\_maybe\_dispatch\_remote`). So the worker's re-validation is purely redundant. Now let me see how `ComputeJobSpec.config\_overrides` becomes worker `--set` args, to decide how to thread a skip signal.

33. user (2026-07-07T14:30:00.564Z)

===== compute package files =====

```

__init__.py
base.py
cloud_run.py

```

===== ComputeJobSpec def + config_overrides + --set construction =====

```

backend/compute/base.py:21:class ComputeJobSpec:
backend/compute/base.py:29:     # Raw worker ``--set k=v`` overrides. A tuple (not list/dict) so
the spec stays frozen/hashable.
backend/compute/base.py:30:     config_overrides: tuple[str, ...] = ()
backend/compute/base.py:51:     """Backend-agnostic compute. ``run_infer`` dispatches ONE
inference job and returns only once
backend/compute/base.py:56:     def run_infer(
backend/compute/cloud_run.py:5: 1. Dispatch a Cloud Run Job execution whose container runs
``python -m backend.worker infer
backend/compute/cloud_run.py:47:# (compute_target=local_gpu disarms the cmd_infer dispatch
guard; native = the L4 WASB runner.)
backend/compute/cloud_run.py:58:     overrides the job's container ``args`` for this execution
and starts it."""
backend/compute/cloud_run.py:113:
run_v2.RunJobRequest.Overrides.ContainerOverride(args=argv)
backend/compute/cloud_run.py:171:         # None ? a fresh uuid4 per run_infer call (prod);
injected ? deterministic (tests).
backend/compute/cloud_run.py:179:     def run_infer(

```

```

backend/compute/cloud_run.py:189:         "infer",
backend/compute/cloud_run.py:199:         for kv in (*spec.config_overrides,
*self._container_overrides):
backend/compute/cloud_run.py:200:         argv += ["--set", kv]
backend/compute/cloud_run.py:216:         label="cloud-run-infer",

```

34. user (2026-07-07T14:30:06.520Z)

```

1      """The ``ComputeBackend`` ABC ? one interface for *where* the rally core runs.
2
3      COMPUTE_DECOUPLED_SERVING M6. Mirrors the ``StorageBackend`` ABC+factory idiom
4      (``backend/storage/``): a ``ComputeBackend`` decides **where** the pure
DETECT?WINDOW?STITCH
5      core executes ? in-process on the local box (truly-local) or on a **Cloud Run GPU Job**
6      (cloud-serving) ? without the core ever branching on the profile (D1).
7
8      The seam is **default-OFF**: only ``deployment.compute_target == "cloud_run"`` selects a
9      non-local backend. Every other target (``"" / ``local_gpu`` / ``cpu_mock``) keeps the
worker's
10     existing in-process path **byte-identical** ? the factory returns ``None`` (= "run
inline").
11     """
12
13     from __future__ import annotations
14
15     from abc import ABC, abstractmethod
16     from dataclasses import dataclass, field
17     from typing import Callable, Optional
18
19
20     @dataclass(frozen=True)
21     class ComputeJobSpec:
22         """One inference job, profile-agnostic. ``video_id`` is the MD5 the worker will
(re)compute
23         from the SAME input bytes, so the output always lands at
``<out_uri>/outputs/<video_id>/`` ?
24         deterministic regardless of where the job ran."""
25
26         video_uri: str # input (gs://? for cloud; a path/local:// for local)
27         out_uri: str # output ROOT (gs://? bucket for cloud)
28         segmenter: Optional[str] = None
29         # Raw worker ``--set k=v`` overrides. A tuple (not list/dict) so the spec stays
frozen/hashable.
30         config_overrides: tuple[str, ...] = ()
31
32
33     @dataclass(frozen=True)
34     class ComputeResult:
35         """The terminal state of a dispatched job. ``returncode`` mirrors the worker's exit
code
36         (0 ok / 2 validation-or-config / 3 segmenter-failure), plus **4 = remote dispatch
timeout or an
37         empty/unreadable completion marker** (the cloud path only). ``video_id`` is known
once the job's
38         done-marker is read; ``outputs_uri`` is where ``intervals.csv`` / ``report.json``
landed."""
39
40         returncode: int
41         outputs_uri: str = ""
42         video_id: str = ""
43         report: dict = field(default_factory=dict)
44         # The backend's execution identity for PROVENANCE (e.g. the Cloud Run execution
name) ? surfaced
45         # on the job result so "the cloud path actually ran" is independently verifiable
(gccloud run jobs

```

```

46         # executions describe <execution>). Empty for the in-process path / backends without
one.
47         execution: str = ""
48
49
50     class ComputeBackend(ABC):
51         """Backend-agnostic compute. ``run_infer`` dispatches ONE inference job and returns
only once
52         its outputs are durably available (bucket-poll for cloud), so a caller treats local
and cloud
53         execution identically."""
54
55         @abstractmethod
56         def run_infer(
57             self,
58             spec: ComputeJobSpec,
59             *,
60             on_wait: "Callable[[float], None] | None" = None,
61         ) -> ComputeResult:
62             """Run ``spec`` to completion and return its ``ComputeResult``.
63
64             ``on_wait(elapsed_sec)`` (optional) is invoked periodically while the backend is
65             still WAITING on the remote work ? the caller's live-progress heartbeat (#482),
so a
66             long GPU run doesn't look frozen. Implementations without a wait loop may ignore
it."""
67

```

35. user (2026-07-07T14:30:07.122Z)

```

179     def run_infer(
180         self,
181         spec: ComputeJobSpec,
182         *,
183         on_wait: "Callable[[float], None] | None" = None,
184     ) -> ComputeResult:
185         out_root = spec.out_uri.rstrip("/")
186         token = self._token or uuid.uuid4().hex
187         done_uri = f"{out_root}/_dispatch/{token}.done"
188         argv: List[str] = [
189             "infer",
190             "--video-uri",
191             spec.video_uri,
192             "--out-uri",
193             spec.out_uri,
194             "--done-uri",
195             done_uri,
196         ]
197         if spec.segmenter:
198             argv += ["--segmenter", spec.segmenter]
199         for kv in (*spec.config_overrides, *self._container_overrides):
200             argv += ["--set", kv]
201
202         execution = self._client.run_job(
203             self._job_name, argv, region=self._region, project=self._project
204         )
205         LOG.info(
206             "cloud-run: dispatched job=%s exec=%s; bucket-polling %s",
207             self._job_name,
208             execution,
209             done_uri,
210         )
211
212         try:
213             marker = poll_until(

```

```

214         lambda: self._read_marker(done_uri),
215         self._policy,
216         label="cloud-run-infer",
217         logger=LOG,
218         # Live heartbeat (#482): each still-waiting tick reaches the caller so
219         # job.progress moves during the whole GPU run instead of freezing.
220         on_tick=on_wait,
221     )
222 except PolicyTimeout as timeout_exc:
223     marker = self._handle_poll_timeout(done_uri, str(execution), timeout_exc)

```

36. assistant (2026-07-07T14:30:29.048Z)

I have the full picture. The cloud_run backend is the only one that dispatches a worker subprocess (`get\_compute\_backend` returns `None` for local targets). I'll use a dedicated `--skip-validation` flag threaded through `ComputeJobSpec` ? cleaner than a config knob and it honestly expresses "the dispatcher already validated." Let me check the worker's arg parser and `\_dispatch\_remote` (the worker?worker recursion path) before writing.

37. user (2026-07-07T14:30:31.693Z)

```

===== infer subparser + args in worker =====
backend/worker/__main__.py:30:from backend.compute import ComputeJobSpec, get_compute_backend
backend/worker/__main__.py:174:def _dispatch_remote(compute: Any, args: argparse.Namespace) ->
int:
backend/worker/__main__.py:184:     spec = ComputeJobSpec(
backend/worker/__main__.py:233:def cmd_infer(args: argparse.Namespace) -> int:
backend/worker/__main__.py:686:     p.add_argument(
backend/worker/__main__.py:694:     pi = sub.add_parser("infer", help="run inference on one video
URI and push outputs")
backend/worker/__main__.py:695:     pi.add_argument(
backend/worker/__main__.py:698:     pi.add_argument(
backend/worker/__main__.py:703:     pi.add_argument(
backend/worker/__main__.py:706:     pi.add_argument(
backend/worker/__main__.py:709:     pi.add_argument(
backend/worker/__main__.py:712:     pi.add_argument("--max-frames", type=int, default=None)
backend/worker/__main__.py:713:     pi.add_argument(
backend/worker/__main__.py:720:     pi.add_argument(
backend/worker/__main__.py:727:     pi.set_defaults(func=cmd_infer)
backend/worker/__main__.py:729:     pt = sub.add_parser(
backend/worker/__main__.py:733:     pt.add_argument(
backend/worker/__main__.py:739:     pt.add_argument(
backend/worker/__main__.py:744:     pt.add_argument("--version", required=True, help="semver for
the artifact bundle")
backend/worker/__main__.py:745:     pt.add_argument(
backend/worker/__main__.py:753:     pt.add_argument(
backend/worker/__main__.py:758:     pt.add_argument(
backend/worker/__main__.py:761:     pt.add_argument("--config", default=None)
backend/worker/__main__.py:762:     pt.add_argument("--scratch", default=None)
backend/worker/__main__.py:763:     pt.add_argument(
backend/worker/__main__.py:769:     pt.add_argument(
backend/worker/__main__.py:775:     pt.add_argument("--set", action="append", default=[],
metavar="k=v")
backend/worker/__main__.py:776:     pt.set_defaults(func=cmd_train)

```

38. user (2026-07-07T14:30:36.796Z)

```

174     def _dispatch_remote(compute: Any, args: argparse.Namespace) -> int:
175         """M6: hand ONE infer job to a remote ComputeBackend (Cloud Run) and return its rc
once the
176         outputs are durably in the bucket (the backend bucket-polls the done-marker). The
dispatched
177         container runs this SAME cmd_infer INLINE (compute_target forced local_gpu) ? never
re-dispatch.

```



```

178
179     A remote job that never writes a completion marker (a failed/hung container) makes
the bucket
180     poll exhaust its budget ? ``PolicyTimeout``. Catch it and return a CLEAN rc 4
(remote dispatch)
181     timeout/failure, distinct from the local 2/3) rather than letting a raw traceback
escape."""
182     from backend.infrastructure.execution_policy import PolicyTimeout
183
184     spec = ComputeJobSpec(
185         video_uri=args.video_uri,
186         out_uri=args.out_uri,
187         segmenter=args.segmenter,
188         config_overrides=tuple(args.set or ()),
189     )
190     try:
191         result = compute.run_infer(spec)
192     except PolicyTimeout as e:
193         logger.warning(
194             "[worker] remote compute did not complete (no marker within budget): %s", e
195         )
196         return 4
197     logger.info(
198         "[worker] remote compute done: video_id=%s rc=%d outputs=%s",
199         result.video_id,
200         result.returncode,
201         result.outputs_uri,
202     )
203     return result.returncode

```

39. user (2026-07-07T14:30:43.312Z)

```

694     pi = sub.add_parser("infer", help="run inference on one video URI and push outputs")
695     pi.add_argument(
696         "--video-uri", required=True, help="local path / local:// / s3:// / gcs://"
697     )
698     pi.add_argument(
699         "--out-uri",
700         required=True,
701         help="output prefix (outputs/<video_id>/? is appended)",
702     )
703     pi.add_argument(
704         "--segmenter", default=None, help="default: indexing.default_segmenter"
705     )
706     pi.add_argument(
707         "--config", default=None, help="config.json path (default: ./config.json)"
708     )
709     pi.add_argument(
710         "--scratch", default=None, help="local scratch dir (default: a temp dir)"
711     )
712     pi.add_argument("--max-frames", type=int, default=None)
713     pi.add_argument(
714         "--done-uri",
715         default=None,
716         help="M6: storage URI for a completion MARKER (video_id + rc) written on a "
717         "successful run, so a remote (Cloud Run) dispatcher's bucket-poll terminates. "
718         "Default-OFF: omit it for the normal local run.",
719     )
720     pi.add_argument(
721         "--set",
722         action="append",
723         default=[],
724         metavar="k=v",
725         help="config override, e.g. --set indexing.skip_ai_handoff=true",
726     )

```

```
727         pi.set_defaults(func=cmd_infer)
728
```

40. assistant (2026-07-07T14:31:18.939Z)

I have enough to design the fix. Let me verify a couple of correctness points before writing: all call sites of `\_maybe\_dispatch\_remote` (to confirm every dispatch is post-validation), that `\_ingest\_remote\_segments` doesn't depend on the worker's report metadata, and the existing test conventions in `test\_api\_cloud\_dispatch.py`.

41. user (2026-07-07T14:31:23.717Z)

```
===== all _maybe_dispatch_remote call sites =====
backend/api/job_worker.py:137:         # v8 compute-picker: the worker runs _execute_processing ?
    _maybe_dispatch_remote, which
backend/api/job_worker.py:364:         # Same normalization the execution path applies
    (_maybe_dispatch_remote lowercases
backend/main.py:241:     _maybe_dispatch_remote,
backend/main.py:268:     # default (deployment.compute_target). Only honoured for these two
    values; see _maybe_dispatch_remote.
backend/orchestration.py:4: Cloud Run dispatch + ingest path (`_maybe_dispatch_remote` /
`_ingest_remote_segments`),
backend/orchestration.py:391: def _maybe_dispatch_remote(
backend/orchestration.py:721:     # _maybe_dispatch_remote when it dispatches. None ? nothing ran
    (validation/config fail).
backend/orchestration.py:798:         remote = _maybe_dispatch_remote(

===== _ingest_remote_segments body (does it read report.json?) =====
backend/orchestration.py:108: def _ingest_remote_segments(
backend/orchestration.py:109:     db, video_id: str, outputs_uri: str, storage_config: dict, *,
    max_rows: int = 20000
backend/orchestration.py:111:     """Read a cloud job's `intervals.csv`
    (start,end,shot_count,ending_reason) from the bucket and
backend/orchestration.py:119:     uri = f"{outputs_uri.rstrip('/')}/intervals.csv"
backend/orchestration.py:124:         uri, os.path.join(td, ref.name or "intervals.csv"),
    config=storage_config
backend/orchestration.py:235:         uri, os.path.join(td, ref.name or "report.json"),
    config=storage_config
backend/orchestration.py:243: # from an empty DB must not both ingest intervals.csv ? the loser
    waits, then finds the
backend/orchestration.py:276:         report_uri =
    f"{output_root}/outputs/{video_id}/report.json"
backend/orchestration.py:292:     the DB first, then the bucket (`outputs/<md5>/report.json`
    with `status=success`
backend/orchestration.py:321:         outputs_uri = f"{output_root}/outputs/{video_id}"

===== existing test conventions: ComputeJobSpec / overrides / skip in test file =====
tests/test_api_cloud_dispatch.py:98: def cloud_env(tmp_path, monkeypatch):
tests/test_api_cloud_dispatch.py:102:     monkeypatch.setattr(m, "db_instance", db)
tests/test_api_cloud_dispatch.py:103:     monkeypatch.setattr(m, "db_instance", db)
tests/test_api_cloud_dispatch.py:112:     monkeypatch.setattr(m, "global_config", cfg)
tests/test_api_cloud_dispatch.py:113:     monkeypatch.setattr(m, "global_config", cfg)
tests/test_api_cloud_dispatch.py:114:     monkeypatch.setattr(m, "_get_executor", lambda:
    _InlineExecutor())
tests/test_api_cloud_dispatch.py:118:     monkeypatch.setattr(
tests/test_api_cloud_dispatch.py:121:     monkeypatch.setattr(
tests/test_api_cloud_dispatch.py:124:     return db, bucket, monkeypatch
tests/test_api_cloud_dispatch.py:127: def _inject_fake(monkeypatch, fake):
tests/test_api_cloud_dispatch.py:128:     """Make backend.compute.get_compute_backend (re-
    imported inside _maybe_dispatch_remote) build a
tests/test_api_cloud_dispatch.py:131:     real = compute_pkg.get_compute_backend
tests/test_api_cloud_dispatch.py:132:     monkeypatch.setattr(
tests/test_api_cloud_dispatch.py:134:         "get_compute_backend",
tests/test_api_cloud_dispatch.py:146: def test_cloud_dispatch_ingests_segments(cloud_env):
tests/test_api_cloud_dispatch.py:147:     db, _bucket, monkeypatch = cloud_env
```

```

tests/test_api_cloud_dispatch.py:149:     _inject_fake(monkeypatch, fake)
tests/test_api_cloud_dispatch.py:165: def test_cloud_dispatch_builds_correct_spec(cloud_env):
tests/test_api_cloud_dispatch.py:166:     _db, bucket, monkeypatch = cloud_env
tests/test_api_cloud_dispatch.py:168:     _inject_fake(monkeypatch, fake)
tests/test_api_cloud_dispatch.py:185:         "indexing.skip_ai_handoff=False" in joined
tests/test_api_cloud_dispatch.py:189: def
test_cloud_dispatch_forwards_skip_ai_handoff_true(cloud_env):
tests/test_api_cloud_dispatch.py:190:     """When the engine runs no-AI (skip_ai_handoff=True),
the cloud job must too ? else it tries the
tests/test_api_cloud_dispatch.py:192:     _db, _bucket, monkeypatch = cloud_env
tests/test_api_cloud_dispatch.py:193:     m.global_config.indexing.skip_ai_handoff = True
tests/test_api_cloud_dispatch.py:195:     _inject_fake(monkeypatch, fake)
tests/test_api_cloud_dispatch.py:199:     assert "indexing.skip_ai_handoff=True" in "
".join(fake.calls[0])
tests/test_api_cloud_dispatch.py:202: def
test_cloud_dispatch_forwards_wasb_serving_knobs(cloud_env):
tests/test_api_cloud_dispatch.py:204:     the control-plane MUST forward its resolved WASB
serving knobs (decode_backend + GPU-feed
tests/test_api_cloud_dispatch.py:207:     _db, _bucket, monkeypatch = cloud_env
tests/test_api_cloud_dispatch.py:208:     m.global_config.indexing.wasb.decode_backend =
"nvdec_resident"
tests/test_api_cloud_dispatch.py:212:     _inject_fake(monkeypatch, fake)
tests/test_api_cloud_dispatch.py:217:     assert "indexing.wasb.decode_backend=nvdec_resident" in
joined
tests/test_api_cloud_dispatch.py:222: def test_cloud_dispatch_omits_unset_wasb_tuning(cloud_env):
tests/test_api_cloud_dispatch.py:223:     """decode_backend defaults to cpu (always forwarded ?
harmless, keeps worker == control-plane),
tests/test_api_cloud_dispatch.py:226:     _db, _bucket, monkeypatch = cloud_env
tests/test_api_cloud_dispatch.py:227:     # defaults: decode_backend="cpu", batch_size=None,
prefetch_batches=None
tests/test_api_cloud_dispatch.py:229:     _inject_fake(monkeypatch, fake)
tests/test_api_cloud_dispatch.py:234:     assert "indexing.wasb.decode_backend=cpu" in joined
tests/test_api_cloud_dispatch.py:239: def test_cloud_failure_marks_failed_no_rows(cloud_env):
tests/test_api_cloud_dispatch.py:240:     db, _bucket, monkeypatch = cloud_env
tests/test_api_cloud_dispatch.py:244:     _inject_fake(monkeypatch, fake)
tests/test_api_cloud_dispatch.py:259: def test_cloud_requires_a_bucket_input(cloud_env,
tmp_path):
tests/test_api_cloud_dispatch.py:260:     _db, _bucket, monkeypatch = cloud_env
tests/test_api_cloud_dispatch.py:262:     _inject_fake(monkeypatch, fake)
tests/test_api_cloud_dispatch.py:273: def test_cloud_stages_local_upload_to_bucket(cloud_env,
tmp_path, monkeypatch):
tests/test_api_cloud_dispatch.py:286:     monkeypatch.setattr(m.storage, "put", _fake_put)
tests/test_api_cloud_dispatch.py:288:     _inject_fake(monkeypatch, fake)
tests/test_api_cloud_dispatch.py:314: def
test_cloud_stages_when_input_root_is_cloud_and_input_backend_default(
tests/test_api_cloud_dispatch.py:315:     cloud_env, tmp_path, monkeypatch
tests/test_api_cloud_dispatch.py:331:     monkeypatch.setattr(m.storage, "put", _fake_put)
tests/test_api_cloud_dispatch.py:333:     _inject_fake(monkeypatch, fake)
tests/test_api_cloud_dispatch.py:351: def
test_cloud_local_upload_without_input_root_still_fails(cloud_env, tmp_path, monkeypatch):
tests/test_api_cloud_dispatch.py:356:     _inject_fake(monkeypatch, fake)
tests/test_api_cloud_dispatch.py:358:     monkeypatch.setattr(m.storage, "put", lambda *a, **k:
called.append(a))
tests/test_api_cloud_dispatch.py:368: def test_default_off_runs_in_process(tmp_path,
monkeypatch):
tests/test_api_cloud_dispatch.py:369:     """compute_target='' ? get_compute_backend returns None
? the in-process segmenter runs
tests/test_api_cloud_dispatch.py:372:     monkeypatch.setattr(m, "db_instance", db)
tests/test_api_cloud_dispatch.py:373:     monkeypatch.setattr(m, "db_instance", db)
tests/test_api_cloud_dispatch.py:375:     monkeypatch.setattr(m, "global_config", cfg)

```

42. assistant (2026-07-07T14:31:43.333Z)

Only one call site for `\_maybe\_dispatch\_remote` (orchestration.py:798, post-validation), and `\_ingest\_remote\_segments` reads only `intervals.csv`. But there's a `report.json` consumer

around lines 235?330 (restart-recovery/submit-cache) ? let me confirm skipping worker validation (empty `val\_results` in the worker's report) doesn't break it.

43. user (2026-07-07T14:31:45.086Z)

```
225         "compute": {"path": "cached", "requested": requested_compute},
226     }
227
228
229     def _read_bucket_json(uri: str, storage_config: dict) -> Optional[Dict[str, Any]]:
230         """Fetch + parse one small JSON object (cloud backends require a real dst ? the
231         ``_ingest_remote_segments`` pattern). Raises on fetch/parse errors; callers
232         decide."""
233         ref = storage.parse_uri(uri)
234         with tempfile.TemporaryDirectory(prefix="cr-report-") as td:
235             local = storage.fetch(
236                 uri, os.path.join(td, ref.name or "report.json"), config=storage_config
237             )
238             with open(local, encoding="utf-8") as f:
239                 loaded = json.load(f)
240             return loaded if isinstance(loaded, dict) else None
241
242     # One re-hydrate at a time per video_id (#492 review blocker 1): two concurrent cache
243     hits
244     # from an empty DB must not both ingest intervals.csv ? the loser waits, then finds the
245     # winner's rows in the DB and reuses them. Process-wide is enough: the CP is single-
246     instance
247     # (maxScale=1, spec D8) and SQLite serializes cross-process writers anyway.
248     _REHYDRATE_LOCKS: Dict[str, threading.Lock] = {}
249     _REHYDRATE_LOCKS_GUARD = threading.Lock()
250
251     def _rehydrate_lock(video_id: str) -> threading.Lock:
252         with _REHYDRATE_LOCKS_GUARD:
253             return _REHYDRATE_LOCKS.setdefault(video_id, threading.Lock())
254
255     def probe_process_cache(db, cfg, video_id: str) -> bool:
256         """READ-ONLY: does completed detection for ``video_id`` already exist (DB or
257         bucket)?
258
259         Safe to call before the atomic submit claim ? it writes nothing (#492 review blocker
260         1
261         ordering). Used to decide whether a submit can skip the local-scratch fetch guard
262         (blocker 2): a True probe means the request should never need CP scratch. Best-
263         effort:
264         errors report False (? the normal dispatch path with its guards)."""
265         try:
266             existing = db.get_video(video_id)
267             if (
268                 existing
269                 and existing.get("status") == "processed"
270                 and db.query_play_segments(video_id, {})
271             ):
272                 return True
273             sc = _storage_settings(cfg)
274             if not sc or (sc.get("backend") or "local") == "local":
275                 return False
276             output_root = (sc.get("output_root") or "").rstrip("/")
277             if not output_root:
278                 return False
279             report_uri = f"{output_root}/outputs/{video_id}/report.json"
280             if not storage.exists(report_uri, config=sc):
281                 return False
```

```

279         report = _read_bucket_json(report_uri, sc)
280         return bool(report and report.get("status") == "success")
281     except Exception: # noqa: BLE001 ? a failed probe degrades to the guarded dispatch
path
282         logger.warning(
283             "[API] cache probe failed for %s ? treating as a miss", video_id,
exc_info=True
284         )
285         return False
286
287
288     def cached_process_result(
289         db, cfg, video_id: str, video_path: str, requested_compute=None
290     ) -> Optional[Dict[str, Any]]:
291         """A ready-to-serve /api/process result when this video's detection ALREADY exists ?
292         the DB first, then the bucket (``outputs/<md5>/report.json`` with ``status=success``
293         re-ingests via ``_ingest_remote_segments``): **the DB is a cache, the bucket is the
294         source of truth** (#484 instant re-runs; the #485 re-hydrate mechanism). ``None`` ?
no
295         completed work found (or the check failed) ? the caller dispatches normally. The
bucket
296         branch mirrors the real cloud ingest exactly (watermarks + ``_finalize_reingest``),
so a
297         partial earlier ingest is replaced, never double-counted.
298
299         Concurrency (#492 review blocker 1): the whole check-then-ingest runs under a
300         per-``video_id`` lock ? a concurrent caller blocks, then finds the winner's rows via
the
301         in-lock DB re-check and reuses them (one segment set, ever). Callers must hold a
CLAIMED
302         job before invoking (writes happen post-claim)."""
303         try:
304             with _rehydrate_lock(video_id):
305                 existing = db.get_video(video_id)
306                 if existing and existing.get("status") == "processed":
307                     segs = db.query_play_segments(video_id, {})
308                     if segs:
309                         return _cached_process_payload(
310                             video_id,
311                             existing.get("validation_results", []),
312                             segs,
313                             requested_compute,
314                         )
315                 sc = _storage_settings(cfg)
316                 if not sc or (sc.get("backend") or "local") == "local":
317                     return None
318                 output_root = (sc.get("output_root") or "").rstrip("/")
319                 if not output_root:
320                     return None
321                 outputs_uri = f"{output_root}/outputs/{video_id}"
322                 report_uri = f"{outputs_uri}/report.json"
323                 if not storage.exists(report_uri, config=sc):
324                     return None
325                 report = _read_bucket_json(report_uri, sc)
326                 if not report or report.get("status") != "success":
327                     return None
328                 if not video_path:
329                     # Read-path re-hydrate (#485): a wiped DB has no source record. The
report
330                     # carries the worker's --video-uri (P3); older reports fall back to the
331                     # staged-canonical videos/<md5>/ prefix; else "" ? the segments still
332                     # recover, and the next same-md5 submit re-binds the truthful path via
333                     # add_video's upsert.
334                     video_path = str(report.get("video_uri") or "") or _first_object_under(
335                         f"{output_root}/videos/{video_id}/", sc

```

```

336         )
337         play_wm, cand_wm = db.segment_watermarks(video_id)
338         db.add_video(video_id, video_path, "processed", [])
339         n = _ingest_remote_segments(db, video_id, outputs_uri, sc)
340         if n <= 0:
341             return None
342         segments = [
343             s
344             for s in db.query_play_segments(video_id, {})]

```

44. assistant (2026-07-07T14:31:54.933Z)

Confirmed safe: the `report.json` consumers only read `status` and `video\_uri` (still written when validation is skipped), never `validation\_results`/`ffprobe\_meta`. Now let me read the test file's fixtures and the `FakeCompute` capture pattern so my new test matches conventions.

45. user (2026-07-07T14:31:56.811Z)

```

1     """API cloud-serving dispatch (COMPUTE_DECOUPLED_SERVING M6 ? the API path).
2
3     Proves the /api/process job path can offload DETECT to a Cloud Run GPU Job and ingest
the rally
4     intervals back into the API DB, so /api/query + /api/compile (local stitch) are
unchanged ? all
5     GPU-free + cloud-free via a FAKE CloudRunJobClient that writes report.json +
intervals.csv + the
6     done-marker into a local "bucket". Cardinal invariant: DEFAULT-OFF ? only
compute_target=cloud_run
7     dispatches; every other target runs the in-process segmenter byte-identically.
8     """
9
10    import csv
11    import json
12    import os
13
14    import pytest
15
16    pytest.importorskip("httpx")
17    from fastapi.testclient import TestClient
18
19    import backend.compute as compute_pkg
20    import backend.main as m
21    from backend.compute.cloud_run import CloudRunJobClient
22    from backend.config.models import AppConfig
23    from backend.infrastructure.database import Database
24    from backend.infrastructure.execution_policy import ExecutionPolicy
25    from backend.pipeline.validators.base import ValidationResult
26
27    _FAST = ExecutionPolicy(poll_every_n_sec=0.0, op_timeout_sec=5.0, max_wait_sec=5.0)
28
29
30    def _pass():
31        return [ValidationResult(validator_name="x", passed=True, message="ok", details={})]
32
33
34    class _InlineExecutor:
35        def submit(self, fn, *a, **k):
36            fn(*a, **k)
37
38
39    class _FakeCloudClient(CloudRunJobClient):
40        """Simulates the Cloud Run worker: on dispatch, write report.json + intervals.csv +
the done-marker
41        into the local bucket exactly where the real worker would, so the bucket-poll +
ingest resolve."""

```

```

42
43 def __init__(self, *, video_id="vidCloud", returncode=0, rows=None):
44     self.calls: list[list[str]] = []
45     self.video_id, self.returncode = video_id, returncode
46     self.rows = (
47         rows
48         if rows is not None
49         else [(2.74, 6.91, 5, "winner"), (8.74, 11.38, 3, "unforced_error")]
50     )
51
52 def run_job(self, job_name, argv, *, region, project=None):
53     self.calls.append(list(argv))
54     out_uri = argv[argv.index("--out-uri") + 1].rstrip("/")
55     done_uri = argv[argv.index("--done-uri") + 1]
56     if self.returncode == 0:
57         outputs = os.path.join(out_uri, "outputs", self.video_id)
58         os.makedirs(outputs, exist_ok=True)
59         with open(os.path.join(outputs, "report.json"), "w", encoding="utf-8") as f:
60             json.dump(
61                 {
62                     "video_id": self.video_id,
63                     "segment_count": len(self.rows),
64                     "status": "success",
65                 },
66                 f,
67             )
68         with open(
69             os.path.join(outputs, "intervals.csv"),
70             "w",
71             newline="",
72             encoding="utf-8",
73         ) as f:
74             w = csv.writer(f)
75             w.writerow(["start", "end", "shot_count", "ending_reason"])
76             for s, e, sc, er in self.rows:
77                 w.writerow([f"{s:.3f}", f"{e:.3f}", sc, er])
78         os.makedirs(os.path.dirname(done_uri), exist_ok=True)
79         with open(done_uri, "w", encoding="utf-8") as f:
80             json.dump(
81                 {
82                     "video_id": self.video_id,
83                     "returncode": self.returncode,
84                     "segment_count": len(self.rows),
85                     "status": "success",
86                 },
87                 f,
88             )
89         return "exec-1"
90
91 def cancel_execution(self, execution):
92     raise AssertionError(
93         "cancel must never fire on the happy path"
94     ) # pragma: no cover
95
96
97 @pytest.fixture
98 def cloud_env(tmp_path, monkeypatch):
99     """API wired for cloud_run with a LOCAL bucket (the local storage backend makes the
100     bucket read an
101     identity file read ? no GCS). A gs:// input is 'fetched' to a local fake file for
102     hash+validate."""
103     db = Database(str(tmp_path / "t.db")) # noqa: F841
104     monkeypatch.setattr(m, "db_instance", db)
105     monkeypatch.setattr(m, "db_instance", db)
106     bucket = tmp_path / "bucket"

```

```

105     bucket.mkdir()
106     cfg = AppConfig.model_validate(
107         { # noqa: F841
108             "deployment": {"compute_target": "cloud_run"},
109             "storage": {"backend": "local", "output_root": str(bucket)},
110         }
111     )
112     monkeypatch.setattr(m, "global_config", cfg)
113     monkeypatch.setattr(m, "global_config", cfg)
114     monkeypatch.setattr(m, "_get_executor", lambda: _InlineExecutor())
115     # A real gs:// input would be fetched to hash+validate; return a local fake so that
path is offline.
116     fake_local = tmp_path / "fetched.mp4"
117     fake_local.write_bytes(b"FAKEVIDEOBYTES")
118     monkeypatch.setattr(
119         m.storage, "acquire_local_input", lambda path, scfg: str(fake_local)
120     )
121     monkeypatch.setattr(
122         m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {})
123     )
124     return db, bucket, monkeypatch
125
126
127     def _inject_fake(monkeypatch, fake):
128         """Make backend.compute.get_compute_backend (re-imported inside
129         _maybe_dispatch_remote) build a
130         real CloudRunCompute around the FAKE client + the fast no-wait policy + a fixed
131         token. ``**kw``
132         forwards the per-request ``target_override`` (item 3) so the cloud-picker path
133         resolves too."""
134         real = compute_pkg.get_compute_backend
135         monkeypatch.setattr(
136             compute_pkg,
137             "get_compute_backend",
138             lambda config, **kw: real(
139                 config, run_client=fake, policy=_FAST, token="tok", **kw
140             ),
141         )
142
143     def _meta(row):
144         md = row.get("metadata")
145         return json.loads(md) if isinstance(md, str) else (md or {})
146
147     def test_cloud_dispatch_ingests_segments(cloud_env):
148         db, _bucket, monkeypatch = cloud_env
149         fake = _FakeCloudClient()
150         _inject_fake(monkeypatch, fake)
151
152         out = m._execute_processing(
153             m.ProcessRequest(video_path="gs://b/clip.mp4", segmenter="wasb_hybrid")
154         )
155
156         assert out["status"] == "success"
157         assert len(out["play_segments"]) == 2
158         rows = db.query_play_segments(out["video_id"], {})
159         assert len(rows) == 2
160         metas = [_meta(r) for r in rows]
161         assert all(md["source"] == "cloud_run" for md in metas)
162         assert {md.get("shot_count") for md in metas} == {5, 3}
163         assert {md.get("ending_reason") for md in metas} == {"winner", "unforced_error"}
164
165     def test_cloud_dispatch_builds_correct_spec(cloud_env):

```



```

166     _db, bucket, monkeypatch = cloud_env
167     fake = _FakeCloudClient()
168     _inject_fake(monkeypatch, fake)
169
170     m._execute_processing(
171         m.ProcessRequest(video_path="gs://b/clip.mp4", segmenter="wasb_hybrid")
172     )
173
174     assert len(fake.calls) == 1 # exactly one Cloud Run Job execution
175     argv = fake.calls[0]
176     assert argv[argv.index("--video-uri") + 1] == "gs://b/clip.mp4"
177     assert argv[argv.index("--out-uri") + 1] == str(bucket)
178     assert argv[argv.index("--segmenter") + 1] == "wasb_hybrid"
179     joined = " ".join(argv)
180     assert (
181         "deployment.compute_target=local_gpu" in joined
182     ) # container runs INLINE (never re-dispatch)
183     assert "indexing.detector_impl=native" in joined
184     assert (
185         "indexing.skip_ai_handoff=false" in joined
186     ) # the engine's AI-handoff stance is forwarded
187
188
189     def test_cloud_dispatch_forwards_skip_ai_handoff_true(cloud_env):
190         """When the engine runs no-AI (skip_ai_handoff=True), the cloud job must too ? else
191         it tries the
192         Gemini handoff (which the cloud job has no key for) and yields 0 rallies. The flag
193         is forwarded."""
194         _db, _bucket, monkeypatch = cloud_env
195         m.global_config.indexing.skip_ai_handoff = True
196         fake = _FakeCloudClient()
197         _inject_fake(monkeypatch, fake)
198         m._execute_processing(
199             m.ProcessRequest(video_path="gs://b/clip.mp4", segmenter="wasb_hybrid")
200         )
201         assert "indexing.skip_ai_handoff=true" in " ".join(fake.calls[0])
202
203     def test_cloud_dispatch_forwards_wasb_serving_knobs(cloud_env):
204         """The worker Job has no DEPLOYMENT_PROFILE, so it never applies the cloud-serving
205         preset ?
206         the control-plane MUST forward its resolved WASB serving knobs (decode_backend +
207         GPU-feed
208         tuning) or the worker silently reverts to the model defaults (cpu decode, batch 8),
209         making the
210         preset flip (#506/#507) a no-op on the real dispatch path. Regression guard for that
211         wiring."""
212         _db, _bucket, monkeypatch = cloud_env
213         m.global_config.indexing.wasb.decode_backend = "nvdec_resident"
214         m.global_config.indexing.wasb.batch_size = 64
215         m.global_config.indexing.wasb.prefetch_batches = 4
216         fake = _FakeCloudClient()
217         _inject_fake(monkeypatch, fake)
218         m._execute_processing(
219             m.ProcessRequest(video_path="gs://b/clip.mp4", segmenter="wasb_hybrid")
220         )
221         joined = " ".join(fake.calls[0])
222         assert "indexing.wasb.decode_backend=nvdec_resident" in joined
223         assert "indexing.wasb.batch_size=64" in joined
224         assert "indexing.wasb.prefetch_batches=4" in joined
225
226     def test_cloud_dispatch_omits_unset_wasb_tuning(cloud_env):
227         """decode_backend defaults to cpu (always forwarded ? harmless, keeps worker ==
228         control-plane),

```

```

224         but batch_size/prefetch_batches default to None and must NOT be forwarded (None =
the worker's
225         parity default; forwarding an empty value would reach argparse as a bad token)."""
226         _db, _bucket, monkeypatch = cloud_env
227         # defaults: decode_backend="cpu", batch_size=None, prefetch_batches=None
228         fake = _FakeCloudClient()
229         _inject_fake(monkeypatch, fake)
230         m._execute_processing(
231             m.ProcessRequest(video_path="gs://b/clip.mp4", segmenter="wasb_hybrid")
232         )
233         joined = " ".join(fake.calls[0])
234         assert "indexing.wasb.decode_backend=cpu" in joined
235         assert "indexing.wasb.batch_size" not in joined
236         assert "indexing.wasb.prefetch_batches" not in joined
237
238

```

46. assistant (2026-07-07T14:32:06.863Z)

Test conventions are clear (`\_FakeCloudClient.run\_job` captures argv ? assert on `fake.calls[0]`). Let me find the existing worker `cmd\_infer` tests to mirror for a worker-side skip test.

47. user (2026-07-07T14:32:09.663Z)

```

===== worker infer test files =====
tests/test_api_cloud_dispatch.py
tests/test_api_endpoints.py
tests/test_async_jobs.py
tests/test_compute_backend.py
tests/test_job_worker_loop.py
tests/test_profile_parity.py
tests/test_startup_checks.py
tests/test_validators_base.py
tests/test_worker.py

```

```

===== tests referencing worker cmd_infer directly =====
tests/test_compute_backend.py
tests/test_profile_parity.py
tests/test_startup_checks.py
tests/test_worker.py

```

48. user (2026-07-07T14:32:15.869Z)

```

===== test_worker.py: cmd_infer usage patterns =====
tests/test_worker.py:3:`cmd_infer` is exercised end-to-end over the LOCAL storage backend with
the validator +
tests/test_worker.py:29:def test_coerce(raw, expected):
tests/test_worker.py:33:def test_apply_overrides_on_typed_config():
tests/test_worker.py:48:def test_apply_overrides_rejects_bad_format():
tests/test_worker.py:54:def test_write_intervals_csv(tmp_path):
tests/test_worker.py:72:def test_write_report_json(tmp_path):
tests/test_worker.py:82:def test_parser_infer_accepts_set_and_uris():
tests/test_worker.py:100:# -----
cmd_infer (local, mocked)
tests/test_worker.py:149:def mocked_pipeline(monkeypatch):
tests/test_worker.py:150:     monkeypatch.setattr(wmain, "compute_video_id", lambda p: "vid123")
tests/test_worker.py:151:     monkeypatch.setattr(
tests/test_worker.py:153:         "run_all_with_context",
tests/test_worker.py:156:     monkeypatch.setattr(wmain.segmenter_registry, "get", lambda name:
_FakeSeg)
tests/test_worker.py:157:     monkeypatch.setattr(wmain, "emit_indexer_report", lambda **k: None)
tests/test_worker.py:160:def test_cmd_infer_local_roundtrip(tmp_path, mocked_pipeline):
tests/test_worker.py:218:def test_setup_logging_levels():

```

```

tests/test_worker.py:227: def test_cmd_infer_zero_segments_is_logged_loudly(tmp_path,
monkeypatch, caplog):
tests/test_worker.py:229:     monkeypatch.setattr(wmain, "compute_video_id", lambda p: "vidZ")
tests/test_worker.py:230:     monkeypatch.setattr(
tests/test_worker.py:232:         "run_all_with_context",
tests/test_worker.py:235:     monkeypatch.setattr(wmain.segmenter_registry, "get", lambda name:
_EmptySeg)
tests/test_worker.py:236:     monkeypatch.setattr(wmain, "emit_indexer_report", lambda **k: None)
tests/test_worker.py:237:     monkeypatch.setenv("RALLY_DETECTOR_IMPL", "native")
tests/test_worker.py:242:     # Call cmd_infer directly (not via main(), whose _setup_logging
force-reconfigures handlers and
tests/test_worker.py:243:     # would detach caplog's). The warning is emitted by cmd_infer
regardless of who configures logging.
tests/test_worker.py:258:         rc = wmain.cmd_infer(args)
tests/test_worker.py:265:     # (contrast test_cmd_infer_detector_failure_*, where a Failure
surfaces as status="failed").
tests/test_worker.py:270: def test_cmd_infer_detector_failure_marks_report_and_marker_failed(
tests/test_worker.py:271:     tmp_path, monkeypatch
tests/test_worker.py:274:     as a 0-segment "success". cmd_infer returns rc 3 and writes
report.json + the done-marker with
tests/test_worker.py:276:     monkeypatch.setattr(wmain, "compute_video_id", lambda p: "vidX")
tests/test_worker.py:277:     monkeypatch.setattr(
tests/test_worker.py:279:         "run_all_with_context",
tests/test_worker.py:282:     monkeypatch.setattr(wmain.segmenter_registry, "get", lambda name:
_FailSeg)
tests/test_worker.py:283:     monkeypatch.setattr(wmain, "emit_indexer_report", lambda **k: None)
tests/test_worker.py:304:     # A detector failure is rc 3 (distinct from validation's rc 2 and a
clean rc 0).
tests/test_worker.py:316: def
test_cmd_infer_real_trajectory_segmenter_failure_flows_to_report(tmp_path, monkeypatch):
tests/test_worker.py:318:     through cmd_infer -> _fail(3) -> a status="failed" report.json +
done-marker. Locks the two
tests/test_worker.py:329:     monkeypatch.setattr(WasbHybridSegmenter, "_resolve_runner", lambda
self, cfg: (runner, {}))
tests/test_worker.py:330:     monkeypatch.setattr(
tests/test_worker.py:333:     monkeypatch.setattr(
tests/test_worker.py:336:     monkeypatch.setattr(wmain, "compute_video_id", lambda p: "vidR")
tests/test_worker.py:337:     monkeypatch.setattr(
tests/test_worker.py:338:         wmain.validator_registry, "run_all_with_context", lambda path,
cfg: ([_OKResult()], {}))
tests/test_worker.py:340:     monkeypatch.setattr(wmain.segmenter_registry, "get", lambda name:
WasbHybridSegmenter)
tests/test_worker.py:341:     monkeypatch.setattr(wmain, "emit_indexer_report", lambda **k: None)
tests/test_worker.py:372: def test_cmd_infer_validation_failure_returns_2(tmp_path, monkeypatch):
tests/test_worker.py:373:     monkeypatch.setattr(wmain, "compute_video_id", lambda p: "vidF")
tests/test_worker.py:374:     monkeypatch.setattr(
tests/test_worker.py:376:         "run_all_with_context",
tests/test_worker.py:379:     monkeypatch.setattr(wmain, "emit_indexer_report", lambda **k: None)
tests/test_worker.py:402:     # A validation failure is ALSO written as an explicit failure (not
a missing artifact / false
tests/test_worker.py:433: def test_label_clip_name_strips_date_prefix(fname, expected):
tests/test_worker.py:438: def test_cmd_train_detector_joins_dated_labels_to_bare_videos(tmp_path,
monkeypatch):
tests/test_worker.py:446:     monkeypatch.setenv("RALLY_DETECTOR_IMPL", "stub") # CPU stub
runner (no GPU/WSL)
tests/test_worker.py:447:     monkeypatch.setattr(wmain, "_probe_video_fps_width", lambda p:
(30.0, 1280.0))
tests/test_worker.py:479:     monkeypatch.setattr(subprocess, "run", fake_run)
tests/test_worker.py:509: def
test_cmd_train_detector_skips_label_with_no_matching_video(tmp_path, monkeypatch):
tests/test_worker.py:515:     monkeypatch.setenv("RALLY_DETECTOR_IMPL", "stub")
tests/test_worker.py:516:     monkeypatch.setattr(wmain, "_probe_video_fps_width", lambda p:
(30.0, 1280.0))

```

Let me read the parser test, the args helper, and the validation-failure test to mirror conventions precisely.

50. user (2026-07-07T14:32:24.782Z)

```
82     def test_parser_infer_accepts_set_and_uris():
83         args = wmain.build_parser().parse_args(
84             [
85                 "infer",
86                 "--video-uri",
87                 "s3://b/v.mp4",
88                 "--out-uri",
89                 "s3://b",
90                 "--set",
91                 "indexing.skip_ai_handoff=true",
92                 "--set",
93                 "sport=tennis",
94             ]
95         )
96         assert args.cmd == "infer" and args.video_uri == "s3://b/v.mp4"
97         assert args.set == ["indexing.skip_ai_handoff=true", "sport=tennis"]
98
99
100     # ----- cmd_infer (local,
mocked)
101
102
103     class _OKResult:
104         passed = True
105         validator_name = "fake"
106         message = "ok"
107
108         def model_dump(self):
109             return {"validator_name": "fake", "passed": True, "message": "ok"}
110
111
112     class _FailResult(_OKResult):
113         passed = False
114         message = "too blurry"
115
116
117     class _FakeSeg:
118         def __init__(self, db, config):
119             pass
120
121         def process_video(self, video_id, path, max_frames=None):
122             return [
123                 {
124                     "start_time": 2.0,
125                     "end_time": 5.0,
126                     "shot_count": 4,
127                     "shot_count_confidence": "LocalNoAI",
128                 },
129                 {
130                     "start_time": 9.0,
131                     "end_time": 12.0,
132                     "shot_count": 6,
133                     "shot_count_confidence": "LocalNoAI",
134                 },
135             ]
136
137
138     def _fresh_config():
139         # Load defaults without touching any cwd config.json.
140         import tempfile
```

```

141
142     p = tempfile.NamedTemporaryFile("w", suffix=".json", delete=False)
143     p.write("{}")
144     p.close()
145     return load_config(p.name)
146
147
148 @pytest.fixture
149 def mocked_pipeline(monkeypatch):
150     monkeypatch.setattr(wmain, "compute_video_id", lambda p: "vid123")
151     monkeypatch.setattr(
152         wmain.validator_registry,
153         "run_all_with_context",
154         lambda path, cfg: ([_OKResult()], {}),
155     )
156     monkeypatch.setattr(wmain.segmenter_registry, "get", lambda name: _FakeSeg)
157     monkeypatch.setattr(wmain, "emit_indexer_report", lambda **k: None)
158
159
160 def test_cmd_infer_local_roundtrip(tmp_path, mocked_pipeline):
161     video = tmp_path / "in.mp4"
162     video.write_bytes(b"FAKEVIDEO")
163     cfg = tmp_path / "config.json"
164     cfg.write_text("{}")
165     out = tmp_path / "bucket"
166     scratch = tmp_path / "scratch"
167
168     rc = wmain.main(
169         [
170             "infer",
171             "--video-uri",
172             str(video),
173             "--out-uri",
174             str(out),
175             "--config",
176             str(cfg),
177             "--scratch",
178             str(scratch),
179             "--segmenter",
180             "wasb_hybrid",
181             "--set",
182             "indexing.skip_ai_handoff=true",
183         ]
184     )
185     assert rc == 0
186     intervals = out / "outputs" / "vid123" / "intervals.csv"
187     report = out / "outputs" / "vid123" / "report.json"
188     assert intervals.exists() and report.exists()
189     rows = intervals.read_text().splitlines()
190     assert len(rows) == 3 # header + 2 rallies
191     assert json.loads(report.read_text())["segment_count"] == 2
192
193
194 class _EmptySeg:
195     """A segmenter that finds nothing ? a LEGITIMATE 0-rally match (a bare [])."""
196
197     def __init__(self, db, config):
198         pass
199
200     def process_video(self, video_id, path, max_frames=None):
201         return []
202
203
204 class _FailSeg:
205     """A segmenter whose DETECTOR failed/timed out ? returns a Failure (e.g. WASB

```

```

produced no
206     trajectory)), which is NOT the same as a 0-rally match. This is the wf294 shape."""
207
208     def __init__(self, db, config):
209         pass
210
211     def process_video(self, video_id, path, max_frames=None):
212         return Failure(
213             message="WASB detector produced no trajectory (timed out or aborted)",
214             is_recoverable=True,
215         )
216
217
218     def test_setup_logging_levels():
219         import logging
220
221         wmain._setup_logging(False)
222         assert logging.getLogger().level == logging.INFO
223         wmain._setup_logging(True)
224         assert logging.getLogger().level == logging.DEBUG
225
226
227     def test_cmd_infer_zero_segments_is_logged_loudly(tmp_path, monkeypatch, caplog):
228         # The previous silent "0 segment(s)" gave nothing to analyse (the real cold-start
bug).
229         monkeypatch.setattr(wmain, "compute_video_id", lambda p: "vidZ")
230         monkeypatch.setattr(
231             wmain.validator_registry,
232             "run_all_with_context",
233             lambda path, cfg: ([_OKResult()], {}),
234         )
235         monkeypatch.setattr(wmain.segmenter_registry, "get", lambda name: _EmptySeg)
236         monkeypatch.setattr(wmain, "emit_indexer_report", lambda **k: None)
237         monkeypatch.setenv("RALLY_DETECTOR_IMPL", "native")
238         video = tmp_path / "in.mp4"
239         video.write_bytes(b"FAKE")
240         out = tmp_path / "bucket"
241
242         # Call cmd_infer directly (not via main()), whose _setup_logging force-reconfigures
handlers and
243         # would detach caplog's). The warning is emitted by cmd_infer regardless of who
configures logging.
244         args = wmain.build_parser().parse_args(
245             [
246                 "infer",
247                 "--video-uri",
248                 str(video),
249                 "--out-uri",
250                 str(out),
251                 "--scratch",
252                 str(tmp_path / "s"),
253                 "--segmenter",
254                 "wasb_hybrid",
255             ]
256         )
257         with caplog.at_level("WARNING", logger="backend.worker"):
258             rc = wmain.cmd_infer(args)
259             assert rc == 0 # still a clean run, just empty ? but now EXPLAINED
260             msgs = " ".join(r.getMessage() for r in caplog.records)
261             assert "0 segments" in msgs and "vidZ" in msgs
262             assert "detector_impl=native" in msgs # surfaces the resolved detector
263             assert "native runner UNAVAILABLE" in msgs # points at the likely cause
264             # A segmenter that returns a bare [] is a LEGITIMATE 0-rally match ? status stays
"success"
265             # (contrast test_cmd_infer_detector_failure_*, where a Failure surfaces as

```

```

status="failed").
266     report = json.loads((out / "outputs" / "vidZ" / "report.json").read_text())
267     assert report["segment_count"] == 0 and report["status"] == "success"
268
269
270     def test_cmd_infer_detector_failure_marks_report_and_marker_failed(
271         tmp_path, monkeypatch
272     ):
273         """wf294 fix: a segmenter that returns a Failure (detector timeout/abort) must NOT
be written
274         as a 0-segment "success". cmd_infer returns rc 3 and writes report.json + the done-
marker with
275         status="failed" so the control plane / alpha-user surface can show an error /
retry."""
276         monkeypatch.setattr(wmain, "compute_video_id", lambda p: "vidX")
277         monkeypatch.setattr(
278             wmain.validator_registry,
279             "run_all_with_context",
280             lambda path, cfg: ([_OKResult()], {}),
281         )
282         monkeypatch.setattr(wmain.segmenter_registry, "get", lambda name: _FailSeg)
283         monkeypatch.setattr(wmain, "emit_indexer_report", lambda **k: None)
284         video = tmp_path / "in.mp4"
285         video.write_bytes(b"FAKE")
286         out = tmp_path / "bucket"
287         done = tmp_path / "bucket" / "_dispatch" / "tok.done"
288
289         rc = wmain.main(
290             [
291                 "infer",
292                 "--video-uri",
293                 str(video),
294                 "--out-uri",
295                 str(out),
296                 "--scratch",
297                 str(tmp_path / "s"),
298                 "--segmenter",
299                 "wasb_hybrid",

```

51. user (2026-07-07T14:32:30.741Z)

```

372     def test_cmd_infer_validation_failure_returns_2(tmp_path, monkeypatch):
373         monkeypatch.setattr(wmain, "compute_video_id", lambda p: "vidF")
374         monkeypatch.setattr(
375             wmain.validator_registry,
376             "run_all_with_context",
377             lambda path, cfg: ([_FailResult()], {}),
378         )
379         monkeypatch.setattr(wmain, "emit_indexer_report", lambda **k: None)
380         video = tmp_path / "in.mp4"
381         video.write_bytes(b"X")
382         cfg = tmp_path / "config.json"
383         cfg.write_text("{}")
384         out = tmp_path / "o"
385         done = tmp_path / "o" / "_dispatch" / "tok.done"
386         rc = wmain.main(
387             [
388                 "infer",
389                 "--video-uri",
390                 str(video),
391                 "--out-uri",
392                 str(out),
393                 "--config",
394                 str(cfg),
395                 "--scratch",

```

```

396         str(tmp_path / "s"),
397         "--done-uri",
398         str(done),
399     ]
400 )
401 assert rc == 2
402 # A validation failure is ALSO written as an explicit failure (not a missing
artifact / false
403 # success): report.json status="failed" + the done-marker rc 2 / status="failed".
404 report = json.loads((out / "outputs" / "vidF" / "report.json").read_text())
405 assert report["status"] == "failed" and report["segment_count"] == 0
406 marker = json.loads(done.read_text())
407 assert marker["returncode"] == 2 and marker["status"] == "failed"
408
409
410 # --- cmd_train: pull labels -> synth trajectories -> manifest -> harness -> push bundle
---
411
412
413 @pytest.mark.parametrize(
414     "fname,expected",
415     [
416         (
417             "2026-06-22_GX020094.rallies.csv",
418             "GX020094",
419         ), # canonical dated label (DATA_IN_GCS §7.1)
420         ("GX020094.rallies.csv", "GX020094"), # legacy bare label -> SAME clip stem
421         (
422             "GX020094_proxy.rallies.csv",
423             "GX020094_proxy",
424         ), # _proxy is part of the name, kept
425         (
426             "2026-06-21_mahadevpura_1.rallies.csv",
427             "mahadevpura_1",
428         ), # underscores in <name> survive
429         ("v1.csv", "v1"), # bare .csv suffix
430         ("2026-06-22_GX020094.csv", "GX020094"), # dated + bare .csv
431     ],
432 )
433 def test_label_clip_name_strips_date_prefix(fname, expected):

```

52. assistant (2026-07-07T14:33:21.461Z)

I now have complete understanding. Let me mark the implementation phase and make the edits. The plan (skip-on-dispatch via a dedicated `--skip-validation` flag):

1. `ComputeJobSpec.skip\_validation` field
2. `cloud\_run.py` emits `--skip-validation` when set
3. Worker: parser arg + `cmd\_infer` guard + propagate through `\_dispatch\_remote`
4. `orchestration.py` sets `skip\_validation=True` on dispatch
5. Tests (dispatch + worker-side)

53. user (2026-07-07T14:33:22.159Z)

Chapter marked: "Implement skip-on-dispatch fix". Continue your current work.

54. assistant (2026-07-07T14:33:25.980Z)

Now the edits. Starting with `ComputeJobSpec`:

55. user (2026-07-07T14:33:32.519Z)

The file C:\Users\avidu\Projects\khelsutra-guru\bhi-wt-worker-reval\backend\compute\base.py has been updated successfully. (file state is current in your context ? no need to Read it back)

56. user (2026-07-07T14:33:39.513Z)

The file C:\Users\avidu\Projects\khelsutra-guru\bhi-wt-worker-reval\backend\compute\cloud_run.py has been updated successfully. (file state is current in your context ? no need to Read it back)

57. assistant (2026-07-07T14:33:44.471Z)

Now the worker: the `cmd_infer` validation guard, the parser arg, and `_dispatch_remote` propagation.

58. user (2026-07-07T14:33:50.824Z)

The file C:\Users\avidu\Projects\khelsutra-guru\bhi-wt-worker-reval\backend\worker__main__.py has been updated successfully. (file state is current in your context ? no need to Read it back)

59. user (2026-07-07T14:33:56.942Z)

The file C:\Users\avidu\Projects\khelsutra-guru\bhi-wt-worker-reval\backend\worker__main__.py has been updated successfully. (file state is current in your context ? no need to Read it back)

60. user (2026-07-07T14:34:04.690Z)

The file C:\Users\avidu\Projects\khelsutra-guru\bhi-wt-worker-reval\backend\worker__main__.py has been updated successfully. (file state is current in your context ? no need to Read it back)

61. assistant (2026-07-07T14:34:08.823Z)

Now the dispatcher sets `skip_validation=True`:

62. user (2026-07-07T14:34:14.208Z)

The file C:\Users\avidu\Projects\khelsutra-guru\bhi-wt-worker-reval\backend\orchestration.py has been updated successfully. (file state is current in your context ? no need to Read it back)

63. assistant (2026-07-07T14:34:21.780Z)

Now the tests. First the dispatch-side test (assert `--skip-validation` reaches the worker argv):

64. user (2026-07-07T14:34:28.980Z)

The file C:\Users\avidu\Projects\khelsutra-guru\bhi-wt-worker-reval\tests\test_api_cloud_dispatch.py has been updated successfully. (file state is current in your context ? no need to Read it back)

65. assistant (2026-07-07T14:34:53.713Z)

Now the worker-side tests ? parser acceptance and a `cmd_infer` skip test proving the validators are never invoked:

66. user (2026-07-07T14:34:57.851Z)

The file C:\Users\avidu\Projects\khelsutra-guru\bhi-wt-worker-reval\tests\test_worker.py has been updated successfully. (file state is current in your context ? no need to Read it back)

67. user (2026-07-07T14:35:10.842Z)

The file C:\Users\avidu\Projects\khelsutra-guru\bhi-wt-worker-reval\tests\test_worker.py has been updated successfully. (file state is current in your context ? no need to Read it back)

68. assistant (2026-07-07T14:35:24.056Z)

All code and tests are written. Now let me set up to run them. Let me find the Python

environment with the repo's deps installed (the fresh worktree has no venv of its own).

69. user (2026-07-07T14:35:29.834Z)

```
===== python =====
Python 3.13.13
/c/Users/avidu/AppData/Local/Programs/Python/Python313/python
===== venvs in main checkout =====
===== deps check (system python) =====
core deps OK
===== ruff/mypy present? =====
ruff 0.15.16
mypy 2.1.0 (compiled: yes)
```

70. assistant (2026-07-07T14:35:36.959Z)

System Python 3.13 has the deps, and ruff/mypy match CI versions exactly. Let me run the targeted tests for my changes first.

71. user (2026-07-07T14:36:47.049Z)

```
..... [ 84%]
..... [100%]
===== warnings summary =====
tests/test_api_cloud_dispatch.py::test_full_api_process_to_query_via_cloud
tests/test_api_cloud_dispatch.py::test_full_api_process_to_query_via_cloud
tests/test_api_cloud_dispatch.py::test_local_override_survives_api_boundary
tests/test_api_cloud_dispatch.py::test_local_override_survives_api_boundary
tests/test_api_cloud_dispatch.py::test_cloud_override_survives_api_boundary_on_local_server
tests/test_api_cloud_dispatch.py::test_cloud_override_survives_api_boundary_on_local_server
tests/test_api_cloud_dispatch.py::test_full_api_process_job_result_carries_cloud_provenance
tests/test_api_cloud_dispatch.py::test_full_api_process_job_result_carries_cloud_provenance
C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\site-
packages\google\auth\_default.py:113: UserWarning: Your application has authenticated using end
user credentials from Google Cloud SDK without a quota project. You might receive a "quota
exceeded" or "API not enabled" error. See the following page for troubleshooting:
https://cloud.google.com/docs/authentication/adc-troubleshooting/user-creds.
  warnings.warn(_CLOUD_SDK_CREDENTIALS_WARNING)

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
85 passed, 8 warnings in 64.56s (0:01:04)
```

72. assistant (2026-07-07T14:36:56.696Z)

All 85 targeted tests pass (including my 3 new ones). Now the pre-merge gate. Let me run ruff and mypy first (fast), then the full suite with coverage.

73. user (2026-07-07T14:38:20.329Z)

```
===== RUFF =====
All checks passed!
EXIT_RUFF=0

===== MYPY (backend training) =====
backend\annotations\base.py:88: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs [annotation-unchecked]
backend\sports\_init__.py:9: note: By default the bodies of untyped functions are not checked,
consider using --check-untyped-defs [annotation-unchecked]
backend\pipeline\validators\base.py:44: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs [annotation-unchecked]
backend\providers\_init__.py:9: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs [annotation-unchecked]
Success: no issues found in 148 source files
EXIT_MYPY=0
```

74. assistant (2026-07-07T14:38:28.366Z)

Ruff and mypy both clean. Now the full suite with coverage. Let me check how `run\_all\_tests.py` runs so I invoke it correctly.

75. user (2026-07-07T14:38:30.729Z)

```
#!/usr/bin/env python
```

```
"""Run the FULL test suite for the badminton-highlight-indexer.
```

A single, discoverable entry point so you (or a reviewer / CI) can verify the whole suite before opening a PR:

```
python run_all_tests.py          # full suite, quiet
python run_all_tests.py -v       # verbose (passthrough to pytest)
python run_all_tests.py -k config # filter by keyword (passthrough)
python run_all_tests.py --cov=backend --cov-report=term-missing # coverage
```

Anything after the script name is passed straight through to pytest.

Notes

- The suite is fully mocked (no GPU, no network, no WSL) and runs offline.
- The observability **reporting** tests skip cleanly if the optional `obsreport` package isn't installed (they use `pytest.importorskip`). To exercise them too:
 `pip install -e ../sports-obsreport --no-build-isolation`
- Exit code is pytest's own, so this is CI / pre-PR friendly (0 = all green).

```
from __future__ import annotations
```

```
import importlib.util
import subprocess
import sys
```

```
def _installed(module: str) -> bool:
    try:
        return importlib.util.find_spec(module) is not None
    except (ImportError, ValueError):
        return False
```

```
def main() -> int:
    args = list(sys.argv[1:])
    # Default to quiet unless the caller chose a verbosity.
    if not any(a in ("-q", "-v", "-vv", "-vvv", "--quiet", "--verbose") for a in args):
        args = ["-q", *args]
    cmd = [sys.executable, "-m", "pytest", *args]

    obs = (
        "available"
        if _installed("obsreport")
        else "NOT installed ? reporting tests will SKIP"
    )
    bar = "=" * 72
    print(bar)
    print(" Full test suite: " + " ".join(cmd))
    print(f" obsreport (optional reporting dep): {obs}")
    print(bar, flush=True)

    rc = subprocess.run(cmd).returncode

    print(bar)
    print(
```

```

" RESULT: "
+ ("PASS - all tests green" if rc == 0 else f"FAIL - pytest exit {rc}")
...
run_all_tests.py:8:    python run_all_tests.py -v          # verbose (passthrough to pytest)
run_all_tests.py:10:   python run_all_tests.py --cov=backend --cov-report=term-missing #
coverage
run_all_tests.py:12:Anything after the script name is passed straight through to pytest.
run_all_tests.py:18:  package isn't installed (they use ``pytest.importorskip``). To exercise
them too:
run_all_tests.py:20:- Exit code is pytest's own, so this is CI / pre-PR friendly (0 = all
green).
run_all_tests.py:37:def main() -> int:
run_all_tests.py:42:    cmd = [sys.executable, "-m", "pytest", *args]
run_all_tests.py:60:    + ("PASS - all tests green" if rc == 0 else f"FAIL - pytest exit
{rc}")

```

76. assistant (2026-07-07T14:38:43.451Z)

`run\_all\_tests.py` passes through to pytest; coverage is opt-in. Let me check the optional `obsreport` dep (so reporting tests don't silently skip) and any pytest config, then launch the full suite with coverage.

77. user (2026-07-07T14:38:48.273Z)

```

===== obsreport installed? =====
AAAAAAAAAAAAAAAAAAAA
ModuleNotFoundError: No module named 'obsreport'
===== sibling pkg present? =====
/c/Users/avidu/Projects/khelsutra-guru/sports-obsreport
===== pytest config (cov floor / addopts) =====
pytest.ini:1:[pytest]

```

78. assistant (2026-07-07T14:38:58.301Z)

obsreport isn't installed but its sibling package exists ? per my LGTM gate I should install it so the reporting tests actually run (relevant here since my skip path passes empty `val\_context` into reporting). Let me read `pytest.ini` and install obsreport.

79. user (2026-07-07T14:38:59.315Z)

```

1    [pytest]
2    # When `pytest` is invoked without an explicit path, only collect from tests/.
3    # scratch/ holds debugging scripts that hit real APIs (Gemini upload, etc.)
4    # and should never run as part of the regular test suite.
5    testpaths = tests
6

```

80. user (2026-07-07T14:39:10.813Z)

```

[notice] A new release of pip is available: 26.0.1 -> 26.1.2
[notice] To update, run: python.exe -m pip install --upgrade pip
---verify---
obsreport OK C:\Users\avidu\Projects\khelsutra-guru\sports-obsreport\src\obsreport\__init__.py

```

81. assistant (2026-07-07T14:39:29.035Z)

obsreport installed; `pytest.ini` only sets `testpaths` (the coverage floor is a CI flag). Let me run the full suite with coverage in the background (it's ~1700+ tests).

82. user (2026-07-07T14:39:30.444Z)

Command running in background with ID: b7biy10f1. Output is being written to:
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\902e27a6-e026-4b55-8c26-f2c47329cc44\tasks\b7biy10f1.output. You will be notified when

it completes. To check interim output, use Read on that file path.

83. assistant (2026-07-07T14:39:37.865Z)

While the suite runs, let me review my complete diff for coherence.

84. user (2026-07-07T14:39:39.979Z)

```
backend/compute/base.py          | 7 ++++++
backend/compute/cloud_run.py     | 4 ++++
backend/orchestration.py         | 7 ++++++
backend/worker/__main__.py       | 32 ++++++-----
tests/test_api_cloud_dispatch.py | 16 ++++++
tests/test_worker.py             | 45 ++++++
6 files changed, 107 insertions(+), 4 deletions(-)
===== FULL DIFF =====
diff --git a/backend/compute/base.py b/backend/compute/base.py
index 279606e..31af3fe 100644
--- a/backend/compute/base.py
+++ b/backend/compute/base.py
@@ -28,6 +28,13 @@ class ComputeJobSpec:
     segmenter: Optional[str] = None
     # Raw worker ``--set k=v`` overrides. A tuple (not list/dict) so the spec stays
    frozen/hashable.
    config_overrides: tuple[str, ...] = ()
+   # Skip the worker's redundant re-validation. The DISPATCHER (the cloud-serving control-
+   plane) is
+   # the authoritative validation gate: it runs the SAME validators with ITS resolved config
+   and only
+   # dispatches on PASS, so a worker re-run is (a) redundant, (b) a second full-video decode,
+   and
+   # (c) outright WRONG when the worker's config resolves different thresholds ? it can REJECT
+   a clip
+   # the gate already passed. Default False keeps the direct ``worker infer`` CLI (no control-
+   plane)
+   # validating exactly as before.
+   skip_validation: bool = False

    @dataclass(frozen=True)
diff --git a/backend/compute/cloud_run.py b/backend/compute/cloud_run.py
index 841785d..7247ace 100644
--- a/backend/compute/cloud_run.py
+++ b/backend/compute/cloud_run.py
@@ -196,6 +196,10 @@ class CloudRunCompute(ComputeBackend):
    ]
    if spec.segmenter:
        argv += ["--segmenter", spec.segmenter]
+   if spec.skip_validation:
+       # The control-plane already validated + only dispatches on PASS ? tell the worker
+       not to
+       # re-decode and re-validate (redundant, and its config may resolve different
+       thresholds).
+       argv.append("--skip-validation")
    for kv in (*spec.config_overrides, *self._container_overrides):
        argv += ["--set", kv]

diff --git a/backend/orchestration.py b/backend/orchestration.py
index 5e9f510..79b4fed 100644
--- a/backend/orchestration.py
+++ b/backend/orchestration.py
@@ -568,11 +568,18 @@ def _maybe_dispatch_remote(
    prefetch = getattr(wasb_cfg, "prefetch_batches", None)
    if prefetch:
        overrides.append(f"indexing.wasb.prefetch_batches={prefetch}")
```

```

+ # Tell the worker to SKIP its redundant re-validation: THIS control-plane is the
authoritative
+ # gate ? it already ran the validators with its resolved config just above (only a PASS
reaches
+ # here; a failure returned "failed" before dispatch). A worker re-run would decode the
video a
+ # second time and, because its config isn't fully forwarded, can resolve DIFFERENT
thresholds and
+ # REJECT a clip this gate already passed (observed 2026-07-07: passed at
min_blur_threshold=8.0,
+ # then worker-rejected at its default 20.0 after a successful GPU dispatch).
spec = ComputeJobSpec(
    video_uri=req.video_path,
    out_uri=out_root,
    segmenter=seg_name,
    config_overrides=tuple(overrides),
+ skip_validation=True,
)
try:
    # Dispatch the Cloud Run Job + bucket-poll to completion. on_wait is the live
diff --git a/backend/worker/__main__.py b/backend/worker/__main__.py
index 88c50f6..9b19215 100644
--- a/backend/worker/__main__.py
+++ b/backend/worker/__main__.py
@@ -186,6 +186,7 @@ def _dispatch_remote(compute: Any, args: argparse.Namespace) -> int:
    out_uri=args.out_uri,
    segmenter=args.segmenter,
    config_overrides=tuple(args.set or ()),
+ skip_validation=bool(getattr(args, "skip_validation", False)),
)
try:
    result = compute.run_infer(spec)
@@ -259,10 +260,25 @@ def cmd_infer(args: argparse.Namespace) -> int:
    # 2. IDENTITY ? whole-file MD5 (the labels?video join key; pulled bytes must be byte-
identical).
    video_id = compute_video_id(local_video)

- # 3. VALIDATE (same registry as the main CLI).
- val_results, val_context = validator_registry.run_all_with_context(
-     local_video, config
- )
+ # 3. VALIDATE (same registry as the main CLI) ? UNLESS the dispatcher already did.
+ # On the cloud-serving path the CONTROL-PLANE is the authoritative validation gate: it runs
the
+ # SAME validators with ITS resolved config and only dispatches on PASS. Re-validating here
is
+ # redundant, decodes the whole video a SECOND time, and is outright WRONG when the worker's
config
+ # resolves different thresholds than the control-plane's ? observed 2026-07-07: a 5312x2988
clip
+ # (sharpness 11.9) PASSED the control-plane at validation.min_blur_threshold=8.0, then the
worker
+ # REJECTED it at its own default 20.0, so a SUCCESSFUL GPU dispatch failed after the fact.
+ # --skip-validation (set by the dispatcher in orchestration._maybe_dispatch_remote) skips
it; the
+ # direct ``worker infer`` CLI has no control-plane, so it still validates by default.
+ if getattr(args, "skip_validation", False):
+     print(
+         "[worker] validation SKIPPED (--skip-validation): the dispatcher is the
authoritative "
+         "gate and already validated this input."
+     )
+     val_results, val_context = [], {}
+ else:
+     val_results, val_context = validator_registry.run_all_with_context(

```

```

+         local_video, config
+     )
+     failures = [r for r in val_results if not r.passed]
+     db = Database(os.path.join(scratch, config.output.db_name))
+     db.add_video(
@@ -724,6 +740,14 @@ def build_parser() -> argparse.ArgumentParser:
+         metavar="k=v",
+         help="config override, e.g. --set indexing.skip_ai_handoff=true",
+     )
+     pi.add_argument(
+         "--skip-validation",
+         action="store_true",
+         help="skip the worker's re-validation: the dispatcher (control-plane) already ran the "
+         "validators with its resolved config and only dispatches on PASS, so re-validating here "
+         "is "
+         "redundant, decodes the video again, and can contradict the gate. Default-OFF: the "
+         "direct "
+         "`worker infer` CLI still validates.",
+     )
+     pi.set_defaults(func=cmd_infer)

+     pt = sub.add_parser(
diff --git a/tests/test_api_cloud_dispatch.py b/tests/test_api_cloud_dispatch.py
index 8382234..19c37d9 100644
--- a/tests/test_api_cloud_dispatch.py
+++ b/tests/test_api_cloud_dispatch.py
@@ -236,6 +236,22 @@ def test_cloud_dispatch_omits_unset_wasb_tuning(cloud_env):
+     assert "indexing.wasb.prefetch_batches" not in joined

+def test_cloud_dispatch_skips_worker_revalidation(cloud_env):
+    """The control-plane is the AUTHORITATIVE validation gate: it validated + only dispatches
+    on PASS
+    (registry.run_all_with_context is stubbed to pass in cloud_env). Re-validating on the
+    worker is
+    redundant, decodes the video a second time, and ? because the control-plane's validation
+    config
+    isn't forwarded ? can REJECT a clip the gate already passed (observed 2026-07-07: passed at
+    validation.min_blur_threshold=8.0, then worker-rejected at its default 20.0). So the
+    dispatch MUST
+    carry --skip-validation, telling the worker to trust the gate. Regression guard for that
+    wiring."""
+    _db, _bucket, monkeypatch = cloud_env
+    fake = _FakeCloudClient()
+    _inject_fake(monkeypatch, fake)
+    m._execute_processing(
+        m.ProcessRequest(video_path="gs://b/clip.mp4", segmenter="wasb_hybrid")
+    )
+    assert "--skip-validation" in fake.calls[0]
+
+def test_cloud_failure_marks_failed_no_rows(cloud_env):
+    db, _bucket, monkeypatch = cloud_env
+    # pre-seed a prior good row ? must be preserved on a cloud failure (destroy-AFTER-confirm).
diff --git a/tests/test_worker.py b/tests/test_worker.py
index 8fb5e16..72b2514 100644
--- a/tests/test_worker.py
+++ b/tests/test_worker.py
@@ -97,6 +97,14 @@ def test_parser_infer_accepts_set_and_uris():
+     assert args.set == ["indexing.skip_ai_handoff=true", "sport=tennis"]

+def test_parser_infer_skip_validation_flag():
+     base = ["infer", "--video-uri", "s3://b/v.mp4", "--out-uri", "s3://b"]
+     # Default-OFF: the direct `worker infer` CLI still validates.

```

```

+   assert wmain.build_parser().parse_args(base).skip_validation is False
+   # The dispatcher passes it to disable the redundant worker re-validation.
+   assert wmain.build_parser().parse_args(base + ["--skip-validation"]).skip_validation is
True
+
+
+   # ----- cmd_infer (local, mocked)

@@ -407,6 +415,43 @@ def test_cmd_infer_validation_failure_returns_2(tmp_path, monkeypatch):
    assert marker["returncode"] == 2 and marker["status"] == "failed"

+def test_cmd_infer_skip_validation_bypasses_validators(tmp_path, monkeypatch):
+   """--skip-validation (set by the dispatcher, the authoritative gate) must NOT run the
worker's
+   validators ? no second decode, and no chance of a threshold mismatch REJECTING a clip the
+   control-plane already passed (the live 2026-07-07 CUJ failure). Prove it by making
+   run_all_with_context RAISE: a skipped run never touches it, so the job completes rc 0 with
a
+   status="success" report. Contrast test_cmd_infer_validation_failure_returns_2 (no flag ? rc
2)."""
+   monkeypatch.setattr(wmain, "compute_video_id", lambda p: "vidSkip")
+
+   def _boom(*a, **k): # the validators must never be reached on the skip path
+       raise AssertionError("run_all_with_context must not run under --skip-validation")
+
+   monkeypatch.setattr(wmain.validator_registry, "run_all_with_context", _boom)
+   monkeypatch.setattr(wmain.segmenter_registry, "get", lambda name: _FakeSeg)
+   monkeypatch.setattr(wmain, "emit_indexer_report", lambda **k: None)
+   video = tmp_path / "in.mp4"
+   video.write_bytes(b"FAKE")
+   out = tmp_path / "bucket"
+
+   rc = wmain.main(
+       [
+           "infer",
+           "--video-uri",
+           str(video),
+           "--out-uri",
+           str(out),
+           "--scratch",
+           str(tmp_path / "s"),
+           "--segmenter",
+           "wasb_hybrid",
+           "--skip-validation",
+       ]
+   )
+   assert rc == 0 # a clip that WOULD fail re-validation still succeeds ? the gate is trusted
+   report = json.loads((out / "outputs" / "vidSkip" / "report.json").read_text())
+   assert report["status"] == "success" and report["segment_count"] == 2
+
+
+   # --- cmd_train: pull labels -> synth trajectories -> manifest -> harness -> push bundle ---

```

85. assistant (2026-07-07T14:40:11.164Z)

The diff is clean and focused (107 insertions across 6 files). Let me wait for the full suite to finish ? I'll check its progress once rather than poll.

86. user (2026-07-07T14:40:14.268Z)

<tool_use_error>Blocked: sleep 45 followed by: tail -5
/c/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/902e27a6-e026-4b55-8c26-f2c47329cc44/tasks/b7biy10f1.output echo "no output yet". To

wait for a condition, use Monitor with an until-loop (e.g. `until <check>; do sleep 2; done`). To wait for a command you started, use `run_in_background: true`. Do not chain shorter sleeps to work around this block.</tool_use_error>

87. assistant (2026-07-07T14:40:20.781Z)

Right ? I'll let the background run notify me. Let me just read the interim output file directly.

88. user (2026-07-07T14:40:21.671Z)

<system-reminder>Warning: the file exists but is shorter than the provided offset (1). The file has 1 lines.</system-reminder>

89. user (2026-07-07T14:40:28.753Z)

```
1      =====
2      Full test suite: C:\Users\avidu\AppData\Local\Programs\Python\Python313\python.exe -m
pytest -q --cov=backend --cov-report=term-missing --cov-fail-under=70
3      obsreport (optional reporting dep): available
4      =====
5      .....SS..... [ 4%]
6      .....
```

90. assistant (2026-07-07T14:40:40.387Z)

The suite is running with obsreport available (4% in). I'll wait for it to complete ? I'll be notified. Here's where things stand while it runs:

****Root cause (confirmed by reading the code):**** The control-plane validates at `orchestration.py:747` and ****only dispatches on PASS**** (failures return `"failed"` before `\_maybe\_dispatch\_remote`). But the worker's `cmd\_infer` (`worker/\_\_main\_\_.py:263`) unconditionally re-ran `validator\_registry.run\_all\_with\_context` with its *own* config ? so a clip that passed at `min\_blur\_threshold=8.0` got rejected at the worker's default `20.0`.

****Fix (skip-on-dispatch, the preferred option):****

- `ComputeJobSpec.skip\_validation` (new field, default `False`) ? `cloud\_run.py` emits `--skip-validation` ? `cmd\_infer` honors it (skips the re-decode + re-validation) ?
- `orchestration.\_maybe\_dispatch\_remote` sets it `True` on every dispatch.
- The direct `worker infer` CLI (no control-plane) still validates by default.
- Verified `val\_context` only feeds the observability report (not segmentation), and the `report.json` cache consumers read only `status`/`video\_uri` ? so skipping is functionally safe.

****Gate so far:**** ruff clean, mypy clean (148 files), targeted tests 85/85 pass (incl. 3 new). Full suite with coverage is running now.

91. user (2026-07-07T14:40:41.375Z)

```
5      .....SS..... [ 4%]
6      .....
```

92. assistant (2026-07-07T14:40:49.019Z)

I'll wait for the full-suite completion notification before proceeding to commit and open the PR.

93. user (2026-07-07T14:43:30.034Z)

```
<task-notification>
<task-id>b7biy10f1</task-id>
<tool-use-id>toolu_01J5b9AfNNETot84iSwAwXDL</tool-use-id>
<output-file>C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-
highlight-indexer\902e27a6-e026-4b55-8c26-f2c47329cc44\tasks\b7biy10f1.output</output-file>
<status>completed</status>
```

<summary>Background command "Run full test suite with coverage (background)" completed (exit code 0)</summary>
</task-notification>

94. assistant (2026-07-07T14:43:35.531Z)

The full suite passed (exit 0). Let me read the result and coverage summary.

95. user (2026-07-07T14:43:39.517Z)

backend\storage__init__.py	85	3	96%	113-114, 116
backend\storage\base.py	16	1	94%	75
backend\storage\capabilities.py	59	10	83%	51-56, 63-64, 104-105
backend\storage\gcs.py	80	3	96%	58, 63, 130
backend\storage\gdrive.py	61	3	95%	78-79, 86
backend\storage\gdrive_api.py	105	6	94%	78-79, 157, 189, 215, 239
backend\storage\local.py	49	3	94%	38, 66-67
backend\storage\ref.py	49	0	100%	
backend\storage\s3.py	47	5	89%	37-40, 49, 52
backend\tools__init__.py	0	0	100%	
backend\tools\export.py	75	2	97%	30-31
backend\tools\rally_slicer.py	105	47	55%	53-54, 74, 110, 116-134, 146-195, 199-223, 227
backend\utils__init__.py	0	0	100%	
backend\utils\annotation_proxy.py	64	11	83%	66-67, 81, 107-110, 119-122
backend\utils\app_home.py	40	0	100%	
backend\utils\ffmpeg.py	47	3	94%	49-51
backend\utils\freespace.py	32	2	94%	44, 46
backend\utils\geometry.py	29	0	100%	
backend\utils\hashing.py	9	0	100%	
backend\utils\logging_setup.py	11	0	100%	
backend\utils\paths.py	44	3	93%	38, 52-53
backend\utils\redact.py	16	0	100%	
backend\utils\run_telemetry.py	126	12	90%	55-56, 81, 103-104, 113-114, 121, 130-131, 250-251
backend\utils\single_instance.py	43	11	74%	49-51, 61-62, 69, 77-81, 85-86
backend\utils\video.py	34	3	91%	80-82
backend\utils\workspace.py	49	0	100%	
backend\worker__init__.py	0	0	100%	
backend\worker__main__.py	315	29	91%	193-197, 230-231, 309-310, 332-336, 469-474, 499-503, 506-510, 520-521, 542, 559-563, 669-676, 680-681, 828

TOTAL	12135	1769	85%	
Required test coverage of 70% reached. Total coverage: 85.42%				
1779 passed, 8 skipped, 25 warnings in 234.83s (0:03:54)				
=====				
RESULT: PASS - all tests green				
=====				

96. assistant (2026-07-07T14:44:27.331Z)

Full gate is **green**: **1779 passed, 8 skipped, 0 failures**, coverage **85.42%** (?70 floor, no regression vs the ~85% base), ruff + mypy clean.

Ultracode is on, and this touches the live cloud-serving path ? so before I sign off and open the PR, I'll run an adversarial multi-agent review of the diff to hunt for anything my tests don't cover (other dispatch paths, downstream consumers of the now-empty validation results, and rollout/version-skew risk).

97. user (2026-07-07T14:45:11.240Z)

Workflow launched in background. Task ID: w4sk786za

Summary: Adversarially review the skip-worker-revalidation fix on the cloud dispatch path

Transcript dir: C:\Users\avidu\claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\902e27a6-e026-4b55-8c26-f2c47329cc44\subagents\workflows\wf_c287c071-8e9

Script file: C:\Users\avidu\claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\902e27a6-e026-4b55-8c26-f2c47329cc44\workflows\scripts\review-skip-revalidation-wf_c287c071-8e9.js

(Edit this file with Write/Edit and re-invoke Workflow with {scriptPath:

"C:\Users\avidu\claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\902e27a6-e026-4b55-8c26-f2c47329cc44\workflows\scripts\review-skip-revalidation-wf_c287c071-8e9.js"} to iterate without resending the script.)

Run ID: wf_c287c071-8e9

To resume after editing the script: Workflow({scriptPath: "C:\Users\avidu\claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\902e27a6-e026-4b55-8c26-f2c47329cc44\workflows\scripts\review-skip-revalidation-wf_c287c071-8e9.js", resumeFromRunId: "wf_c287c071-8e9"}) ? completed agents return cached results (cached results may themselves be empty ? inspect journal.jsonl before assuming there is something to recover).

You will be notified when it completes. Use /workflows to watch live progress.

98. assistant (2026-07-07T14:45:23.778Z)

The adversarial review is running in the background. I'll wait for it before committing, in case any finding needs a code change. While it runs, let me draft the commit message and PR body so they're ready.

99. assistant (2026-07-07T14:45:36.361Z)

fix(serving): worker skips redundant re-validation on the cloud dispatch path

On the cloud-serving path the control-plane is the AUTHORITATIVE validation gate: backend/orchestration.py runs the validators with its resolved config and only dispatches to the Cloud Run worker on PASS (a failure returns "failed" before `_maybe_dispatch_remote`). The worker's `cmd_infer` then RE-RAN validation with its OWN config ? and since the control-plane's resolved validation.* keys are not forwarded on dispatch (#509 forwards only `skip_ai_handoff` + the `wasb.*` knobs), the worker used its default thresholds. Contradictory decisions result.

Live 2026-07-07 CUJ: a 5312x2988 10-bit HEVC clip (sharpness 11.9) PASSED the control-plane at `validation.min_blur_threshold=8.0`, then the worker REJECTED it at its default `20.0` ? "[worker] validation FAILED: `blur_check ... 11.9 < 20.0`", `rc=2`. A successful GPU dispatch failed after the fact.

Fix (skip-on-dispatch ? also saves the redundant second full-video decode):

- `ComputeJobSpec.skip_validation` (new bool, default False)
- `CloudRunCompute` emits a "--skip-validation" worker flag when set
- worker `cmd_infer` honors it: skips `validator_registry.run_all_with_context` (`val_results=[]`, `val_context={}`); the direct ``worker infer`` CLI still validates by default (no control-plane there)
- `_maybe_dispatch_remote` sets `skip_validation=True` on every dispatch (only a validated PASS ever reaches it)
- `_dispatch_remote` propagates the flag through the worker->worker recursion path

`val_context` feeds only the observability report (`ffprobe_meta`), not segmentation; the `report.json` cache consumers read only `status/video_uri` ? so skipping is functionally safe for detection and ingest.

Tests: `tests/test_worker.py` (parser accepts the flag; `cmd_infer` under `--skip-validation` never touches the validators and a would-fail clip still succeeds `rc=0`) and `tests/test_api_cloud_dispatch.py` (the dispatch argv carries `--skip-validation`).

Full suite 1779 passed / coverage 85.42% / ruff + mypy clean.

100. user (2026-07-07T14:45:36.397Z)

File created successfully at: C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\902e27a6-e026-4b55-8c26-f2c47329cc44\scratchpad\commit_msg.txt (file state is current in your context ? no need to Read it back)

101. assistant (2026-07-07T14:45:39.764Z)

I'll wait for the adversarial review to complete before committing.

102. user (2026-07-07T14:56:05.322Z)

```
<task-notification>
<task-id>w4sk786za</task-id>
<tool-use-id>toolu_01CeHhsVdGhxUXSP4akVE4ap</tool-use-id>
<output-file>C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\902e27a6-e026-4b55-8c26-f2c47329cc44\tasks\w4sk786za.output</output-file>
<status>completed</status>
<summary>Dynamic workflow "Adversarially review the skip-worker-revalidation fix on the cloud dispatch path" completed</summary>
<result>{"confirmed":[{"dimension":"rollout","severity":"major","category":"rollout-skew","file":"backend/compute/cloud_run.py","line":202,"summary":"The new `--skip-validation` worker CLI flag is emitted on every cloud dispatch, but an older worker Job image that lacks the argparse arg aborts (rc 2) BEFORE writing the done-marker, causing a 30-minute bucket-poll hang then an rc-4 failure on every dispatch during a control-plane-ahead-of-worker rollout window.","detail":"The control-plane (Cloud Run Service) and worker (Cloud Run Job) are the same image but SEPARATE resources with independently-pinned digests; README deploy/cloudrun/README.md §2a already warns they can skew. Unlike #507/#509 which forwarded decode_backend via the PRE-EXISTING `--set k=v` arg (orchestration.py:564 ? never argparse-aborts), this change adds a genuinely NEW argparse flag: worker __main__.py:743 (`add_argument(\"--skip-validation\")`), consumed by strict `parse_args` at __main__.py:822. orchestration.py:582 sets skip_validation=True on EVERY dispatch, and cloud_run.py:202 appends `--skip-validation` to the container argv. If the control-plane is updated to the new image but the worker Job still points at an older digest (the natural 'redploy the service, forget/lag the `jobs update`' sequence ? #510's runlog shows the control-plane being redeployed on its own), the old worker runs `python3 -m backend.worker infer ... --skip-validation` (ENTRYPOINT is bare `python3 -m backend.worker`, no wrapper) and argparse calls SystemExit(2) inside main() BEFORE cmd_infer/_fail/_write_done_marker execute. No done-marker is written. The dispatcher (cloud_run.py run_infer) only bucket-polls the marker (never the Cloud Run API status), so poll_until runs the full ExecutionPolicy.max_wait_sec=1800s (30 min), then PolicyTimeout -> orchestration.py:593-595 surfaces a generic 'cloud dispatch error' (rc 4). This affects EVERY dispatch, including clips the control-plane PASSED ? a strict regression versus the pre-fix fast rc-2, and the failure is opaque (the real 'unrecognized arguments: --skip-validation' only appears in the Job execution log, not to the control-plane). This is stricter coupling than #507's decode_backend yet ships NO deploy-together release note (README §2a got one for #507) and no code guard. Correct mitigation: (a) a README §2a-style 'roll worker image + control plane together, worker-first' note, since a worker-side parse_known_args guard added now cannot retroactively fix the already-deployed old image; reverse skew (new worker, old control-plane) is safe ? the old control-plane simply never sends the flag. Separately verified NON-issue: adding skip_validation as a trailing `bool = False` field to the frozen ComputeJobSpec is benign ? bool is hashable, all `ComputeJobSpec` call sites use kwargs, and the spec is in-memory only (not pickled/persisted across versions).","failure_scenario":"Operator builds the new image, redeploys the rally-control-plane Service to it, but the rally-worker Job still pins the prior digest (its `jobs update` is lagged or forgotten). A user uploads a clip; the control-plane validates it PASS at min_blur_threshold=8.0 and dispatches with --skip-validation. The old worker Job argparse-aborts rc 2 with no done-marker; the control-plane bucket-polls for 30 minutes, then fails the job with 'cloud dispatch error' (rc 4). Every dispatch in the skew window behaves this way, and the underlying cause (an unrecognized CLI arg) is invisible from the control-plane surface."},"verdict":{"reasoning":"CONFIRMED ? every mechanical claim verifies against the real code, and the actual behavior is slightly worse than the finding states.\n\n(1) `--skip-validation` is a genuinely NEW argparse flag: backend/worker/__main__.py:743 adds it, and main() consumes it via strict `build_parser().parse_args(argv)` at :822. I confirmed there
```

is NO `--decode-backend` argparse flag anywhere in the worker ? decode_backend (#507) is forwarded through the PRE-EXISTING `--set k=v` arg (orchestration.py:564: `overrides.append(f"indexing.wasb.decode\_backend={decode\_backend}")`), which an old worker's argparse accepts. So the finding's central distinction holds: this is the first NEW argparse flag on the dispatch path.

(2) It is emitted UNCONDITIONALLY on every cloud dispatch: orchestration.py:582 sets `skip\_validation=True` on the ComputeJobSpec for every dispatch; cloud_run.py:199-202 appends `--skip-validation` to the container argv whenever the spec flag is set. (Contrast #507's decode_backend, only forwarded when the preset sets it.)

(3) An old worker image argparse-aborts BEFORE any done-marker exists. main() (___main___py:822) runs `parse\_args` and only then calls `args.func(args)`; the done-marker is written exclusively inside cmd_infer (success path and the _fail helper at :312-315). An unrecognized flag makes argparse call SystemExit(2) at parse time, so cmd_infer/_fail/_write_done_marker never run and no marker is written. The Dockerfile ENTRYPOINT is bare `["python3","-m","backend.worker"]` (deploy/cloudrun/Dockerfile:123) and `\_\_\_main\_\_\_` does `sys.exit(main())` ? nothing catches SystemExit ? so the container exits rc 2 with no marker.

(4) The dispatcher only bucket-polls the marker, never the Cloud Run API status: cloud_run.py:217 `poll\_until(lambda: self.\_read\_marker(done\_uri), ...)`. With no marker, poll runs the full budget, raises PolicyTimeout, `\_handle\_poll\_timeout` does one final (absent) marker check, best-effort cancels, and re-raises; orchestration.py:593-595 catches it as a generic `cloud dispatch error` (rc-4 on the worker-CLI path via _dispatch_remote). This hits EVERY dispatch in the skew window, including clips the control-plane already PASSED ? a strict regression versus pre-fix (where the old worker simply re-validated and mostly succeeded, or fast-failed rc 2 WITH a failure marker).

(5) Two refinements, both worsening the finding rather than refuting it: (a) the poll budget is NOT 1800s in production ? get_compute_backend derives it from deployment.remote_poll_max_wait_sec, default 3900.0s (~65 min) per config/models.py:483 (the 1800s is only the DEFAULT_POLICY fallback, overridden from config). So the opaque hang is ~65 min, not 30. (b) The README §2a deploy-together caveat exists ONLY for #507's decode_backend (deploy/cloudrun/README.md:76-78); no equivalent note was added for --skip-validation, and there is no code guard.

(6) The finding's self-declared NON-issue is correct: all ComputeJobSpec construction sites (orchestration.py:577, worker/___main___py:184, tests) use kwargs, skip_validation is a trailing `bool=False` field on the frozen dataclass (hashable), and the spec is in-memory only ? benign.

The skew is a real operational state: control-plane is a Cloud Run Service (rally-control-plane) and worker is a separate Cloud Run Job (rally-worker), each pinning a digest independently (README §2a exists precisely because they can skew), and the natural `services replace` / lagged `jobs update` sequence produces control-plane-ahead skew.

Severity nuance: the finding labels this `major`; I'd call it moderate. It manifests only during a transient control-plane-ahead rollout window, is fully avoidable by worker-first deploy ordering, reverse skew (new worker/old control-plane) is safe (the old control-plane never emits the flag), and no code change can retroactively guard the already-built old image ? so the proportionate mitigation is a deploy note, exactly as #507 got. But it is a genuine, opaque ~65-min-then-fail operational regression that no unit test can catch, so it is worth reporting.

"recommended_fix": "Minimal fix (matches how #507 was handled): add a README §2a-style deploy-together note stating that because the control-plane now emits `--skip-validation` on EVERY dispatch, the worker Job image must be rolled to the new image BEFORE (or together with) the control-plane Service. An old worker argparse-aborts (rc 2) on t ... (truncated 6809 chars, full result in C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\902e27a6-e026-4b55-8c26-f2c47329cc44\tasks\w4sk786za.output)</result>

<diagnostics>Per-agent results: C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\902e27a6-e026-4b55-8c26-f2c47329cc44\subagents\workflows\wf_c287c071-8e9\journal.jsonl ? one {"type":"result",...} line per completed agent with its full return value.

If the result above is empty or unexpected, Read this file BEFORE diagnosing ? do not assume agents returned non-empty results.

To re-run with edited post-processing: Workflow({scriptPath: 'C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\902e27a6-e026-4b55-8c26-f2c47329cc44\workflows\scripts\review-skip-revalidation-wf_c287c071-8e9.js', resumeFromRunId: 'wf_c287c071-8e9'}) ? agents whose (prompt, opts) are unchanged replay from cache.</diagnostics>

<usage><agent_count>8</agent_count><agents_done>8</agents_done><agents_error>0</agents_error><agents_skipped>0</agents_skipped><agents_empty_result>1</agents_empty_result><subagent_tokens>610906</subagent_tokens><tool_uses>119</tool_uses><duration_ms>654028</duration_ms></usage>

</task-notification>

The review surfaced **one confirmed finding** (rollout-skew, rated moderate): because `--skip-validation`` is a genuinely **new** argparse flag (unlike #509's `--set``, which old workers accept), a control-plane-ahead-of-worker deploy skew would make an old worker image argparse-abort **before** writing the done-marker ? the dispatcher bucket-polls the full budget (~65 min) then fails rc-4 on **every** dispatch. Recommended mitigation: a deploy-together note (matching #507's precedent), since no code can fix an already-built old image.

Let me read the full output, the `_apply_overrides`` unknown-key behavior (to judge whether a graceful `--set``-based approach beats the flag), and the existing #507 deploy note.

104. user (2026-07-07T14:57:00.199Z)

```
1      {
2      "summary": "Adversarially review the skip-worker-revalidation fix on the cloud
dispatch path",
3      "agentCount": 8,
4      "logs": [
5      "reviewed 4 raw findings ? 2 CONFIRMED, 0 UNCERTAIN"
6      ],
7      "result": {
8      "confirmed": [
9      {
10     "dimension": "rollout",
11     "severity": "major",
12     "category": "rollout-skew",
13     "file": "backend/compute/cloud_run.py",
14     "line": 202,
15     "summary": "The new `--skip-validation` worker CLI flag is emitted on every
cloud dispatch, but an older worker Job image that lacks the argparse arg aborts (rc 2) BEFORE
writing the done-marker, causing a 30-minute bucket-poll hang then an rc-4 failure on every
dispatch during a control-plane-ahead-of-worker rollout window.",
16     "detail": "The control-plane (Cloud Run Service) and worker (Cloud Run Job) are
the same image but SEPARATE resources with independently-pinned digests; README
deploy/cloudrun/README.md §2a already warns they can skew. Unlike #507/#509 which forwarded
decode_backend via the PRE-EXISTING `--set k=v` arg (orchestration.py:564 ? never argparse-
aborts), this change adds a genuinely NEW argparse flag: worker __main__.py:743
(`add_argument(`--skip-validation`)`), consumed by strict `parse_args` at __main__.py:822.
orchestration.py:582 sets skip_validation=True on EVERY dispatch, and cloud_run.py:202 appends
`--skip-validation` to the container argv. If the control-plane is updated to the new image but
the worker Job still points at an older digest (the natural 'redploy the service, forget/lag
the `jobs update`' sequence ? #510's runlog shows the control-plane being redeployed on its
own), the old worker runs `python3 -m backend.worker infer ... --skip-validation` (ENTRYPOINT is
bare `python3 -m backend.worker`, no wrapper) and argparse calls SystemExit(2) inside main()
BEFORE cmd_infer/_fail/_write_done_marker execute. No done-marker is written. The dispatcher
(cloud_run.py run_infer) only bucket-polls the marker (never the Cloud Run API status), so
poll_until runs the full ExecutionPolicy.max_wait_sec=1800s (30 min), then PolicyTimeout ->
orchestration.py:593-595 surfaces a generic 'cloud dispatch error' (rc 4). This affects EVERY
dispatch, including clips the control-plane PASSED ? a strict regression versus the pre-fix fast
rc-2, and the failure is opaque (the real 'unrecognized arguments: --skip-validation' only
appears in the Job execution log, not to the control-plane). This is stricter coupling than
#507's decode_backend yet ships NO deploy-together release note (README §2a got one for #507)
and no code guard. Correct mitigation: (a) a README §2a-style 'roll worker image + control plane
together, worker-first' note, since a worker-side parse_known_args guard added now cannot
retroactively fix the already-deployed old image; reverse skew (new worker, old control-plane)
is safe ? the old control-plane simply never sends the flag. Separately verified NON-issue:
adding skip_validation as a trailing `bool = False` field to the frozen ComputeJobSpec is benign
? bool is hashable, all `ComputeJobSpec` call sites use kwargs, and the spec is in-memory only
(not pickled/persisted across versions).",
17     "failure_scenario": "Operator builds the new image, redeploys the rally-control-
plane Service to it, but the rally-worker Job still pins the prior digest (its `jobs update` is
lagged or forgotten). A user uploads a clip; the control-plane validates it PASS at
min_blur_threshold=8.0 and dispatches with --skip-validation. The old worker Job argparse-aborts
rc 2 with no done-marker; the control-plane bucket-polls for 30 minutes, then fails the job with
'cloud dispatch error' (rc 4). Every dispatch in the skew window behaves this way, and the
underlying cause (an unrecognized CLI arg) is invisible from the control-plane surface.",
```

```

18         "verdict": {
19             "reasoning": "CONFIRMED ? every mechanical claim verifies against the real
code, and the actual behavior is slightly worse than the finding states.\n\n(1) `--skip-
validation` is a genuinely NEW argparse flag: backend/worker/__main__.py:743 adds it, and main()
consumes it via strict `build_parser().parse_args(argv)` at :822. I confirmed there is NO
`--decode-backend` argparse flag anywhere in the worker ? decode_backend (#507) is forwarded
through the PRE-EXISTING `--set k=v` arg (orchestration.py:564:
`overrides.append(f"indexing.wasb.decode_backend={decode_backend}")`), which an old worker's
argparse accepts. So the finding's central distinction holds: this is the first NEW argparse
flag on the dispatch path.\n\n(2) It is emitted UNCONDITIONALLY on every cloud dispatch:
orchestration.py:582 sets `skip_validation=True` on the ComputeJobSpec for every dispatch;
cloud_run.py:199-202 appends `--skip-validation` to the container argv whenever the spec flag is
set. (Contrast #507's decode_backend, only forwarded when the preset sets it.)\n\n(3) An old
worker image argparse-aborts BEFORE any done-marker exists. main() (__main__.py:822) runs
`parse_args` and only then calls `args.func(args)`; the done-marker is written exclusively
inside cmd_infer (success path and the _fail helper at :312-315). An unrecognized flag makes
argparse call SystemExit(2) at parse time, so cmd_infer/_fail/_write_done_marker never run and
no marker is written. The Dockerfile ENTRYPOINT is bare
`["python3", "-m", "backend.worker"]` (deploy/cloudrun/Dockerfile:123) and `__main__` does
`sys.exit(main())` ? nothing catches SystemExit ? so the container exits rc 2 with no
marker.\n\n(4) The dispatcher only bucket-polls the marker, never the Cloud Run API status:
cloud_run.py:217 `poll_until(lambda: self._read_marker(done_uri), ...)` . With no marker, poll
runs the full budget, raises PolicyTimeout, `_handle_poll_timeout` does one final (absent)
marker check, best-effort cancels, and re-raises; orchestration.py:593-595 catches it as a
generic `cloud dispatch error` (rc-4 on the worker-CLI path via _dispatch_remote). This hits
EVERY dispatch in the skew window, including clips the control-plane already PASSED ? a strict
regression versus pre-fix (where the old worker simply re-validated and mostly succeeded, or
fast-failed rc 2 WITH a failure marker).\n\n(5) Two refinements, both worsening the finding
rather than refuting it: (a) the poll budget is NOT 1800s in production ? get_compute_backend
derives it from deployment.remote_poll_max_wait_sec, default 3900.0s (~65 min) per
config/models.py:483 (the 1800s is only the DEFAULT_POLICY fallback, overridden from config). So
the opaque hang is ~65 min, not 30. (b) The README §2a deploy-together caveat exists ONLY for
#507's decode_backend (deploy/cloudrun/README.md:76-78); no equivalent note was added for
--skip-validation, and there is no code guard.\n\n(6) The finding's self-declared NON-issue is
correct: all ComputeJobSpec construction sites (orchestration.py:577, worker/__main__.py:184,
tests) use kwargs, skip_validation is a trailing `bool=False` field on the frozen dataclass
(hashable), and the spec is in-memory only ? benign.\n\nThe skew is a real operational state:
control-plane is a Cloud Run Service (rally-control-plane) and worker is a separate Cloud Run
Job (rally-worker), each pinning a digest independently (README §2a exists precisely because
they can skew), and the natural `services replace` / lagged `jobs update` sequence produces
control-plane-ahead skew.\n\nSeverity nuance: the finding labels this `major`; I'd call it
moderate. It manifests only during a transient control-plane-ahead rollout window, is fully
avoidable by worker-first deploy ordering, reverse skew (new worker/old control-plane) is safe
(the old control-plane never emits the flag), and no code change can retroactively guard the
already-built old image ? so the proportionate mitigation is a deploy note, exactly as #507 got.
But it is a genuine, opaque ~65-min-then-fail operational regression that no unit test can
catch, so it is worth reporting.",
20             "recommended_fix": "Minimal fix (matches how #507 was handled): add a README
§2a-style deploy-together note stating that because the control-plane now emits `--skip-
validation` on EVERY dispatch, the worker Job image must be rolled to the new image BEFORE (or
together with) the control-plane Service. An old worker argparse-aborts (rc 2) on the unknown
flag and writes NO done-marker, so the dispatcher bucket-polls to the full
`deployment.remote_poll_max_wait_sec` (~65 min default) and then fails opaquely as `cloud
dispatch error`. Worker-first ordering eliminates the window; reverse skew (new worker, old
control-plane) is inherently safe because the old control-plane never sends the flag. A worker-
side `parse_known_args` guard added now CANNOT fix the already-deployed old image, so the doc
note is the correct mitigation for this release.\n\nStronger, forward-looking (optional, larger
change): make the dispatcher consult the Cloud Run execution status (Failed/Cancelled) in
addition to bucket-polling the marker, so ANY worker crash-before-marker ? not just this flag ?
fails fast instead of hanging the entire poll budget. This would collapse the ~65-min opaque
hang to a prompt, diagnosable failure for the whole class of `control-plane sends something the
old worker can't handle` skews.",
21         "verdict": "CONFIRMED"
22     }
23 },

```

```

24     {
25         "dimension": "tests",
26         "severity": "major",
27         "category": "test-coverage",
28         "file": "tests/test_compute_backend.py",
29         "line": 201,
30         "summary": "No test asserts that a default spec (skip_validation=False) OMTS
--skip-validation from the worker argv, leaving the 'direct CLI still validates on the cloud
path' safety invariant unguarded at the argv builder.",
31         "detail": "cloud_run.py::run_infer (backend/compute/cloud_run.py:199-202)
appends '--skip-validation' gated on `if spec.skip_validation:`. Every test that exercises argv
building either uses a default spec without asserting the flag's ABSENCE
(test_dispatch_argv_forces_inline_and_carries_overrides, line 201, builds a spec with
skip_validation at its default False but never checks `--skip-validation' not in argv`) or goes
through the control-plane which always sets True (all test_cloud_dispatch_* in
test_api_cloud_dispatch.py). So the invariant the change explicitly promises to preserve --
'Default False keeps the direct `worker infer` CLI validating exactly as before' -- has no
regression guard at the builder that constructs the flag. Add an assertion to
test_dispatch_argv_forces_inline_and_carries_overrides (or a dedicated test) that a default
ComputeJobSpec yields argv WITHOUT '--skip-validation', and a paired one that
skip_validation=True yields it.",
32         "failure_scenario": "A future refactor of run_infer's argv construction drops or
inverts the `if spec.skip_validation:` guard (e.g. appends the flag unconditionally, or a bad
merge). The full suite still passes green: the only argv-builder test uses a default spec and
never asserts the flag is absent, and every control-plane dispatch test legitimately expects
True. The remote worker on the direct-CLI cloud dispatch path (_dispatch_remote,
skip_validation=False) then silently skips validation -- reintroducing exactly the class of
'validation was bypassed when it shouldn't be' defect this change is guarding, undetected.",
33         "verdict": {
34             "reasoning": "The finding is factually accurate against the real code.
cloud_run.py:199 appends \"--skip-validation\" only under `if spec.skip_validation:`. The sole
argv-builder test that uses a default spec,
test_dispatch_argv_forces_inline_and_carries_overrides (tests/test_compute_backend.py:201),
builds ComputeJobSpec(...) with skip_validation at its default False and asserts only via
`in`/index membership checks ? it never asserts \"--skip-validation\" not in argv`. A full-
suite grep confirms every assertion of the flag's PRESENCE is either through the control-plane
(orchestration._maybe_dispatch_remote hardcodes skip_validation=True at orchestration.py:582, so
test_cloud_dispatch_skips_worker_revalidation legitimately expects True), or at the parser level
(test_parser_infer_skip_validation_flag), or at the worker-behavior level
(test_cmd_infer_skip_validation_bypasses_validators). None guard the \"default spec omits the
flag\" invariant at the builder ? which is exactly the safety invariant the change advertises
(\"Default False keeps the direct `worker infer` CLI validating exactly as before\").\n\nI
verified the concrete failure scenario is demonstrable: no test in the suite does an exact-argv
equality/length/count assertion (grep for `assert argv ==`, `len(argv)`, `argv.count` returns
nothing), and no test has a real worker parse the built argv (the worker is faked via
_FakeRunClient/_FakeCloudClient in both test_compute_backend.py and test_api_cloud_dispatch.py).
Therefore a regression that drops the guard and appends \"--skip-validation\" unconditionally
would leave every existing `in`/index-based assertion satisfied and the full suite green ? while
the direct-CLI cloud path (worker/__main__.py:186 _dispatch_remote, skip_validation defaults
False) would silently emit the flag and bypass validation, reintroducing precisely the class of
defect this change guards against.\n\nOne caveat that narrows the finding versus its wording:
the \"inverts the guard\" mutation is NOT uncaught ?
test_cloud_dispatch_skips_worker_revalidation would fail if the guard were inverted, because the
control-plane sets True and that test asserts the flag is present. So only the \"drop guard /
append unconditionally\" mutation escapes. The exposure is therefore narrower than \"drops or
inverts,\" but the core gap (no negative guard at the builder for a default spec) is real. This
is a genuine test-coverage gap, not a bug in shipped code ? the shipped code is correct.",
35         "recommended_fix": "Add a one-line negative assertion to the existing builder
test, which already constructs a default spec, and (optionally) a paired positive builder-level
test.\n\nIn tests/test_compute_backend.py, inside
test_dispatch_argv_forces_inline_and_carries_overrides (after `argv =
client.calls[0][\"argv\"]`, ~line 217):\n\n    # default spec (skip_validation=False) must NOT
carry the flag ? the direct `worker infer`\n    # CLI still validates; guards against dropping
the `if spec.skip_validation:` gate.\n    assert \"--skip-validation\" not in argv\n\nAnd a
dedicated positive test at the builder (independent of the control-plane path):\n\n    def

```



```

test_dispatch_argv_carries_skip_validation_when_set(tmp_path):\n        client =
_FakeRunClient()\n        backend = CloudRunCompute(\n        run_client=client,
job_name="\j\", region="\r\", policy=_FAST, token="\tok\"\n        )\n
backend.run_infer(\n        ComputeJobSpec(\n        video_uri="\gs://b/v.mp4\", \n
out_uri=str(tmp_path / \"bk\"), \n        skip_validation=True, \n        )\n
)\n        assert \"--skip-validation\" in client.calls[0][\"argv\"]\n\nThe negative assertion
is the load-bearing addition: it makes the \"drop the guard / append unconditionally\"
regression fail the suite.\",
36         \"verdict\": \"CONFIRMED\"
37     }
38 }
39 ],
40 \"uncertain\": [],
41 \"raw_count\": 4
42 },
43 \"workflowProgress\": [
44     {
45         \"type\": \"workflow_phase\",
46         \"index\": 1,
47         \"title\": \"Review\"
48     },
49     {
50         \"type\": \"workflow_phase\",
51         \"index\": 2,
52         \"title\": \"Verify\"
53     },
54     {
55         \"type\": \"workflow_agent\",
56         \"index\": 1,
57         \"label\": \"review:completeness\",
58         \"phaseIndex\": 1,
59         \"phaseTitle\": \"Review\",
60         \"agentId\": \"aa08899520aa6f0ed\",
61         \"model\": \"claude-opus-4-8\",
62         \"state\": \"done\",
63         \"startedAt\": 1783435511299,
64         \"queuedAt\": 1783435511254,
65         \"attempt\": 1,
66         \"lastToolName\": \"StructuredOutput\",
67         \"promptPreview\": \"You are reviewing an uncommitted change in a git worktree at:\n
C:/Users/avidu/Projects/khelsutra-guru/bhi-wt-worker-reval\nBranch: fix/worker-skip-
revalidation-on-dispatch (based on origin/master).\n\nTHE BUG BEING FIXED: On the cloud-serving
path, the control-plane (backend/orchestration.py) runs\nvalidators and ONLY dispatches to the
Cloud Run worker on PASS. The worker (backend/worker/__main__.py\n?\",
68         \"lastProgressAt\": 1783435765836,
69         \"tokens\": 77386,
70         \"toolCalls\": 22,
71         \"durationMs\": 254537,
72         \"resultPreview\": \"{\\\"findings\\\":[]}\"
73     },
74     {
75         \"type\": \"workflow_agent\",
76         \"index\": 2,
77         \"label\": \"review:consumers\",
78         \"phaseIndex\": 1,
79         \"phaseTitle\": \"Review\",
80         \"agentId\": \"ab26009d1c2eea60c\",
81         \"model\": \"claude-opus-4-8\",
82         \"state\": \"done\",
83         \"startedAt\": 1783435511302,
84         \"queuedAt\": 1783435511255,
85         \"attempt\": 1,
86         \"lastToolName\": \"StructuredOutput\",
87         \"promptPreview\": \"You are reviewing an uncommitted change in a git worktree at:\n
C:/Users/avidu/Projects/khelsutra-guru/bhi-wt-worker-reval\nBranch: fix/worker-skip-

```

```

revalidation-on-dispatch (based on origin/master).\n\nTHE BUG BEING FIXED: On the cloud-serving
path, the control-plane (backend/orchestration.py) runs\nvalidators and ONLY dispatches to the
Cloud Run worker on PASS. The worker (backend/worker/__main__.py\n?",
88         "lastProgressAt": 1783435832858,
89         "tokens": 106584,
90         "toolCalls": 19,
91         "durationMs": 321556,
92         "resultPreview": "{\n  \"findings\": [{\n    \"severity\": \"major\", \"category\": \"rollout-
skew\", \"file\": \"backend/orchestration.py\", \"line\": 582, \"summary\": \"Control-plane forwards
the brand-new hard argparse flag --skip-validation on EVERY cloud dispatch unconditionally; a
control-plane-ahead-of-worker deploy makes every dispatch argparse-fail (rc 2) on the old worker
image.\", \"detail\": \"_maybe_dispatch_remote sets skip_validation=True with?\"
93       },
94       {
95         \"type\": \"workflow_agent\",
96         \"index\": 3,
97         \"label\": \"review:rollout\",
98         \"phaseIndex\": 1,
99         \"phaseTitle\": \"Review\",
100        \"agentId\": \"ab1e72897df2aae47\",
101        \"model\": \"claude-opus-4-8\",
102        \"state\": \"done\",
103        \"startedAt\": 1783435511303,
104        \"queuedAt\": 1783435511255,
105        \"attempt\": 1,
106        \"lastToolName\": \"StructuredOutput\",
107        \"promptPreview\": \"You are reviewing an uncommitted change in a git worktree at:\n
C:/Users/avidu/Projects/khelsutra-guru/bhi-wt-worker-reval\nBranch: fix/worker-skip-
revalidation-on-dispatch (based on origin/master).\n\nTHE BUG BEING FIXED: On the cloud-serving
path, the control-plane (backend/orchestration.py) runs\nvalidators and ONLY dispatches to the
Cloud Run worker on PASS. The worker (backend/worker/__main__.py\n?",
108        \"lastProgressAt\": 1783435827831,
109        \"tokens\": 91923,
110        \"toolCalls\": 22,
111        \"durationMs\": 316528,
112        \"resultPreview\": "{\n  \"findings\": [{\n    \"severity\": \"major\", \"category\": \"rollout-
skew\", \"file\": \"backend/compute/cloud_run.py\", \"line\": 202, \"summary\": \"The new `--skip-
validation` worker CLI flag is emitted on every cloud dispatch, but an older worker Job image
that lacks the argparse arg aborts (rc 2) BEFORE writing the done-marker, causing a 30-minute
bucket-poll hang then an rc-4 failure on every dispatch during a control-plan?\"
113      },
114      {
115        \"type\": \"workflow_agent\",
116        \"index\": 4,
117        \"label\": \"review:tests\",
118        \"phaseIndex\": 1,
119        \"phaseTitle\": \"Review\",
120        \"agentId\": \"a1503123b7babe194\",
121        \"model\": \"claude-opus-4-8\",
122        \"state\": \"done\",
123        \"startedAt\": 1783435511303,
124        \"queuedAt\": 1783435511255,
125        \"attempt\": 1,
126        \"lastToolName\": \"StructuredOutput\",
127        \"promptPreview\": \"You are reviewing an uncommitted change in a git worktree at:\n
C:/Users/avidu/Projects/khelsutra-guru/bhi-wt-worker-reval\nBranch: fix/worker-skip-
revalidation-on-dispatch (based on origin/master).\n\nTHE BUG BEING FIXED: On the cloud-serving
path, the control-plane (backend/orchestration.py) runs\nvalidators and ONLY dispatches to the
Cloud Run worker on PASS. The worker (backend/worker/__main__.py\n?",
128        \"lastProgressAt\": 1783435729970,
129        \"tokens\": 88227,
130        \"toolCalls\": 15,
131        \"durationMs\": 218667,
132        \"resultPreview\": "{\n  \"findings\": [{\n    \"severity\": \"major\", \"category\": \"test-
coverage\", \"file\": \"tests/test_compute_backend.py\", \"line\": 201, \"summary\": \"No test asserts

```

```

that a default spec (skip_validation=False) OMITs --skip-validation from the worker argv,
leaving the 'direct CLI still validates on the cloud path' safety invariant unguarded at the
argv builder.\",\"detail\": \"cloud_run.py::run_infer (backend/compute/cloud_run.py:1?)\"
133     },
134     {
135         \"type\": \"workflow_agent\",
136         \"index\": 5,
137         \"label\": \"verify:tests:test-coverage\",
138         \"phaseIndex\": 2,
139         \"phaseTitle\": \"Verify\",
140         \"agentId\": \"a3561f765bd1892ef\",
141         \"model\": \"claude-opus-4-8\",
142         \"state\": \"done\",
143         \"startedAt\": 1783435729998,
144         \"queuedAt\": 1783435729973,
145         \"attempt\": 1,
146         \"lastToolName\": \"StructuredOutput\",
147         \"lastToolSummary\": \"The finding is factually accurate against the real code. cl?\",
148         \"promptPreview\": \"You are reviewing an uncommitted change in a git worktree at:\\n
C:/Users/avidu/Projects/khelsutra-guru/bhi-wt-worker-reval\\nBranch: fix/worker-skip-
revalidation-on-dispatch (based on origin/master).\\n\\nTHE BUG BEING FIXED: On the cloud-serving
path, the control-plane (backend/orchestration.py) runs\\nvalidators and ONLY dispatches to the
Cloud Run worker on PASS. The worker (backend/worker/__main__.py\\n?\",
149         \"lastProgressAt\": 1783435885665,
150         \"tokens\": 51769,
151         \"toolCalls\": 9,
152         \"durationMs\": 155667,
153         \"resultPreview\": \"{\\\"reasoning\\\":\\\"The finding is factually accurate against the
real code. cloud_run.py:199 appends \\\"\\\"--skip-validation\\\"\\\" only under `if
spec.skip_validation`. The sole argv-builder test that uses a default spec,
test_dispatch_argv_forces_inline_and_carries_overrides (tests/test_compute_backend.py:201),
builds ComputeJobSpec(...) with skip_validation at its default False and asserts only via
`in`/ind?\"
154     },
155     {
156         \"type\": \"workflow_agent\",
157         \"index\": 6,
158         \"label\": \"verify:tests:test-coverage\",
159         \"phaseIndex\": 2,
160         \"phaseTitle\": \"Verify\",
161         \"agentId\": \"a57cdf56a84824b9c\",
162         \"model\": \"claude-opus-4-8\",
163         \"state\": \"done\",
164         \"startedAt\": 1783435729999,
165         \"queuedAt\": 1783435729973,
166         \"attempt\": 1,
167         \"lastToolName\": \"StructuredOutput\",
168         \"lastToolSummary\": \"The finding's factual observation is accurate but it does n?\",
169         \"promptPreview\": \"You are reviewing an uncommitted change in a git worktree at:\\n
C:/Users/avidu/Projects/khelsutra-guru/bhi-wt-worker-reval\\nBranch: fix/worker-skip-
revalidation-on-dispatch (based on origin/master).\\n\\nTHE BUG BEING FIXED: On the cloud-serving
path, the control-plane (backend/orchestration.py) runs\\nvalidators and ONLY dispatches to the
Cloud Run worker on PASS. The worker (backend/worker/__main__.py\\n?\",
170         \"lastProgressAt\": 1783435846460,
171         \"tokens\": 47927,
172         \"toolCalls\": 6,
173         \"durationMs\": 116461,
174         \"resultPreview\": \"{\\\"reasoning\\\":\\\"The finding's factual observation is accurate
but it does not identify a defect, so it is below this review's reportable bar (default-REFUTED
unless a concrete failure exists in the current code).\\n\\n\\nVERIFIED FACTS:\\n1. Routing:
backend/worker/__main__.py:246 `return _dispatch_remote(compute, args)` fires when
compute_target is cloud. So test_cmd_infer_dispatches_when_cloud_target (t?\"
175     },
176     {
177         \"type\": \"workflow_agent\",

```

```

178         "index": 7,
179         "label": "verify:rollout:rollout-skew",
180         "phaseIndex": 2,
181         "phaseTitle": "Verify",
182         "agentId": "ac549cfab257e5c64",
183         "model": "claude-opus-4-8",
184         "state": "done",
185         "startedAt": 1783435827842,
186         "queuedAt": 1783435827839,
187         "attempt": 1,
188         "lastToolName": "StructuredOutput",
189         "lastToolSummary": "CONFIRMED ? every mechanical claim verifies against the rea?",
190         "promptPreview": "You are reviewing an uncommitted change in a git worktree at:\n
C:/Users/avidu/Projects/khelsutra-guru/bhi-wt-worker-reval\nBranch: fix/worker-skip-
revalidation-on-dispatch (based on origin/master).\n\nTHE BUG BEING FIXED: On the cloud-serving
path, the control-plane (backend/orchestration.py) runs\nvalidators and ONLY dispatches to the
Cloud Run worker on PASS. The worker (backend/worker/__main__.py\n?",
191         "lastProgressAt": 1783436165227,
192         "tokens": 83879,
193         "toolCalls": 18,
194         "durationMs": 337385,
195         "resultPreview": "{\n  \"reasoning\": \"CONFIRMED ? every mechanical claim verifies
against the real code, and the actual behavior is slightly worse than the finding
states.\n\n(1) `--skip-validation` is a genuinely NEW argparse flag:
backend/worker/__main__.py:743 adds it, and main() consumes it via strict
`build_parser().parse_args(argv)` at :822. I confirmed there is NO `--decode-backend` argparse
flag anywhere in the w?\"
196     },
197     {
198         "type": "workflow_agent",
199         "index": 8,
200         "label": "verify:consumers:rollout-skew",
201         "phaseIndex": 2,
202         "phaseTitle": "Verify",
203         "agentId": "a76da7693ae61b81c",
204         "model": "claude-opus-4-8",
205         "state": "done",
206         "startedAt": 1783435832861,
207         "queuedAt": 1783435832861,
208         "attempt": 1,
209         "lastToolName": "StructuredOutput",
210         "lastToolSummary": "The argparse mechanism is genuine: worker/__main__.py::main?",
211         "promptPreview": "You are reviewing an uncommitted change in a git worktree at:\n
C:/Users/avidu/Projects/khelsutra-guru/bhi-wt-worker-reval\nBranch: fix/worker-skip-
revalidation-on-dispatch (based on origin/master).\n\nTHE BUG BEING FIXED: On the cloud-serving
path, the control-plane (backend/orchestration.py) runs\nvalidators and ONLY dispatches to the
Cloud Run worker on PASS. The worker (backend/worker/__main__.py\n?",
212         "lastProgressAt": 1783436051425,
213         "tokens": 63211,
214         "toolCalls": 8,
215         "durationMs": 218563,
216         "resultPreview": "{\n  \"reasoning\": \"The argparse mechanism is genuine:
worker/__main__.py::main() does `build_parser().parse_args(argv)` with no guard, and `--skip-
validation` is a new hard `store_true` flag (build_parser) that cloud_run.py appends to argv and
orchestration.py:582 sets unconditionally (skip_validation=True). So an old worker image
receiving `--skip-validation` would raise SystemExit(2). That part is ac?\"
217     }
218 ],
219     "totalTokens": 610906,
220     "totalToolCalls": 119
221 }

```

105. assistant (2026-07-07T14:57:33.765Z)

Two confirmed findings, both legitimate:

1. ****Rollout-skew**** (moderate): the **new** `--skip-validation`` argparse flag makes an old worker image abort without writing a done-marker ? ~65-min opaque poll-hang during a control-plane-ahead deploy window. Mitigation: a deploy-together note (the #507/#509 precedent).
2. ****Test gap**** (shipped code is correct): no test asserts a **default** spec omits the flag, so a "drop the guard" regression would stay green while the direct-CLI cloud path silently skips validation.

Before deciding the rollout mitigation, one design question the reviewer raised: #509's `--set`` forwarding never argparse-aborts. Whether an old worker would degrade *\*gracefully\** under `--set`` hinges on how `_apply_overrides`` treats unknown keys. Let me check that, plus the test conventions and the #507 deploy note.

106. user (2026-07-07T14:57:34.764Z)

```
60
61     def _apply_overrides(config: Any, sets: List[str]) -> None:
62         """Apply ``a.b.c=value`` overrides onto the typed config (e.g.
indexing.skip_ai_handoff=true)."""
63         for kv in sets or []:
64             key, sep, val = kv.partition("=")
65             if not sep:
66                 raise ValueError(f"--set expects k=v, got {kv!r}")
67             parts = key.split(".")
68             obj = config
69             for p in parts[:-1]:
70                 obj = getattr(obj, p)
71             setattr(obj, parts[-1], _coerce(val))
72
73
74     def _write_intervals_csv(segments: List[dict], path: str) -> None:
75         """Decimal-second ``[start,end)`` rallies + shot count / ending reason ? the join-
friendly
76         artifact (same schema as the rally-annotator labels)."""
77         with open(path, "w", newline="") as f:
78             w = csv.writer(f)
79             w.writerow(["start", "end", "shot_count", "ending_reason"])
80             for s in segments:
81                 w.writerow(
82                     [
83                         f"{float(s.get('start_time', 0.0)):.3f}",
84                         f"{float(s.get('end_time', 0.0)):.3f}",
85                         s.get("shot_count", ""),
86                         s.get("ending_reason", s.get("shot_count_confidence", "")),
87                     ]
88                 )
89
90
91     def _write_report_json(
92         video_id: str, segments: List[dict], status: str, path: str, video_uri: str = ""
93     ) -> None:
94         with open(path, "w") as f:
95             json.dump(
96                 {
97                     "video_id": video_id,
98                     "status": status,
99                     "segment_count": len(segments),
100                     # The SOURCE the detection ran on (CP-rebuild P3, #485): a restart-wiped
101                     # control plane re-hydrating from outputs/<md5>/ restores the video row
102                     # with its truthful path ? without this, only the segments recover.
103                     "video_uri": video_uri,
104                     "segments": [
105                         {
106                             "start": float(s.get("start_time", 0.0)),
107                             "end": float(s.get("end_time", 0.0)),
108
```

```

109         for s in segments
110     ],
111     },
112     f,
113     indent=2,
114 )
115
116
117 def _serialize_and_push_outputs(
118     scratch: str,
119     out_uri: str,
120     video_id: str,
121     segments: List[dict],
122     status: str,
123     storage_cfg: dict,
124     video_uri: str = "",
125 ) -> List[str]:
126     """Write intervals.csv + report.json under ``<scratch>/outputs/<video_id>/`` and
push both to
127     ``<out_uri>/outputs/<video_id>/``; return the pushed URIs.
128
129     Shared by the SUCCESS path and the failure path (``_fail``) so a detector
timeout/abort leaves a
130     ``status="failed"`` report.json in the bucket ? the control plane's source of truth
131     (``orchestration.cached_process_result`` / ``probe_process_cache`` treat any
non-``success``
132     report as "no usable result" ? retry, and the restart-recovery poll waits for
report.json to
133     EXIST) ? instead of a missing artifact or a false 0-segment ``"success"``.
134     out_local = os.path.join(scratch, "outputs", video_id)
135     os.makedirs(out_local, exist_ok=True)
136     _write_intervals_csv(segments, os.path.join(out_local, "intervals.csv"))
137     _write_report_json(
138         video_id,
139         segments,
140         status,
141         os.path.join(out_local, "report.json"),
142         video_uri=video_uri,
143     )
144     out_prefix = out_uri.rstrip("/")
145     pushed: List[str] = []
146     for fn in sorted(os.listdir(out_local)):
147         uri = f"{out_prefix}/outputs/{video_id}/{fn}"
148         storage.put(os.path.join(out_local, fn), uri, config=storage_cfg)
149         pushed.append(uri)
150     return pushed
151
152
153 def _run_startup_checks(config: Any) -> bool:
154     """M5 per-profile startup checks for the worker (the compute process, so it probes
CUDA
155     best-effort). Logs warnings; returns False on a FATAL coherence error so the caller
aborts
156     cleanly (rc 2) rather than run an expensive job under an incoherent profile. A true
no-op
157     (returns True, ZERO work) when no deployment.profile is selected.
158
159     The CUDA probe is gated behind an active profile: ``probe_cuda_available()`` lazily
imports

```

107. assistant (2026-07-07T14:58:07.869Z)

Decisive finding: ``\_apply\_overrides`` (line 61-71) does ``setattr`` on the typed config, so an old worker receiving a ``--set validation.xxx`` key it doesn't know would *also* fail before writing a marker ? meaning ``--set`` is *not* more graceful than the flag for skew. So switching

mechanisms buys no robustness; I'll keep the clean `--skip-validation` flag and mitigate the skew exactly as #507 did ? a deploy-together note.

Let me read the test conventions and the existing #507 deploy note to fix both findings.

108. user (2026-07-07T14:58:09.182Z)

```
175         job_name="rally-worker",
176         region="asia-south1",
177         project="proj",
178         storage_config={},
179         policy=_FAST,
180         token="tok",
181     )
182     out = str(tmp_path / "bucket")
183     result = backend.run_infer(
184         ComputeJobSpec(
185             video_uri="gs://b/videos/clip.mp4", out_uri=out, segmenter="wasb_hybrid"
186         )
187     )
188
189     assert result.returncode == 0
190     assert result.video_id == "abc123"
191     assert result.outputs_uri == f"{out}/outputs/abc123/"
192     assert result.report.get("segment_count") == 3
193     # exactly one Cloud Run Job execution was triggered, in the right region/project
194     assert len(client.calls) == 1
195     assert (
196         client.calls[0]["job"] == "rally-worker"
197         and client.calls[0]["region"] == "asia-south1"
198     )
199
200
201     def test_dispatch_argv_forces_inline_and_carries_overrides(tmp_path):
202         """The dispatched container must run INLINE (never re-dispatch) ?
compute_target=local_gpu +
203         native detector appended AFTER the user overrides (so they win); --done-uri always
present."""
204         client = _FakeRunClient()
205         backend = CloudRunCompute(
206             run_client=client, job_name="j", region="r", policy=_FAST, token="tok"
207         )
208         backend.run_infer(
209             ComputeJobSpec(
210                 video_uri="gs://b/v.mp4",
211                 out_uri=str(tmp_path / "bk"),
212                 segmenter="wasb_hybrid",
213                 config_overrides=("sport=tennis",),
214             )
215         )
216
217         argv = client.calls[0]["argv"]
218         assert argv[0] == "infer"
219         assert "--done-uri" in argv and "--video-uri" in argv and "--out-uri" in argv
220         assert argv[argv.index("--segmenter") + 1] == "wasb_hybrid"
221         # the container-inline overrides come last (win) and the user override is carried
through
222         joined = " ".join(argv)
223         assert "deployment.compute_target=local_gpu" in joined
224         assert "indexing.detector_impl=native" in joined
225         assert "sport=tennis" in joined
226         # user override precedes the inline-forcing overrides in --set order
227         assert joined.index("sport=tennis") < joined.index(
228             "deployment.compute_target=local_gpu"
229         )
```

```

230
231
232     def test_unique_token_per_call_when_not_injected(tmp_path):
233         """No injected token ? a fresh uuid per dispatch, so concurrent jobs don't collide
on the marker."""
234         client = _FakeRunClient()
235         backend = CloudRunCompute(run_client=client, job_name="j", region="r", policy=_FAST)
236         backend.run_infer(
237             ComputeJobSpec(video_uri="gs://b/v.mp4", out_uri=str(tmp_path / "bk"))
238         )
239         backend.run_infer(
240             ComputeJobSpec(video_uri="gs://b/v.mp4", out_uri=str(tmp_path / "bk"))
241         )
242         done1 = client.calls[0]["argv"][client.calls[0]["argv"].index("--done-uri") + 1]
243         done2 = client.calls[1]["argv"][client.calls[1]["argv"].index("--done-uri") + 1]
244         assert done1 != done2

```

109. assistant (2026-07-07T14:58:16.439Z)

Test conventions confirmed. Let me fix ****Finding 2**** (test gap): add a negative assertion to the existing builder test and a paired positive builder test.

110. user (2026-07-07T14:58:21.500Z)

The file C:\Users\avidu\Projects\khelsutra-guru\bhi-wt-worker-reval\tests\test_compute_backend.py has been updated successfully. (file state is current in your context ? no need to Read it back)

111. user (2026-07-07T14:58:28.792Z)

The file C:\Users\avidu\Projects\khelsutra-guru\bhi-wt-worker-reval\tests\test_compute_backend.py has been updated successfully. (file state is current in your context ? no need to Read it back)

112. assistant (2026-07-07T14:58:32.254Z)

Now ****Finding 1**** (rollout skew) ? let me read the existing #507 deploy note in the README to match its style and placement.

113. user (2026-07-07T14:58:33.114Z)

```

40     ``
41     - **`--no-gpu-zonal-redundancy` is REQUIRED on the CLI**, not just a cost choice (it
also halves
42         the GPU rate ? right for batch jobs): without the flag `gcloud` asks an interactive
Y/n which,
43         in a non-interactive shell, **hangs forever with zero output and nothing created**. An
empty
44         `gcloud` that "did nothing" ? suspect a hidden prompt.
45     - 8 vCPU / 32 GiB: x264 encode is CPU-side, so CPU headroom keeps the paid GPU busy.
`/tmp` is
46     **RAM-backed** ? the pulled input video lives in memory, so size `--memory` for the
largest
47         expected upload (a 5.8 GB GoPro file ? 6 GiB of tmpfs).
48
49     ## 2a. GPU-resident WASB decode (#506/#507) ? the current serving default
50     The image ships the **py3.10 + torch 2.6.0+cu124 + PyNvVideoCodec==2.1.0** WASB env with
51     `libnvcuid`/`libnvidia-encode` **baked in** (multi-stage layer from the matching NVIDIA
driver ?
52     Cloud Run injects the host driver, not these userspace codec libs).
`indexing.wasb.decode_backend`
53     selects the decode path:
54     - `cpu` (model default) ? the historical cv2 decode; **byte-unchanged** by this work.
55     - `nvdec_resident` ? NVDEC decode + cv2-faithful affine `grid_sample` resize + GPU

```



```

normalize, all
56      on-GPU. **The cloud-serving preset default** (validated: offline golden gate @ SERVED
F1 0.574 ?
57      baseline 0.582, AND on the live L4 the served GX010128 trajectory reproduced the VM
run ?
58      26 479 ? 26 481 visible). CUDA-only (config-validator-enforced).
59
60      **How the knob reaches the worker.** The worker Job runs with NO `DEPLOYMENT_PROFILE`,
so it does
61      **not** apply the cloud-serving preset itself. The control plane (which *does* have the
profile)
62      resolves the config and **forwards**
`indexing.wasb.{decode_backend,batch_size,prefetch_batches}`
63      to the worker as `--set` overrides (`orchestration.py`, alongside `skip_ai_handoff`). So
the
64      **preset is the single source of truth** ? changing `presets.py` + redeploying the
control plane
65      changes serving; no per-Job knob needed.
66
67      **Override / rollback lever (no redeploy).** The native runner reads
`WASB_DECODE_BACKEND` FIRST,
68      so a Job env var wins over the forwarded `--set`:
69      ```bash
70      # force nvdec (or pin cpu) on the Job regardless of the forwarded config; instant
rollback = remove it
71      gcloud run jobs update rally-worker --region $REGION --update-env-vars
WASB_DECODE_BACKEND=nvdec_resident
72      gcloud run jobs update rally-worker --region $REGION --remove-env-vars
WASB_DECODE_BACKEND # ? forwarded/preset default
73      # full revert: pin the prior image by digest
74      gcloud run jobs update rally-worker --region $REGION --image $REGION-
docker.pkg.dev/$PROJECT/$REPO/rally-worker@sha256:<prev>
75      ```
76      ? Forwarding `decode_backend=nvdec_resident` requires a worker image that understands
77      `--decode-backend` (? #507). Roll the **worker image and the control plane together** ?
a new
78      control plane forwarding nvdec to an old worker image argparse-fails (rc 2).
79
80      **Post-deploy smoke.** After a cutover, confirm the real L4 path with
81      `deploy/cloudrun/post_deploy_smoke.ps1` (run from PowerShell ? see $Gotchas on MSYS arg-
mangling):
82      ```powershell
83      ./deploy/cloudrun/post_deploy_smoke.ps1 -Video gs://khelsutra-rally-
corpus/videos/mahadevpura_smoke_180s.mp4 -Decode cpu # image sanity (fast)
84      ./deploy/cloudrun/post_deploy_smoke.ps1 -Video gs://khelsutra-rally-
corpus/videos/GX010128.MP4 -Decode env # nvdec via preset/env (hard clip)
85      ```
86
87      ## 3. Wire the dispatcher (control plane)
88      The control plane runs with the **cloud-serving** profile (`DEPLOYMENT_PROFILE=cloud-
serving` ?
89      `compute_target=cloud_run`). `get_compute_backend(config)` then builds `CloudRunCompute`
from env:
90      ```bash
91      export DEPLOYMENT_PROFILE=cloud-serving
92      export CLOUD_RUN_JOB=rally-worker CLOUD_RUN_REGION=$REGION
GOOGLE_CLOUD_PROJECT=$PROJECT
93      # A `worker infer` (or the API job path) with a gs:// video now DISPATCHES to the Job
instead of
94      # running in-process; the container runs the SAME worker INLINE (compute_target forced
local_gpu).
95      python -m backend.worker infer --video-uri gs://<bucket>/videos/clip.mp4 --out-uri
gs://<bucket> --segmenter wasb_hybrid
96      ```
97      **The AI handoff needs its key IN THE JOB env** (`GEMINI_API_KEY` ? mount a Secret

```

Manager secret
98 on the job), or run fully-local with `--set indexing.skip\_ai\_handoff=true` (candidate
windows
99 emitted as rallies ? the eval-parity mode). With neither, the segmenter used to bail to
a silent
100 0-rally "success" **after** the paid GPU pass; the cloud-serving startup checks now fail
this at
101 boot (`AI\_HANDOFF\_KEY\_MISSING`, rc 2) before the video is even pulled.
102
103 ### 3a. Control-plane SERVICE deploy ? required flags for reliable async jobs
104
105 The API server (`/api/process`, `/api/compile` ? #329) runs its **job worker IN-
PROCESS**: the
106 request returns `202` and a background thread drains the job (GPU detection dispatches
to the Job;
107 the reel stitch runs on the control plane itself). On Cloud Run that background thread
only makes
108 progress while the instance has CPU, so a scale-to-zero **service** deploy MUST set:
109

114. assistant (2026-07-07T14:58:41.353Z)

The README already has a \$2a deploy-together warning for #507's `--decode-backend`. I'll add a parallel note for `--skip-validation` right after it, matching the style:

115. user (2026-07-07T14:58:52.542Z)

The file C:\Users\avidu\Projects\khelsutra-guru\bhi-wt-worker-reval\deploy\cloudrun\README.md has been updated successfully. (file state is current in your context ? no need to Read it back)

116. assistant (2026-07-07T14:59:08.529Z)

Now let me flag the larger forward-looking hardening (making the dispatcher detect a worker crash-before-marker fast, instead of polling the full budget) as an out-of-scope follow-up, then re-run the checks.

117. user (2026-07-07T14:59:19.542Z)

Noted (position 1, task_id: task_4e8bc546). A chip is showing for the user ? they can start it in a fresh worktree with one click, or dismiss it. If this suggestion becomes stale or superseded, call dismiss_task with this task_id. Currently pending: task_4e8bc546 "Dispatcher: fail fast on worker crash-before-marker". Continue your current work.

118. assistant (2026-07-07T14:59:22.230Z)

Now re-run the targeted tests for the changed files plus ruff/mypy:

119. user (2026-07-07T15:00:16.791Z)

```
tests/test_api_cloud_dispatch.py::test_full_api_process_to_query_via_cloud
tests/test_api_cloud_dispatch.py::test_local_override_survives_api_boundary
tests/test_api_cloud_dispatch.py::test_local_override_survives_api_boundary
tests/test_api_cloud_dispatch.py::test_cloud_override_survives_api_boundary_on_local_server
tests/test_api_cloud_dispatch.py::test_cloud_override_survives_api_boundary_on_local_server
tests/test_api_cloud_dispatch.py::test_full_api_process_job_result_carries_cloud_provenance
tests/test_api_cloud_dispatch.py::test_full_api_process_job_result_carries_cloud_provenance
C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\site-
packages\google\auth\_default.py:113: UserWarning: Your application has authenticated using end
user credentials from Google Cloud SDK without a quota project. You might receive a "quota
exceeded" or "API not enabled" error. See the following page for troubleshooting:
https://cloud.google.com/docs/authentication/adc-troubleshooting/user-creds.
warnings.warn(_CLOUD_SDK_CREDENTIALS_WARNING)
```

-- Docs: <https://docs.pytest.org/en/stable/how-to/capture-warnings.html>

86 passed, 8 warnings in 45.03s

===== RUFF =====

All checks passed!

===== MYPY =====

backend\sports__init__.py:9: note: By default the bodies of untyped functions are not checked, consider using --check-untyped-defs [annotation-unchecked]
backend\pipeline\validators\base.py:44: note: By default the bodies of untyped functions are not checked, consider using --check-untyped-defs [annotation-unchecked]
backend\providers__init__.py:9: note: By default the bodies of untyped functions are not checked, consider using --check-untyped-defs [annotation-unchecked]
Success: no issues found in 148 source files

120. assistant (2026-07-07T15:00:37.794Z)

All green: **86 passed** (+1 new positive test), ruff clean, mypy clean (148 files). Production code is unchanged since the last full-suite green run ? only tests and the README were added ? so let me re-run the full suite with coverage in the background to confirm the complete gate honestly, and in parallel verify git state before committing.

121. user (2026-07-07T15:00:40.739Z)

Command running in background with ID: b6w37cbl0. Output is being written to:
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\902e27a6-e026-4b55-8c26-f2c47329cc44\tasks\b6w37cbl0.output. You will be notified when it completes. To check interim output, use Read on that file path.

122. user (2026-07-07T15:00:51.574Z)

===== branch =====
fix/worker-skip-revalidation-on-dispatch
===== HEAD (mine) =====
3be0935 docs(runlog): DEPLOY_REPORT addendum ? control-plane redeployed to the forwarding image (#510)
===== origin/master tip =====
3be0935 docs(runlog): DEPLOY_REPORT addendum ? control-plane redeployed to the forwarding image (#510)
===== commits on my branch vs origin/master (should be empty pre-commit) =====
===== working tree status =====
M backend/compute/base.py
M backend/compute/cloud_run.py
M backend/orchestration.py
M backend/worker/__main__.py
M deploy/cloudrun/README.md
M tests/test_api_cloud_dispatch.py
M tests/test_compute_backend.py
M tests/test_worker.py

123. assistant (2026-07-07T15:01:05.539Z)

Git state is clean and isolated: branch is mine, HEAD = `origin/master` (unchanged ? master hasn't moved), no commits yet, and only my 8 files are modified (the sibling's WIP did not leak into this worktree). Let me finalize the commit message to include the review-driven additions, then stage explicit paths and commit (local; I'll push after the full suite confirms).

124. assistant (2026-07-07T15:01:19.664Z)

fix(serving): worker skips redundant re-validation on the cloud dispatch path

On the cloud-serving path the control-plane is the AUTHORITATIVE validation gate: backend/orchestration.py runs the validators with its resolved config and only dispatches to the Cloud Run worker on PASS (a failure returns "failed" before _maybe_dispatch_remote). The worker's cmd_infer then RE-RAN validation with its OWN config ? and since the control-plane's resolved validation.* keys are not forwarded on dispatch (#509 forwards only skip_ai_handoff + the wasb.*

knobs), the worker used its default thresholds. Contradictory decisions result.

Live 2026-07-07 CUJ: a 5312x2988 10-bit HEVC clip (sharpness 11.9) PASSED the control-plane at validation.min_blur_threshold=8.0, then the worker REJECTED it at its default 20.0 ? "[worker] validation FAILED: blur_check ... 11.9 < 20.0", rc=2. A successful GPU dispatch failed after the fact.

Fix (skip-on-dispatch ? also saves the redundant second full-video decode):

- ComputeJobSpec.skip_validation (new bool, default False)
- CloudRunCompute emits a "--skip-validation" worker flag when set
- worker cmd_infer honors it: skips validator_registry.run_all_with_context (val_results=[], val_context={}); the direct `worker infer` CLI still validates by default (no control-plane there)
- _maybe_dispatch_remote sets skip_validation=True on every dispatch (only a validated PASS ever reaches it)
- _dispatch_remote propagates the flag through the worker->worker recursion path

val_context feeds only the observability report (ffprobe_meta), not segmentation; the report.json cache consumers (probe_process_cache / cached_process_result) read only status/video_uri ? so skipping is functionally safe for detection and ingest.

Rollout (adversarial-review follow-up): --skip-validation is a NEW worker CLI arg, so a control-plane-ahead-of-worker deploy makes an old worker image argparse-abort (rc 2) BEFORE writing the done-marker ? the dispatcher polls the full remote_poll_max_wait_sec (~65 min) then fails opaquely. Documented in deploy/cloudrun/README.md §2a (roll worker image + control plane together, worker-first ? same coupling #507 has for --decode-backend). A follow-up to make the dispatcher fail fast on any worker crash-before-marker is filed separately.

Tests:

- tests/test_worker.py: parser accepts --skip-validation (default off); cmd_infer under the flag never touches the validators and a would-fail clip still succeeds rc=0.
- tests/test_api_cloud_dispatch.py: the control-plane dispatch argv carries --skip-validation.
- tests/test_compute_backend.py: a DEFAULT spec omits the flag (guards the `if spec.skip\_validation:` gate) and skip_validation=True emits it.

Full suite 1779+ passed / coverage 85.4% / ruff + mypy clean.

Co-Authored-By: Claude Opus 4.8 <noreply@anthropic.com>

125. user (2026-07-07T15:01:19.701Z)

The file C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\902e27a6-e026-4b55-8c26-f2c47329cc44\scratchpad\commit_msg.txt has been updated successfully. (file state is current in your context ? no need to Read it back)

126. user (2026-07-07T15:01:25.578Z)

==== staged =====

backend/compute/base.py	7 ++++++
backend/compute/cloud_run.py	4 ++++
backend/orchestration.py	7 ++++++
backend/worker/__main__.py	32 ++++++-----
deploy/cloudrun/README.md	12 ++++++
tests/test_api_cloud_dispatch.py	16 ++++++
tests/test_compute_backend.py	24 ++++++
tests/test_worker.py	45 ++++++

8 files changed, 143 insertions(+), 4 deletions(-)

==== re-verify branch/base right before commit =====

fix/worker-skip-revalidation-on-dispatch

127. assistant (2026-07-07T15:01:30.292Z)

Staged exactly my 8 files; base still clean (no sibling commits). Committing now with the prepared message:

128. user (2026-07-07T15:01:32.991Z)

```
[fix/worker-skip-revalidation-on-dispatch 8586c34] fix(serving): worker skips redundant re-validation on the cloud dispatch path
 8 files changed, 143 insertions(+), 4 deletions(-)
===== HEAD now =====
8586c34 fix(serving): worker skips redundant re-validation on the cloud dispatch path
```

129. assistant (2026-07-07T15:01:52.749Z)

Committed as `8586c34`, and `origin/master..HEAD` lists **only my commit** ? no sibling work rode in. Now let me check whether I can push + open a PR (the expected deliverable, matching the owner's #507/#509/#510 workflow), while the full suite finishes.

130. user (2026-07-07T15:01:58.541Z)

```
===== remotes =====
origin https://github.com/Khelsutra/badminton-highlight-indexer.git (fetch)
origin https://github.com/Khelsutra/badminton-highlight-indexer.git (push)
===== gh version =====
gh version 2.92.0 (2026-04-28)
===== gh auth =====
github.com
? Logged in to github.com account avidullu (keyring)
- Active account: true
- Git operations protocol: https
- Token: gho_*****
- Token scopes: 'gist', 'read:org', 'repo', 'workflow'
```

131. assistant (2026-07-07T15:02:03.209Z)

`gh` is authenticated with `repo` scope. I'll push + open the PR once the full suite confirms green. Let me peek at the suite's progress.

132. user (2026-07-07T15:02:05.585Z)

```
1 =====
2 Full test suite: C:\Users\avidu\AppData\Local\Programs\Python\Python313\python.exe -m
pytest -q --cov=backend --cov-report=term-missing --cov-fail-under=70
3 obsreport (optional reporting dep): available
4 =====
5 .....SS..... [ 4%]
6 .....
```

133. assistant (2026-07-07T15:02:09.643Z)

The suite is running. Let me prepare the PR body so it's ready the moment the suite confirms green.

134. assistant (2026-07-07T15:02:31.493Z)

Problem

On the cloud-serving path, a video that **passes** the control-plane's validation gets **rejected** by the worker's redundant re-validation, because the control-plane's validation config is not forwarded to the worker.

The control-plane (`backend/orchestration.py`) runs the validators with **its** resolved config

and only dispatches to the Cloud Run worker on **PASS** (a failure returns `"failed"` *before* `_maybe_dispatch_remote`). But the worker's `cmd_infer` then **re-ran** validation with **its** own config. Since #509 forwards only `skip_ai_handoff` + the `wasb.` knobs (no `validation.`), the worker used its **default** thresholds ? contradictory decisions.

```
Live 2026-07-07 CUJ ("Video 1 - Badminton.MP4", 5312x2988 10-bit HEVC, sharpness 11.9):
- control-plane baked `validation.min_blur_threshold=8.0` ? PASSED, dispatched to `rally-worker`.
- worker re-validated at its default `20.0` ? `[worker] validation FAILED: blur_check: Average sharpness 11.9 < minimum 20.0`, rc=2.
- Result: `[COMPUTE] requested=cloud actual=cloud_run dispatched=True status=failed` ? the CUJ failed after a successful GPU dispatch.
```

This was redundant (already validated), incorrect (different thresholds), and wasteful (a second full-video decode).

Fix ? skip-on-dispatch (the preferred option)

The control-plane is the authoritative gate, so the worker is told to **skip** its re-validation on the dispatch path (this also saves the redundant decode). The direct `worker infer` CLI (no control-plane) still validates by default.

```
- `ComputeJobSpec.skip_validation` ? new bool, default `False` (`backend/compute/base.py`)
- `CloudRunCompute.run_infer` emits a `--skip-validation` worker flag when set
  (`backend/compute/cloud_run.py`)
- worker `cmd_infer` honors it ? skips `validator_registry.run_all_with_context`
  (`val_results=[]`, `val_context={}`) (`backend/worker/__main__.py`)
- `_maybe_dispatch_remote` sets `skip_validation=True` on every dispatch ? only a validated
  PASS ever reaches it (`backend/orchestration.py`)
- `_dispatch_remote` propagates the flag through the worker?worker recursion path
```

Safety checked: `val_context` feeds only the observability report (`ffprobe_meta`), not segmentation; the `report.json` cache consumers (`probe_process_cache` / `cached_process_result`) read only `status` / `video_uri`. So skipping is functionally safe for detection and ingest.

Acceptance

> a video that passes the control-plane gate is NOT rejected by the worker on the cloud path

Covered by tests:

```
- `tests/test_api_cloud_dispatch.py::test_cloud_dispatch_skips_worker_revalidation` ? the
  control-plane dispatch argv carries `--skip-validation`.
- `tests/test_worker.py::test_cmd_infer_skip_validation_bypasses_validators` ? a clip that
  would fail re-validation still succeeds rc=0 (the validators are never invoked);
  `test_parser_infer_skip_validation_flag` ? default-off.
- `tests/test_compute_backend.py` ? a default spec omits the flag (guards the `if
  spec.skip_validation:` gate); skip_validation=True emits it.
```

Adversarial review follow-ups (this PR)

A multi-agent review of the diff surfaced two confirmed items, both addressed here:

- Rollout skew** ? `--skip-validation` is a *new* worker CLI arg, so a control-plane-ahead-of-worker deploy makes an old worker image `argparse-abort` (rc 2) **before** writing the done-marker ? the dispatcher bucket-polls the full `remote_poll_max_wait_sec` (~65 min) then fails opaquely. Documented in `deploy/cloudrun/README.md` §2a (**roll** worker image + control plane together, **worker-first** ? the same coupling #507 has for `--decode-backend`). A separate follow-up is filed to make the dispatcher fail **fast** on any worker crash-before-marker (consult the Cloud Run execution status), which fixes the whole class rather than just this flag.
- Test gap** ? added a negative builder assertion so a "drop the guard / append unconditionally" refactor can't stay green.

Pre-merge gate (verified locally)

- Full suite: ****1779+ passed, 8 skipped**** (obsreport installed so reporting tests run), 0 failures.
- Coverage: ****85.4%**** (floor 70; no regression vs base).
- ``ruff check .` clean` · ``mypy backend training` clean` (148 files).

Related: #509 (wasb-knob forwarding); the "Guard compute=local on the hosted control-plane" task.

? Generated with [Claude Code](https://claude.com/claude-code)

135. user (2026-07-07T15:02:31.544Z)

File created successfully at: C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\902e27a6-e026-4b55-8c26-f2c47329cc44\scratchpad\pr_body.md (file state is current in your context ? no need to Read it back)

136. assistant (2026-07-07T15:02:33.830Z)

I'll wait for the full-suite completion notification, then push and open the PR.

137. user (2026-07-07T15:04:43.185Z)

```
<task-notification>
<task-id>b6w37cbl0</task-id>
<tool-use-id>toolu_01M33mvZthGzQDQL6newwxRE</tool-use-id>
<output-file>C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\902e27a6-e026-4b55-8c26-f2c47329cc44\tasks\b6w37cbl0.output</output-file>
<status>completed</status>
<summary>Background command "Re-run full suite with coverage (background)" completed (exit code 0)</summary>
</task-notification>
```

138. assistant (2026-07-07T15:04:47.172Z)

Full suite completed (exit 0). Let me confirm the final counts and coverage.

139. user (2026-07-07T15:04:49.240Z)

1780 passed, 8 skipped, 25 warnings in 237.15s (0:03:57)

```
=====
RESULT: PASS - all tests green
=====
```

140. user (2026-07-07T15:05:00.638Z)

TOTAL	12135	1769	85%
-------	-------	------	-----

Required test coverage of 70% reached. Total coverage: 85.42%

141. assistant (2026-07-07T15:05:12.462Z)

Gate fully confirmed with the complete change set: ****1780 passed, 8 skipped, 0 failures****, coverage ****85.42%****, ruff + mypy clean. Let me re-verify the branch base one last time and push.

142. user (2026-07-07T15:05:14.252Z)

Exit code 128
fatal: not a git repository (or any of the parent directories): .git

143. assistant (2026-07-07T15:05:23.560Z)

The Bash cwd reset to the non-git stub dir. Let me use ``git -C <worktree>`` explicitly to avoid

cwd ambiguity.

144. user (2026-07-07T15:05:30.980Z)

```
===== origin/master tip =====
3be0935 docs(runlog): DEPLOY_REPORT addendum ? control-plane redeployed to the forwarding image
(#510)
===== my commits vs origin/master (must be ONLY mine) =====
8586c34 fix(serving): worker skips redundant re-validation on the cloud dispatch path
===== push =====
remote:
remote: Create a pull request for 'fix/worker-skip-revalidation-on-dispatch' on GitHub by
visiting:
remote:      https://github.com/Khelsutra/badminton-highlight-indexer/pull/new/fix/worker-skip-
revalidation-on-dispatch
remote:
branch 'fix/worker-skip-revalidation-on-dispatch' set up to track 'origin/fix/worker-skip-
revalidation-on-dispatch'.
To https://github.com/Khelsutra/badminton-highlight-indexer.git
* [new branch]      fix/worker-skip-revalidation-on-dispatch -> fix/worker-skip-revalidation-
on-dispatch
```

145. assistant (2026-07-07T15:05:34.677Z)

Pushed cleanly (base unchanged, only my commit). Now let me open the PR.

146. user (2026-07-07T15:05:42.098Z)

<https://github.com/Khelsutra/badminton-highlight-indexer/pull/513>

147. assistant (2026-07-07T15:06:05.381Z)

****PR #513 opened:**** <https://github.com/Khelsutra/badminton-highlight-indexer/pull/513>

Let me record this checkpoint in memory (a PR was pushed, and there's a live cross-session hazard worth flagging for a future session), then update the index.

148. assistant (2026-07-07T15:06:20.915Z)

```
---
name: pr513-worker-skip-revalidation
description: "PR #513 ? worker skips redundant re-validation on the cloud dispatch path (fixes
the 2026-07-07 blur-threshold CUJ failure)"
metadata:
  type: project
---
```

****PR #513 OPEN**** (2026-07-07): <https://github.com/Khelsutra/badminton-highlight-indexer/pull/513>
Branch `fix/worker-skip-revalidation-on-dispatch` (from origin/master @ 3be0935, commit 8586c34).

****Bug:**** on the cloud-serving path the control-plane validates + only dispatches on PASS, but the worker's `cmd\_infer` RE-RAN validation with its OWN config (the control-plane's resolved `validation.\*` keys aren't forwarded ? `[[gpu-resident-modernization]]/509` forwards only `skip\_ai\_handoff` + `wasb.\*`).

Live CUJ 2026-07-07: a 5312x2988 clip PASSED at `min\_blur\_threshold=8.0`, worker REJECTED at default 20.0 ? successful GPU dispatch, then `status=failed`.

****Fix (skip-on-dispatch):**** new `ComputeJobSpec.skip\_validation` ? `CloudRunCompute` emits a `--skip-validation` worker flag ? `cmd\_infer` honors it (skips `run\_all\_with\_context`) ? `\_maybe\_dispatch\_remote` sets it True on every dispatch. Direct `worker infer` CLI still

validates.

Gate: 1780 passed / cov 85.42% / ruff + mypy clean.

****Rollout caveat (in the PR + README \$2a):**** `--skip-validation` is a NEW worker CLI arg ? a control-plane-ahead-of-worker deploy makes an OLD worker image `argparse-abort` (rc 2) BEFORE the done-marker ? dispatcher polls the full `remote_poll_max_wait_sec` (~65 min) then fails opaquely.

Deploy ****worker image first****, together with the control plane (same coupling #507 has for `--decode-backend`). Follow-up task filed (task_4e8bc546): make the dispatcher fail FAST on any worker crash-before-marker by consulting the Cloud Run execution status.

****? Concurrent-session hazard (why I used an isolated worktree):**** a SIBLING session is actively editing the SHARED checkout `../khelsutra-guru/badminton-highlight-indexer` on branch `perf/validation-shared-decode-pass` ? a validators shared-decode-pass refactor (base.py + frame_sampler.py + blur/relevance/scene_cut) AND the "Guard compute=local on the hosted control-plane" task (orchestration.py `_WEIGHTS_BACKED_SEGMENTERS`). Its tree mutated under me between commands. I did #513 in a separate `git worktree` at `../khelsutra-guru/bhi-wt-worker-reval` (still on disk, branch pushed) so nothing clobbered. ****That sibling branch also touches `orchestration.py` + `test_api_cloud_dispatch.py` ? expect a merge conflict between it and #513; whichever merges second rebases.**** See `[[actual-repo-location]]`.

149. user (2026-07-07T15:06:20.981Z)

File created successfully at: C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\memory\pr513-worker-skip-revalidation.md (file state is current in your context ? no need to Read it back)

150. user (2026-07-07T15:06:24.667Z)

```
1      # Memory index
2
3      - ? **Start here (GPU-resident rollout):** [session-handoff-gpu-resident.md](session-handoff-gpu-resident.md) ? #506 implemented, PR #507 open; next: on-box parity gate + 15-clip golden gate.
4
5      - [Actual repo location](actual-repo-location.md) ? primary working dir is an empty STUB; real code is under khelsutra-guru/badminton-highlight-indexer.
6      - [Efficient timeouts, retries & Windows gcloud SSH](efficient-timeouts-and-retries.md) ? no silent retry loops; plain gcloud compute ssh (plink/pscp reject -o, mishandle ~); diagnose-first, run long work detached.
7      - [GPU VM setup gotchas](gpu-vm-setup-gotchas.md) ? NVDEC libs, apt-lock-at-boot, torch2.x/pynvc; what #501 should encode.
8      - [LGTM only when merge-ready](lgtm-only-when-merge-ready.md) ? post LGTM only when genuinely satisfied a PR is merge-ready (verified), for all PRs.
9      - [GPU-resident modernization status](gpu-resident-modernization.md) ? #506 validated clean; the resize must match cv2.warpAffine via grid_sample (not F.interpolate).
10     - [compute=local hosted guardrail](compute-local-hosted-guardrail.md) ? two gated-but-UNPUSHED branches (backend 4565c74 + SPA 1470e2a) fixing the "weights not found" trap; contended checkout warning.
11     - [Validation shared decode pass](validation-shared-decode-pass.md) ? validators now share ONE decode pass (branch perf/validation-shared-decode-pass, d53851b, local); pure grab()-through is SLOWER than seeks ? use forward-hybrid.
12
```

151. assistant (2026-07-07T15:06:29.371Z)

The index already tracks the sibling branches (lines 10-11) ? confirming my read of the concurrent work. Let me add the #513 pointer.

152. user (2026-07-07T15:06:33.117Z)

<tool_use_error>File has been modified since read, either by the user or by a linter. Read it again before attempting to write it.</tool_use_error>

153. assistant (2026-07-07T15:06:41.445Z)

The sibling session just wrote to MEMORY.md too. Let me re-read and merge my entry carefully.

154. user (2026-07-07T15:06:41.852Z)

```
1      # Memory index
2
3      - ? **Start here (GPU-resident rollout):** [session-handoff-gpu-resident.md](session-
handoff-gpu-resident.md) ? #506 implemented, PR #507 open; next: on-box parity gate + 15-clip
golden gate.
4
5      - [Actual repo location](actual-repo-location.md) ? primary working dir is an empty
STUB; real code is under `khelsutra-guru/badminton-highlight-indexer`.
6      - [Efficient timeouts, retries & Windows gcloud SSH](efficient-timeouts-and-retries.md)
? no silent retry loops; plain `gcloud compute ssh` (plink/pscp reject `-o`, mishandle `~`);
diagnose-first, run long work detached.
7      - [GPU VM setup gotchas](gpu-vm-setup-gotchas.md) ? NVDEC libs, apt-lock-at-boot,
torch2.x/pynvc; what #501 should encode.
8      - [LGTM only when merge-ready](lgtm-only-when-merge-ready.md) ? post LGTM only when
genuinely satisfied a PR is merge-ready (verified), for all PRs.
9      - [GPU-resident modernization status](gpu-resident-modernization.md) ? #506 validated
clean; the resize must match cv2.warpAffine via grid_sample (not F.interpolate).
10     - [compute=local hosted guardrail](compute-local-hosted-guardrail.md) ? two gated-but-
UNPUSHED branches (backend 4565c74 + SPA 1470e2a) fixing the "weights not found" trap; contended
checkout warning.
11     - [Validation shared decode pass](validation-shared-decode-pass.md) ? validators now
share ONE decode pass (branch `perf/validation-shared-decode-pass`, d53851b, **PR #512** open);
pure grab()-through is SLOWER than seeks ? use forward-hybrid.
12
```

155. user (2026-07-07T15:06:52.353Z)

The file C:\Users\avidu\claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\memory\MEMORY.md has been updated successfully. (file state is current in your context ? no need to Read it back)

156. user (2026-07-07T15:06:59.411Z)

The file C:\Users\avidu\claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\memory\pr513-worker-skip-revalidation.md has been updated successfully. (file state is current in your context ? no need to Read it back)

157. assistant (2026-07-07T15:07:21.851Z)

Done. Here's the summary.

Fixed: worker no longer re-rejects videos the control-plane already passed

****PR #513:**** <https://github.com/Khelsutra/badminton-highlight-indexer/pull/513> (branch `fix/worker-skip-revalidation-on-dispatch`, from `origin/master`)

****Root cause**** (confirmed by reading the code, not the report): the control-plane validates and ****only dispatches on PASS**** ([orchestration.py:747](backend/orchestration.py:747)), but the worker's `cmd\_infer` unconditionally re-ran validation with its ***own*** config ([worker/___main__.py:263](backend/worker/___main__.py:263)). Since #509 forwards only `skip\_ai\_handoff` + `wasb.\*` (no `validation.\*`), the worker used its default `min\_blur\_threshold=20.0` and rejected the sharpness-11.9 clip the control-plane had passed at `8.0`.

****Fix (skip-on-dispatch ? the preferred option, also drops the redundant second decode):**** a new `ComputeJobSpec.skip_validation` ? `CloudRunCompute` emits a --skip-validation` worker flag ? cmd_infer` honors it ? _maybe_dispatch_remote` sets it True` on every dispatch. The direct worker infer` CLI still validates by default. I verified val_context` feeds only the observability report (not segmentation) and the report.json` cache consumers read only status`/video_uri`, so skipping is functionally safe.`

****Verification (all reproduced locally, not trusted from the description):****

- Full suite ****1780** passed, 8 skipped, 0 failures****** (with `obsreport` installed so reporting tests actually run)`
- Coverage ****85.42%**** (floor 70, no regression) . `ruff check .` clean .` mypy backend training` clean (148 files)`

****Adversarial multi-agent review**** of the diff surfaced two confirmed items, both handled in the PR:

1. ****Rollout skew**** ? `--skip-validation` is a new worker CLI arg, so deploying the control-plane ahead of the worker makes an old worker image argparse-abort before the done-marker ? a ~65-min opaque poll-hang. Documented in [README §2a](deploy/cloudrun/README.md) (deploy worker-first, same coupling #507 has). I also filed a follow-up chip to make the dispatcher fail *fast* on any worker crash-before-marker.`
2. ****Test gap**** ? added a negative builder assertion so a future "drop the guard" refactor can't stay green.

Two things to flag

- ****I worked in an isolated `git worktree`, not the shared checkout**`** ? a sibling session is actively editing `.../badminton-highlight-indexer` on perf/validation-shared-decode-pass` (now **PR #512**), and its tree mutated under me mid-command. Nothing was clobbered. But **#512 also touches orchestration.py` and test_api_cloud_dispatch.py`, so it and #513 will merge-conflict** ? whichever lands second needs a rebase.`
- The worktree `.../khelsutra-guru/bhi-wt-worker-reval` is left on disk (branch pushed, PR open) in case you want to address review feedback there; say the word and I'll remove it.`

I did ****not**** post LGTM or merge anything ? that's your call after review.

158. user (2026-07-07T15:21:07.266Z)

<system-reminder>

The user started your suggested background task `task_4e8bc546` ("Dispatcher: fail fast on worker crash-before-marker") in a separate local session. It is running independently. You will be notified here when it ends.

</system-reminder>

I just LGTM'd the PR to merge. Can you merge it and update state

159. assistant (2026-07-07T15:21:23.463Z)

On it. Before merging I need to confirm #513 is actually mergeable ? master may have moved (a sibling could have merged #511 or #512, which overlap the same files), and I want CI green. Let me check.

160. user (2026-07-07T15:21:28.132Z)

==== PR #513 state =====

```
{"baseRefName":"master","headRefName":"fix/worker-skip-revalidation-on-dispatch","isDraft":false,"mergeStateStatus":"CLEAN","mergeable":"MERGEABLE","number":513,"reviewDecision":"","state":"OPEN","title":"fix(serving): worker skips redundant re-validation on the cloud dispatch path"}
```

==== checks =====

Built wheel smoke (install + launch) pass 1m6s <https://github.com/Khelsutra/badminton-highlight-indexer/actions/runs/28876634135/job/85653370038>

Dependency license scan (no copyleft in runtime deps) pass 37s

<https://github.com/Khelsutra/badminton-highlight-indexer/actions/runs/28876634135/job/85653369914>

Full test suite (offline, mocked) pass 2m44s <https://github.com/Khelsutra/badminton-highlight-indexer/actions/runs/28876634135/job/85653369914>

```
highlight-indexer/actions/runs/28876634135/job/85653370012
Golden fixtures cross-OS (macos-latest) pass    25s    https://github.com/Khelsutra/badminton-
highlight-indexer/actions/runs/28876634135/job/85653370206
Golden fixtures cross-OS (ubuntu-latest)    pass    26s
https://github.com/Khelsutra/badminton-highlight-
indexer/actions/runs/28876634135/job/85653370095
Golden fixtures cross-OS (windows-latest)    pass    1m3s
https://github.com/Khelsutra/badminton-highlight-
indexer/actions/runs/28876634135/job/85653370101
Leak scan (PII / attribution guard)    pass    9s    https://github.com/Khelsutra/badminton-
highlight-indexer/actions/runs/28876634135/job/85653370139
Markdown link check (live docs) pass    9s    https://github.com/Khelsutra/badminton-
highlight-indexer/actions/runs/28876634135/job/85653369906
Mypy (lenient baseline) pass    33s    https://github.com/Khelsutra/badminton-highlight-
indexer/actions/runs/28876634135/job/85653369930
Ruff lint    pass    8s    https://github.com/Khelsutra/badminton-highlight-
indexer/actions/runs/28876634135/job/85653369936
Syntax sweep + import smoke (no GPU stack)    pass    20s
https://github.com/Khelsutra/badminton-highlight-
indexer/actions/runs/28876634135/job/85653369979
```

161. assistant (2026-07-07T15:22:11.180Z)

PR #513 is ****MERGEABLE / CLEAN**** with all ****11 CI checks passing**** (full suite, ruff, mypy, golden fixtures x3 OS, wheel smoke, license/leak/link scans). You've LGTM'd it. Merging now with squash (the repo's convention for #507?#510):

162. user (2026-07-07T15:22:17.668Z)

```
===== post-merge state =====
{"mergeCommit":{"oid":"23c186813b980b421a471aca1c0fd292f408ae26"},"mergeStateStatus":"UNKNOWN",
mergedAt":"2026-07-07T15:22:15Z","state":"MERGED"}
```

163. assistant (2026-07-07T15:22:23.513Z)

Merged ? squash commit `23c1868`, state MERGED. Now let me confirm master advanced, then clean up the branch and worktree.

164. user (2026-07-07T15:22:35.733Z)

```
===== origin/master tip (should be the squash commit) =====
23c1868 fix(serving): worker skips redundant re-validation on the cloud dispatch path (#513)
3be0935 docs(runlog): DEPLOY_REPORT addendum ? control-plane redeployed to the forwarding image
(#510)
66756ad chore(serving): forward WASB decode/tuning knobs to worker + release hardening (#509)
===== delete remote branch =====
error: unable to delete 'fix/worker-skip-revalidation-on-dispatch': remote ref does not exist
error: failed to push some refs to 'https://github.com/Khelsutra/badminton-highlight-
indexer.git'
```

165. assistant (2026-07-07T15:22:58.297Z)

origin/master now has the squash commit `23c1868` (#513), and the remote branch was already auto-deleted by the merge. Now remove my (fully-merged) worktree and local branch:

166. user (2026-07-07T15:23:01.513Z)

```
Exit code 255
===== remove worktree =====
error: failed to delete 'C:/Users/avidu/Projects/khelsutra-guru/bhi-wt-worker-reval': Permission
denied
error: failed to delete '.git/worktrees/bhi-wt-worker-reval': Permission denied
```

167. assistant (2026-07-07T15:23:19.115Z)

Windows file-lock on the worktree dir. Let me check whether my shell's cwd is inside it (the usual cause) before retrying once.

168. user (2026-07-07T15:23:21.212Z)

```
shell cwd: /c/Users/avidu/Projects/badminton-highlight-indexer
---
total 257
drwxr-xr-x 1 avidu 197609      0 Jul  7 20:53 .
drwxr-xr-x 1 avidu 197609      0 Jul  7 19:57 ..
```

169. assistant (2026-07-07T15:23:56.099Z)

Shell cwd isn't the culprit (it's the stub) ? an external Windows process is holding the worktree dir. This is just cleanup clutter and the merge itself is done, so I won't spin on it. Let me try a force-remove once, and in parallel do the important state updates (memory).

170. user (2026-07-07T15:23:59.379Z)

```
fatal: 'C:/Users/avidu/Projects/khelsutra-guru/bhi-wt-worker-reval' is not a working tree
CLEANUP_DEFERRED (worktree dir is locked by another process)
```

171. user (2026-07-07T15:24:01.338Z)

The file C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\memory\pr513-worker-skip-revalidation.md has been updated successfully. (file state is current in your context ? no need to Read it back)

172. assistant (2026-07-07T15:24:11.595Z)

The admin entry is now in a half-removed state ("is not a working tree"). Let me prune stale worktree admin and check whether the branch is now free.

173. user (2026-07-07T15:24:19.601Z)

```
===== prune =====
Removing worktrees/a1-master-worktree: gitdir file does not exist
error: failed to delete '.git/worktrees/a1-master-worktree': Permission denied
Removing worktrees/bhi-wt-worker-reval: gitdir file does not exist
error: failed to delete '.git/worktrees/bhi-wt-worker-reval': Permission denied
Removing worktrees/fix-worktree: gitdir file does not exist
error: failed to delete '.git/worktrees/fix-worktree': Permission denied
Removing worktrees/master-compare: gitdir file does not exist
error: failed to delete '.git/worktrees/master-compare': Permission denied
Removing worktrees/pr465: gitdir file does not exist
error: failed to delete '.git/worktrees/pr465': Permission denied
Removing worktrees/pr465-current-merge: gitdir file does not exist
error: failed to delete '.git/worktrees/pr465-current-merge': Permission denied
Removing worktrees/pr465-merge: gitdir file does not exist
error: failed to delete '.git/worktrees/pr465-merge': Permission denied
Removing worktrees/pr467: gitdir file does not exist
error: failed to delete '.git/worktrees/pr467': Permission denied
Removing worktrees/pr469: gitdir file does not exist
error: failed to delete '.git/worktrees/pr469': Permission denied
Removing worktrees/wt-465fix: gitdir file does not exist
error: failed to delete '.git/worktrees/wt-465fix': Permission denied
Removing worktrees/wt-a1-runlog: gitdir file does not exist
error: failed to delete '.git/worktrees/wt-a1-runlog': Permission denied
Removing worktrees/wt-claudemd-r25: gitdir file does not exist
error: failed to delete '.git/worktrees/wt-claudemd-r25': Permission denied
Removing worktrees/wt-cloudrun-cancel: gitdir file does not exist
error: failed to delete '.git/worktrees/wt-cloudrun-cancel': Permission denied
Removing worktrees/wt-cloudrun-dispatch: gitdir file does not exist
error: failed to delete '.git/worktrees/wt-cloudrun-dispatch': Permission denied
```

```

Removing worktrees/wt-cp-rebuild-spec: gitdir file does not exist
error: failed to delete '.git/worktrees/wt-cp-rebuild-spec': Permission denied
Removing worktrees/wt-dockerfile-auth: gitdir file does not exist
error: failed to delete '.git/worktrees/wt-dockerfile-auth': Permission denied
Removing worktrees/wt-docs: gitdir file does not exist
error: failed to delete '.git/worktrees/wt-docs': Permission denied
Removing worktrees/wt-docs-reconcile: gitdir file does not exist
error: failed to delete '.git/worktrees/wt-docs-reconcile': Permission denied
Removing worktrees/wt-drain-concurrency: gitdir file does not exist
error: failed to delete '.git/worktrees/wt-drain-concurrency': Permission denied
Removing worktrees/wt-ffmpeg-timeouts: gitdir file does not exist
error: failed to delete '.git/worktrees/wt-ffmpeg-timeouts': Permission denied
Removing worktrees/wt-gdrive-api-safety: gitdir file does not exist
error: failed to delete '.git/worktrees/wt-gdrive-api-safety': Permission denied
Removing worktrees/wt-idempotent-process: gitdir file does not exist
error: failed to delete '.git/worktrees/wt-idempotent-process': Permission denied
Removing worktrees/wt-input-jail: gitdir file does not exist
error: failed to delete '.git/worktrees/wt-input-jail': Permission denied
Removing worktrees/wt-output-dir-config: gitdir file does not exist
error: failed to delete '.git/worktrees/wt-output-dir-config': Permission denied
Removing worktrees/wt-p1-genyaml: gitdir file does not exist
error: failed to delete '.git/worktrees/wt-p1-genyaml': Permission denied
Removing worktrees/wt-p2-upload-stream: gitdir file does not exist
error: failed to delete '.git/worktrees/wt-p2-upload-stream': Permission denied
Removing worktrees/wt-p4-md5cache: gitdir file does not exist
error: failed to delete '.git/worktrees/wt-p4-md5cache': Permission denied
Removing worktrees/wt-p5a-pollknob: gitdir file does not exist
error: failed to delete '.git/worktrees/wt-p5a-pollknob': Permission denied
Removing worktrees/wt-p6-heartbeat: gitdir file does not exist
error: failed to delete '.git/worktrees/wt-p6-heartbeat': Permission denied
Removing worktrees/wt-pr481-review: gitdir file does not exist
error: failed to delete '.git/worktrees/wt-pr481-review': Permission denied
Removing worktrees/wt-retention-sweep: gitdir file does not exist
error: failed to delete '.git/worktrees/wt-retention-sweep': Permission denied
Removing worktrees/wt-review-492-mergecheck: gitdir file does not exist
error: failed to delete '.git/worktrees/wt-review-492-mergecheck': Permission denied
Removing worktrees/wt-review-493-mergecheck: gitdir file does not exist
error: failed to delete '.git/worktrees/wt-review-493-mergecheck': Permission denied
Removing worktrees/wt-scratch-reclaim: gitdir file does not exist
error: failed to delete '.git/worktrees/wt-scratch-reclaim': Permission denied
Removing worktrees/wt-signed-download: gitdir file does not exist
error: failed to delete '.git/worktrees/wt-signed-download': Permission denied
Removing worktrees/wt-source-cache: gitdir file does not exist
error: failed to delete '.git/worktrees/wt-source-cache': Permission denied
Removing worktrees/wt-upload-dir: gitdir file does not exist
error: failed to delete '.git/worktrees/wt-upload-dir': Permission denied
Removing worktrees/wt-upload-staging: gitdir file does not exist
error: failed to delete '.git/worktrees/wt-upload-staging': Permission denied
===== worktree list =====
C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer
3be0935 [fix/cloud-run-exec-status-fastfail]
C:/Users/avidu/AppData/Local/Temp/claude/c--Users-avidu-Projects-badminton-highlight-
indexer/6e9d0958-a2ec-4a8e-bb2d-6cce9833c527/scratchpad/cfg-wt      20b2d5a [claude/wasb-
config-validation]
C:/Users/avidu/AppData/Local/Temp/claude/c--Users-avidu-Projects-badminton-highlight-
indexer/6e9d0958-a2ec-4a8e-bb2d-6cce9833c527/scratchpad/compile-wt    b09a1d4 [claude/compile-
reliability]
C:/Users/avidu/AppData/Local/Temp/claude/c--Users-avidu-Projects-badminton-highlight-
indexer/6e9d0958-a2ec-4a8e-bb2d-6cce9833c527/scratchpad/pr458-pre-wt  355dad6 (detached HEAD)
C:/Users/avidu/AppData/Local/Temp/claude/c--Users-avidu-Projects-badminton-highlight-
indexer/6e9d0958-a2ec-4a8e-bb2d-6cce9833c527/scratchpad/pr458-wt      5de1ef2 (detached HEAD)
===== branch still present? =====
    fix/worker-skip-revalidation-on-dispatch
===== dir still on disk? =====
DIR EXISTS (locked, remove manually later)

```

174. assistant (2026-07-07T15:24:58.802Z)

The `prune` revealed two things: (1) the main checkout is now on `fix/cloud-run-exec-status-fastfail` ? that's the follow-up task I filed, now actively running in the shared checkout ? and (2) `.git/worktrees/` has broad permission locks from that concurrent git activity. My worktree is already **de-registered** from `git worktree list`; only the locked directory lingers. I'll stop fighting the FS locks (a sibling session is actively using the repo) and finalize state. Let me attempt the branch delete once and refresh the index.

175. user (2026-07-07T15:25:02.210Z)

Deleted branch fix/worker-skip-revalidation-on-dispatch (was 8586c34).
BRANCH_DELETED
===== master tip (confirm #513 landed) =====
23c1868 fix(serving): worker skips redundant re-validation on the cloud dispatch path (#513)

176. user (2026-07-07T15:25:04.376Z)

The file C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\memory\pr513-worker-skip-revalidation.md has been updated successfully. (file state is current in your context ? no need to Read it back)

177. assistant (2026-07-07T15:25:19.104Z)

Local branch deleted; master confirms `23c1868` (#513)`. Merge + branch cleanup complete (only a locked directory lingers on disk ? harmless). Let me refresh the index and the follow-up notes. Let me re-read MEMORY.md (it keeps changing under concurrent sessions).

178. user (2026-07-07T15:25:19.938Z)

```
1      # Memory index
2
3      - ? **Start here (GPU-resident rollout):** [session-handoff-gpu-resident.md](session-
handoff-gpu-resident.md) ? #506 implemented, PR #507 open; next: on-box parity gate + 15-clip
golden gate.
4
5      - [Actual repo location](actual-repo-location.md) ? primary working dir is an empty
STUB; real code is under `kheksutra-guru/badminton-highlight-indexer`.
6      - [Efficient timeouts, retries & Windows gcloud SSH](efficient-timeouts-and-retries.md)
? no silent retry loops; plain `gcloud compute ssh` (plink/pscp reject `-o`, mishandle `~`);
diagnose-first, run long work detached.
7      - [GPU VM setup gotchas](gpu-vm-setup-gotchas.md) ? NVDEC libs, apt-lock-at-boot,
torch2.x/pynvc; what #501 should encode.
8      - [LGTM only when merge-ready](lgtm-only-when-merge-ready.md) ? post LGTM only when
genuinely satisfied a PR is merge-ready (verified), for all PRs.
9      - [GPU-resident modernization status](gpu-resident-modernization.md) ? #506 validated
clean; the resize must match cv2.warpAffine via grid_sample (not F.interpolate).
10     - [compute=local hosted guardrail](compute-local-hosted-guardrail.md) ? PRs #511
(backend) + #169 (SPA) fixing the "weights not found" trap, gated + pushed. ? #511 touches
orchestration.py + test_api_cloud_dispatch.py ? OVERLAPS PRs #512/#513 (merge-conflict; whoever
lands 2nd rebases).
11     - [Validation shared decode pass](validation-shared-decode-pass.md) ? validators now
share ONE decode pass (branch `perf/validation-shared-decode-pass`, d53851b, **PR #512** open);
pure grab()-through is SLOWER than seeks ? use forward-hybrid.
12     - [PR #513 worker skip-revalidation](pr513-worker-skip-revalidation.md) ? worker skips
redundant re-validation on cloud dispatch (fixes the blur-threshold CUJ); PR open, done in an
isolated worktree ? conflicts with `perf/validation-shared-decode-pass` (**PR #512**) on
orchestration.py + test_api_cloud_dispatch.py.
13
```

179. user (2026-07-07T15:25:28.646Z)

The file C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\memory\MEMORY.md has been updated successfully. (file state is current in your context ?

no need to Read it back)

180. assistant (2026-07-07T15:25:35.446Z)

Now update the follow-up + conflict notes in the #513 memory file to reflect that #513 landed first:

181. user (2026-07-07T15:25:39.040Z)

The file C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\memory\pr513-worker-skip-revalidation.md has been updated successfully. (file state is current in your context ? no need to Read it back)

182. user (2026-07-07T15:25:47.997Z)

The file C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\memory\pr513-worker-skip-revalidation.md has been updated successfully. (file state is current in your context ? no need to Read it back)

183. assistant (2026-07-07T15:26:03.074Z)

Merged and state updated.

? PR #513 merged

- **Squash-merged to `master`** as `23c1868` ? `fix(serving): worker skips redundant re-validation on the cloud dispatch path (#513)`.
- Pre-merge state verified before merging: **MERGEABLE / CLEAN**, all **11** CI checks green (full suite, ruff, mypy, golden fixtures x3 OS, wheel smoke, license/leak/link scans).
- Remote branch auto-deleted by the merge; local branch `fix/worker-skip-revalidation-on-dispatch` deleted.

State updated

- **Memory**: `pr513-worker-skip-revalidation` and the index now record **MERGED (23c1868)**.
- **Follow-up**: the crash-before-marker fast-fail task you started is running in the main checkout on branch `fix/cloud-run-exec-status-fastfail` ? noted in memory.

Two things to flag

1. **#511 and #512 now need a rebase.** #513 landed first on `backend/orchestration.py` and `tests/test_api_cloud_dispatch.py`, which both of those open PRs also touch ? so whichever of them merges next will hit a conflict and needs to rebase onto the new `master`.
2. **Worktree cleanup deferred (cosmetic).** My isolated worktree was de-registered from git and its branch deleted, but the physical directory `?/khehsutra-guru/bhi-wt-worker-reval` couldn't be deleted ? a Windows file lock, almost certainly from the concurrent sibling/follow-up git activity in the shared checkout. It's harmless clutter. Remove it when no other session is active:

```
```bash
rm -rf /c/Users/avidu/Projects/khehsutra-guru/bhi-wt-worker-reval
git -C /c/Users/avidu/Projects/khehsutra-guru/badminton-highlight-indexer worktree prune
```
```

I didn't force it because a sibling session is actively using the repo and I didn't want to risk interfering.